

FIREFLY SOFTWARE FOUNDATION



Firefly

for Rust

by Example

Event-Driven Rust Microservices
with the Firefly Framework

 **FIREFLY** · tokio + axum · Rust 1.85+

Firefly for Rust by Example



Event-Driven Rust Microservices with the Firefly Framework

Firefly Software Foundation

Copyright © 2026 Firefly Software Foundation.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this material except in compliance with the License. You may obtain a copy of the License at <https://www.apache.org/licenses/LICENSE-2.0>.

Unless required by applicable law or agreed to in writing, software and documentation distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

First edition, 2026.

All code listings in this book were written and verified against the Firefly framework for Rust, version 26.6.x, targeting Rust 1.85+ on the `tokio` + `axum` stack.

Published by the Firefly Software Foundation.

Preface

Rust gives you fearless concurrency, zero-cost abstractions, and a compiler that refuses to ship a data race. What it does not give you is *cohesion*. Every new back-office service forces the same cascade of decisions before a single line of business logic is written: which HTTP layer, which database story, how to wire dependencies, how to handle configuration, errors, correlation IDs, metrics, and graceful shutdown. **Firefly** changes that. It brings the cohesive, convention-over-configuration experience that Spring Boot gave the JVM world, rebuilt from the ground up for Rust 1.85+ on `tokio` and `axum`.

This book teaches Firefly **by doing**. You build one real application from an empty crate to a secured, observable, event-driven service — making every concept concrete before moving to the next. The code in these pages is not illustrative pseudocode: every listing was written against the real crate APIs and compiles against the framework at version 26.6.x. What you read is what actually works.

Who This Book Is For

This book is for intermediate Rust developers comfortable with `async` / `await`, traits, and the basics of HTTP services. You need no prior framework expertise — if you have built anything with `axum`, `actix`, or `sqlx`, you are well prepared.

Spring Boot, WebFlux, and Project Reactor developers will feel especially at home. Wherever Firefly mirrors a concept you already know — beans, stereotypes, declarative transactions, application events, `Mono` / `Flux` — a **Spring parity** or **Reactor parity** callout draws the parallel explicitly, so you map what you already know rather than learning from zero.

What You Will Build

Every chapter advances **Lumen**, a digital-wallet and ledger service. The journey follows a deliberate arc, one part at a time:

- **Part I — Foundations.** You scaffold the first Lumen service, wire Firefly's dependency-injection container, bind typed configuration and profiles, master the **Mono** / **Flux** reactive surface, and expose your first validated REST endpoints.
- **Part II — Modeling & Persisting.** You persist wallets through reactive repositories, model the domain with a **Money** value object and a **Wallet** aggregate, and split reads from writes with CQRS command and query handlers dispatched through a bus.
- **Part III — Event-Driven.** The aggregate raises domain events; a listener projects them; an **event-sourced ledger** rebuilds every balance by replaying its event stream; and the same events flow out to Kafka or RabbitMQ for other services.
- **Part IV — Into Microservices.** Lumen reaches beyond its own process: a typed HTTP client calls an external service, a thin experience-tier BFF composes downstream calls, and an orchestrated **transfer saga** moves money across wallets and *compensates* when a step fails.
- **Part V — Secure · Observe · Ship.** You secure the endpoints, make the service observable with metrics, tracing, and health checks, add caching, scheduling, and notifications, test the whole stack against real infrastructure, and finally ship it to production behind the **firefly** CLI.

By the last page you have a working, tested, observable, secured service — and the mental model to extend it.

How to Use This Book

Read sequentially. Each chapter builds on the one before, and the Lumen codebase grows incrementally; skipping ahead leaves gaps. The **Reactive Model** chapter is the keystone — the whole reactive surface builds on `Mono` and `Flux`, so read it before the service-building chapters.

Type every listing yourself. Reading and typing code at the same time is how the patterns stick. Resist copy-pasting until you have written each listing at least once.

Run it. Lumen really runs — `cargo run` boots the service and `cargo test` exercises it. Whenever a chapter adds a feature, start the app or the tests and watch it work. Seeing real JSON come back from a real endpoint is worth a hundred diagrams.

Conventions in Brief

Typographic and structural conventions — code-listing captions, callout types, and figure numbering — are demonstrated, with live examples, in the **Conventions** section that follows.

The Companion Code

The complete, runnable Lumen project lives in the framework's `examples/lumen` directory. It is a single, layered Firefly workspace that you grow chapter by chapter; the finished source there is the destination this book walks you to. Build it once with `cargo build`, and use it to compare your work, catch up if you fall behind, or simply run the parts you are reading about.

Conventions

This page explains the typographic and structural conventions used throughout the book.

Code Listings

Every multi-line code example carries a **language tab** in its top-left corner naming the language or file it belongs to. Inline code references within prose use `monospace` font, as in "the `#[component]` attribute registers the type with Firefly's container."

Rust

```
use firefly_core::{Core, CoreConfig};

#[tokio::main]
async fn main() -> Result<(), FireflyError> {
    let core = Core::new(CoreConfig::default()).await?;
    core.serve().await
}
```

Snippets marked `rust,ignore` or `rust,no_run` elide setup for brevity, but the API names, types, and method signatures are exactly what the crates expose.

Callouts

Five callout styles appear throughout the body:

NOTE

Notes provide supplementary context or clarify a subtlety in the main text. Worth reading, but not blocking.

TIP

Tips share a shortcut, idiom, or best practice that will save you time in real projects.

WARNING

Warnings flag a common mistake or a sharp edge that can cause hard-to-debug problems if ignored.

SPRING PARITY

Spring parity callouts map a Firefly concept directly to its Spring Boot equivalent — ideal for developers migrating from the JVM ecosystem.

REACTOR PARITY

Reactor parity callouts map Firefly's **Mono** / **Flux** surface onto Project Reactor and WebFlux, so the reactive idioms you know transfer directly.

Figures & Mapping Tables

Diagrams open each chapter as inline SVG, so they render crisply at any zoom in both the screen and print editions. Mapping tables line up a familiar concept (Reactor, Spring, Spring Data) against its Firefly spelling, so you translate by lookup rather than by guesswork.

Recap & Exercises

Each chapter closes with a **Recap** of what changed in the Lumen codebase and a set of **Exercises** that push one step further. The exercises are optional but recommended for anything you intend to apply immediately.

Contents

PART I Foundations

1. Why Firefly for Rust?	12
2. Quickstart: Zero to a Running Service	18
3. Configuration, Profiles & Secrets	25
5. Dependency Wiring & Composition	33
6. The Reactive Model — Mono & Flux	40

PART II Modeling & Persisting the Domain

7. Your First HTTP API	56
8. Persistence & Reactive Repositories	66
9. Domain-Driven Design	78
10. CQRS: Commands & Queries	86

PART III Event-Driven Architecture

11. Event-Driven Architecture & Messaging	95
12. Event Sourcing the Ledger	105

PART IV Into Microservices

13. HTTP Clients & Calling Other Services	114
15. Distributed Transactions: Sagas, Workflows & TCC	121

PART V Secure, Observe & Ship

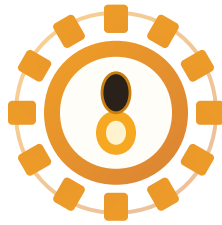
16. Security, Sessions & Identity	128
17. Observability & Health	136
18. Caching & Resilience	142
19. Scheduling, Notifications & Webhooks	149

CONTENTS

20. Declarative Services with Macros.....	154
21. Testing Firefly Applications.....	164
22. The firefly CLI.....	172
23. Extending Firefly & Going to Production.....	177

Appendices

A. Spring Boot → Firefly Cheat-Sheet.....	185
B. Crate & Module Index.....	194
Glossary.....	203



PART I

Foundations





CHAPTER 1

Why Firefly for Rust?

By the end of this chapter you will understand the problem Firefly solves, how its four tiers fit together, and why a single `Core::new` call replaces the two weeks of architectural decisions that usually precede your first handler.

The cohesion problem

Picture your first day on a new Rust microservice. Before you write one line of business logic, you face a cascade of choices. Which HTTP layer — axum, actix, warp, poem? Which database story — sqlx, SeaORM, diesel, raw `tokio-postgres`? How do you wire dependencies — a hand-rolled `AppState`, a DI crate, lazy statics? How do you handle configuration, errors, correlation IDs, metrics, graceful shutdown? Every team invents its own answer.

You assemble a bespoke stack, glue it together with good intentions, and ship. Six months later a second team starts a second service and makes entirely different choices. Now you have two codebases with incompatible conventions, different error shapes, different observability stories, and no shared understanding of how anything works.

Rust gives you infinite choice. What it does not give you is cohesion.

The stack-assembly problem is not a skills failure — it is a tooling gap. Java developers solved it with Spring Boot: one opinionated framework that makes sensible choices, lets you override what matters, and enforces a consistent idiom across every service. Firefly brings that same discipline to Rust.

What is Firefly?

Firefly is a **cohesive, reactive, async-native framework** for building production-grade Rust services. It makes the cross-cutting decisions for you — HTTP middleware, configuration, caching, CQRS, messaging, security, observability — all integrated, all consistent, with production-ready defaults from the very first `cargo run`.

Under the hood Firefly delegates to battle-tested libraries — `tokio` for the runtime, `axum` / `tower` for HTTP, `serde` for serialization, `tracing` for logging, RustCrypto for crypto — but you depend on **Firefly's ports** (object- safe `async_trait` traits), and you select concrete adapters at wiring time. Swap PostgreSQL for an in-memory store, or Kafka for RabbitMQ, without touching a single line of business logic.

Firefly's defining principles:

- **Composed, not constructed.** A single `Core::new(CoreConfig { .. })` call wires the whole infrastructure tier — middleware chain, cache, CQRS bus, event broker, health composite, metrics, scheduler, lifecycle. You write commands, queries, handlers, and routes; nothing more.

- **Symmetric across runtimes.** The wire contract, the `application/problem+json` shape, the `Idempotency-Key` semantics, the saga step definitions, the event envelopes, the HMAC webhook signatures — all identical to the Java, .NET, Go, and Python siblings.
- **Pluggable at the adapter layer.** Each integration point (cache, broker, IDP, ECM, notification channel) is an object-safe port with multiple adapter implementations selected at wiring time as an `Arc<dyn Port>`.
- **Observable by default.** `tracing` structured logging with correlation-id enrichment, actuator health/metrics endpoints, RFC 7807 error envelopes, and a startup banner are all on out of the box.
- **Reactive to the core.** A first-class `Mono` / `Flux` reactive surface — the Rust analog of Project Reactor — runs from reactive endpoints through reactive repositories, the reactive `WebClient`, and reactive EDA/CQRS.

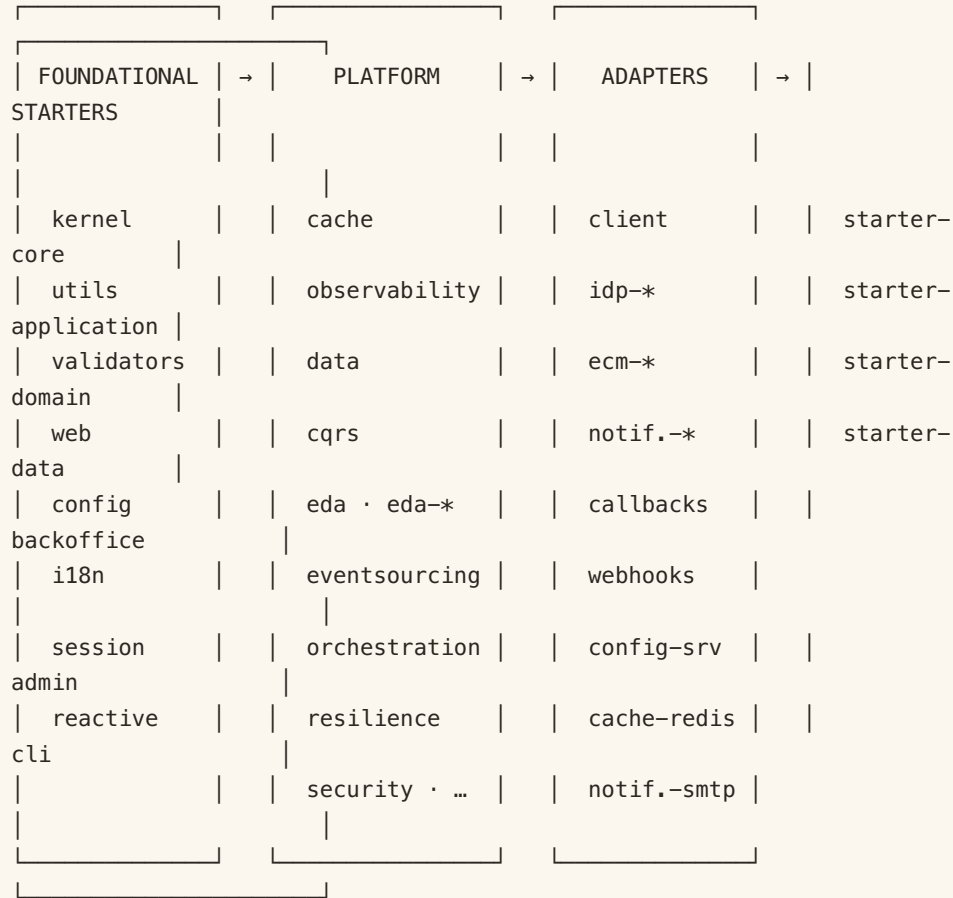
SPRING PARITY

If you come from Spring Boot, `Core::new` is your `@SpringBootApplication` auto-configuration. The `firefly.yaml` configuration hierarchy (defaults → profile → env vars) maps directly to `application.yaml` + profiles. Returning a `Mono<T>` / `Flux<T>` from a handler is exactly the WebFlux `@RestController` model. A **Spring parity** callout appears wherever the concepts align closely enough to save you the translation.

The four tiers

The framework is organised into four strictly-layered tiers, with a left-to-right dependency direction. Each tier may depend on the tiers to its left, never to its right; the Cargo crate graph enforces the layering.

Text



- **Foundational** crates are the vocabulary: `firefly-kernel` (errors, clock, correlation scopes, DDD kit), `firefly-reactive` (Mono/Flux), `firefly-web` (middleware), `firefly-config`, `firefly-validators`, `firefly-i18n`.
- **Platform** crates are the capabilities: caching, CQRS, EDA, event sourcing, orchestration, scheduling, resilience, security, observability, migrations, SSE, WebSockets.

- **Adapters** are the concrete integrations: the REST/reactive `WebClient`, the IDP vendors (Keycloak, Azure AD, Cognito), ECM (S3, Blob, e-sign), notifications (SMTP, Twilio, Firebase), and the event transports (Kafka, RabbitMQ, Postgres outbox, Redis Streams).
- **Starters** bundle a sensible default stack so a service depends on one crate. `firefly-starter-core` is the one most services begin with.

For the full per-crate catalogue see the [Module Index](#).

Choosing your tier and adapters

Start from a **starter** and add only the adapters you need.

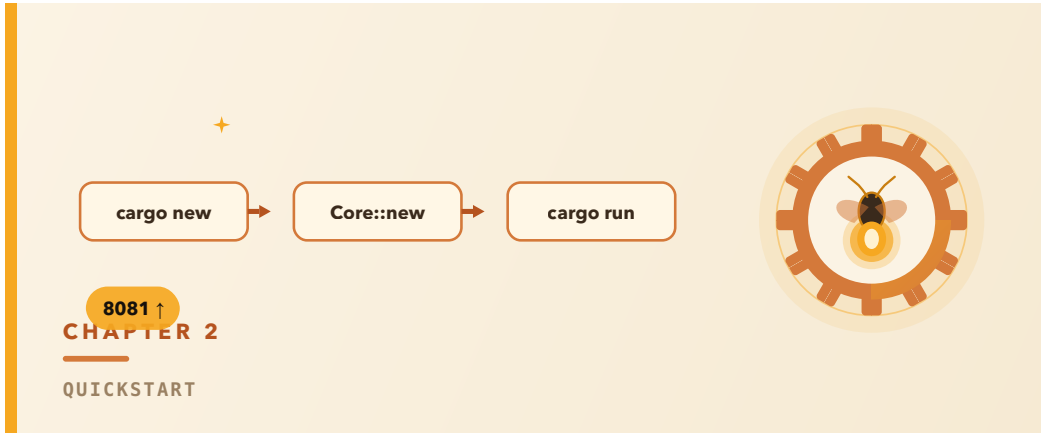
- **Default, zero infrastructure.** `firefly-starter-core` boots with the in-process `MemoryAdapter` cache and `InMemoryBroker` event bus. Nothing external is required — a service runs against pure-Rust defaults.
- **Pick a cache backend.** Drop in `firefly-cache-redis` (`RedisAdapter`) wherever an `Arc<dyn cache::Adapter>` is expected.
- **Pick an event transport.** `firefly-eda-kafka`, `-rabbitmq`, `-postgres` (durable outbox), or `-redis` (Streams) each implement the same `Broker` port; swap the constructor, keep your handlers.
- **Pick vendors.** Code against the parent-port trait (`notifications::Channel`, `idp::Adapter`, `ecm::ContentStore`) and pull in the concrete adapter crate at wiring time, so heavy SDKs stay out of services that do not use them.

What you will build

Throughout the book the running examples revolve around an **Orders** service and a small **banking ledger**: a REST API that records orders, a reactive endpoint that streams them, CQRS handlers that mutate state, domain events that fan out over a

broker, a saga that coordinates a transfer, and the observability, security, and testing that make it production-ready. Starting from a well-structured skeleton saves you the pain of retrofitting architecture later — and the `firefly CLI` scaffolds that skeleton for you.

The next chapter gets you running. Turn to the [Quickstart](#).



CHAPTER 2

Quickstart: Zero to a Running Service

This chapter gets you from an empty directory to a running service with an idempotent REST endpoint, a streaming reactive endpoint, and a live actuator — in a few minutes. Two paths: the `firefly` CLI (fastest) or plain `cargo`.

Prerequisites

Shell

```
rustc --version    # 1.85 or later (edition 2021)
cargo --version
```

That is all you need. The default stack requires **no external infrastructure** — the in-process cache and event broker are pure Rust.

Path A – scaffold with the **firefly** CLI

Install the developer CLI once, then scaffold a web-api project:

Shell

```
cargo install --path crates/cli      # from a checkout of the framework
# or: cargo install firefly-cli

firefly new orders --archetype web-api --features web,cqrs --git
cd orders
cargo run
```

firefly new generates a workspace-less Cargo crate with a **src/** tree, a **firefly.yaml**, a **.gitignore**, a **README.md**, a **Dockerfile**, and a **tests/** directory. See [The CLI](#) for every archetype and generator.

TIP

Run **firefly new --list** to see every archetype (**core**, **web-api**, **web**, **hexagonal**, **library**, **cli**) and feature flag, or **firefly new svc --dry-run** to preview the plan without writing files.

Path B – start from cargo

Create a binary crate and add the starter plus axum and tokio:

Shell

```
cargo new orders
cd orders
```

TOML

```
# Cargo.toml
[dependencies]
firefly-starter-core = "26.6.3"
firefly-reactive = "26.6.3"
firefly-web = "26.6.3"
axum = "0.7"
tokio = { version = "1", features = ["rt-multi-thread", "macros", "net",
"signal"] }
serde_json = "1"
```

Prefer the one-dependency front door? Replace the three `firefly-*` lines with a single `firefly = "26.6.3"` and `use firefly::prelude::*`; . See the [Declarative Services with Macros](#) chapter.

A running service in one file

Replace `src/main.rs` with the following. One `Core::new` wires the problem renderer, correlation propagation, idempotency replay, cache, CQRS bus, event broker, health, metrics, and scheduler. We mount a plain JSON route, a reactive `Mono` route, and a streaming `Flux` (NDJSON) route — then serve the public API and the actuator on separate ports with signal-aware graceful shutdown.

Rust

```

use axum::{routing::get, Router};
use firefly_reactive::{Flux, Mono};
use firefly_starter_core::{Core, CoreConfig};
use firefly_web::{MonoJson, NdJson};

// A plain JSON handler.
async fn list_orders() -> &'static str {
    "[]"
}

// A reactive handler: resolve a Mono to 200 application/json
// (Ok(None) -> 404 problem+json, Err -> that error's RFC 7807 response).
async fn one_order() -> MonoJson<serde_json::Value> {
    MonoJson(Mono::just(serde_json::json!({ "id": "o1", "customer": "alice" })))
}

// A streaming handler: each Flux element becomes one application/x-ndjson
// line, flushed incrementally with real backpressure.
async fn stream_orders() -> NdJson<i64> {
    NdJson(Flux::range(1, 3))
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let core = Core::new(CoreConfig {
        app_name: "orders".into(),
        app_version: "1.0.0".into(),
        ..CoreConfig::default()
    });
    core.init_logging()?;
    core.print_banner();

    let api = core.apply_middleware(
        Router::new()
            .route("/orders", get(list_orders))
            .route("/orders/one", get(one_order))
            .route("/orders/stream", get(stream_orders)),
    );
}

```

```

let admin = core.actuator_router(Vec::new());

let app = core
    .new_application()
    .on_server("api", move |shutdown| async move {
        let listener = tokio::net::TcpListener::bind("0.0.0.0:8080").await?;
        axum::serve(listener, api)
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    })
    .on_server("admin", move |shutdown| async move {
        let listener = tokio::net::TcpListener::bind("0.0.0.0:8081").await?;
        axum::serve(listener, admin)
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    });

app.run().await?; // blocks until ctrl-c / SIGTERM
Ok(())
}

```

Shell

```
cargo run
```

What you just got

Hit the endpoints:

Shell

```

# Plain JSON.
curl localhost:8080/orders
# []

# Reactive Mono -> 200 application/json.
curl localhost:8080/orders/one
# {"id":"o1","customer":"alice"}

# Reactive Flux -> streamed application/x-ndjson, one line per element.
curl -N localhost:8080/orders/stream
# 1
# 2
# 3

# Actuator health, info, metrics - on the admin port.
curl localhost:8081/actuator/health
# {"status":"UP", ...}

```

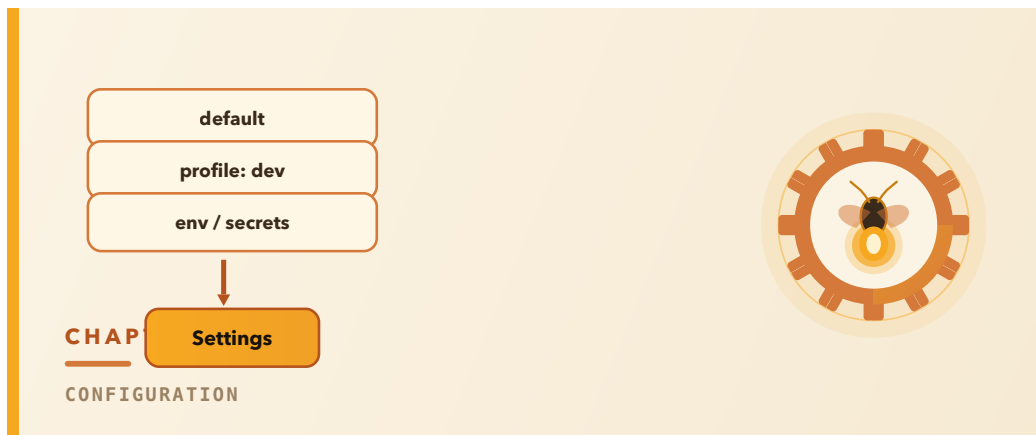
Without writing any of it yourself, the service already has:

- **RFC 7807 errors.** Any handler error renders as `application/problem+json`, and a panic is caught and rendered as a 500 problem.
- **Correlation IDs.** Every response echoes an `X-Correlation-Id`; an incoming one is honoured and scoped through the whole request.
- **Idempotency.** Every `POST` / `PUT` / `PATCH` carrying an `Idempotency-Key` header is recorded; repeating the request replays the stored response with `Idempotent-Replay: true`, and reusing the key with a different body is a 409.
- **A reactive surface.** `Mono` / `Flux` drop straight into axum handlers via the `MonoJson` / `NdJson` / `Sse` responders, with true streaming and backpressure.
- **A management surface.** `/actuator/{health,info,metrics,env,tasks,version}` on a separate listener so it never leaks onto the public network.

- **Graceful shutdown.** `app.run()` traps SIGINT/SIGTERM and drains in-flight requests before exiting.

Where to go next

- The reactive responders you just used are explained in depth in [The Reactive Model](#) — read it next.
- Add typed configuration in [Configuration](#).
- Build out real handlers and routes in [Your First HTTP API](#) and [CQRS](#).



CHAPTER 3

Configuration, Profiles & Secrets

`firefly-config` brings Spring Boot–style typed, layered configuration binding to Rust. You declare a `serde`-deserializable struct, call `load` / `load_from_profile`, and the loader merges sources in precedence order, resolves the active profile, resolves `$ {...}` placeholders, and binds the flat dot-keyed map onto your struct.

🌿 SPRING PARITY

This is `@ConfigurationProperties` + the `application.yaml` → profile → environment hierarchy, re-expressed for Rust.

Defining configuration

A configuration struct is plain `serde`. Nested structs become nested dot-keyed sections (`web.port`, `cache.adapter`):

Rust

```

use serde::Deserialize;

#[derive(Debug, Deserialize)]
struct Web {
    port: u16,
}

#[derive(Debug, Deserialize)]
struct Cache {
    adapter: String,
    ttl: i64,
}

#[derive(Debug, Deserialize)]
struct AppCfg {
    web: Web,
    cache: Cache,
    tags: Vec<String>,
}

```

The binder is **type-driven**: `"9090"` binds onto a `u16`, `"alpha,beta"` splits onto a `Vec<String>`, `"true"` parses onto a `bool`, and missing keys produce zero values — so plain `#[derive(Deserialize)]` structs bind without `#[serde(default)]`.

Loading with profiles

The canonical helper reads `application.yaml`, then the profile-specific `application-{profile}.yaml`, then `FIREFLY_*` environment variables:

Rust

```

use firefly_config::{load_from_profile, ConfigError};

fn main() -> Result<(), ConfigError> {
    // dir, app basename, fallback profile (FIREFLY_PROFILE overrides).
    let cfg: AppCfg = load_from_profile("/etc/orders", "application", "dev")?;
    println!("listening on {:?}", cfg.web.port);
    Ok(())
}

```

`FIREFLY_PROFILE` selects the profile file at runtime — `FIREFLY_PROFILE=prod` reads `application-prod.yaml`. A comma-separated value (`FIREFLY_PROFILE=dev,cloud`) overlays one file per profile in order.

Source precedence

`Layered::new(vec![s1, s2, ...])` merges from left to right — last write wins. The canonical chain is:

Order	Source	Beats
1	Defaults — <code>StaticSource::new(name, entries)</code>	nothing
2	Base YAML — <code>from_optional_yaml("application.yaml")</code>	defaults
3	Profile YAML — <code>from_optional_yaml("application-prod.yaml")</code>	base
4	Environment — <code>from_env("FIREFLY")</code>	YAML files
5	CLI flags — <code>FlagSource::new().set("web.port", "9090")</code>	everything

So an environment override (`FIREFLY_WEB_PORT=9090`) always beats a YAML file, and a CLI override always beats both. Build the chain explicitly when you need the full control:

Rust

```
use firefly_config::{from_env, from_optional_yaml, load, Source, StaticSource};

let sources: Vec<Box<dyn Source>> = vec![
    Box::new(StaticSource::new("defaults", [("web.port".into(),
"8080".into())])),
    Box::new(from_optional_yaml("application.yaml")),
    Box::new(from_env("FIREFLY")),
];
let cfg: AppCfg = load(&sources)?;
```

YAML subset and value rules

Files are parsed by a line-by-line YAML-subset scanner (no general-purpose YAML dependency), so the flattened output is identical across the Java/Go/Rust ports for any given `application.yaml`:

YAML

```
web:
  port: 8080
cache:
  adapter: memory
  ttl: 60000
tags: alpha, beta, gamma  # sequences of scalars are comma-joined
```

- nested mappings become dot-joined, lower-cased keys;
- scalar lexemes are preserved verbatim until the binder parses them against the target field type;
- duplicate keys follow last-write-wins;
- aliases, anchors, multi-doc, tags, and flow sequences are deliberately not interpreted.

Supported leaf kinds: `String`, `bool` (Go `ParseBool` syntax), every integer width, `f32` / `f64`, `char`, unit enums (by variant name), `Option<T>`, sequences of scalars, and `HashMap<String, _>` subtrees. For durations, use an `i64` field plus a conversion: `Duration::from_millis(cfg.cache.ttl as u64)`.

Placeholders

`load` / `bind` run a post-merge pass resolving `${...}` placeholders in values (also exposed standalone as `resolve_placeholders(&flat)`):

YAML

```
app:
  name: orders
datasource:
  url: ${DATABASE_URL:postgres://localhost/orders} # env, else default
  pool: ${app.name}-pool # config reference
```

- `${ENV_VAR}` — a literal environment variable;
- `${app.name}` — a config reference, resolved recursively with a depth-10 guard against cycles;
- `${key:default}` — a fallback when neither environment nor config resolves `key`;
- **environment beats config**: `${app.name}` honours `FIREFLY_APP_NAME` before the merged map.

An unresolvable placeholder without a default raises `ConfigError::Placeholder`.

NOTE

Keys are normalized kebab ↔ snake, so `graceful-timeout:` in YAML binds a `graceful_timeout` serde field.

Runtime reload – the `/actuator/refresh` contract

`ReloadableConfig<T>` replays the full merge → placeholder-resolution → bind pipeline and atomically swaps the snapshot; a failed reload keeps the previous snapshot. This is the hook a `POST /actuator/refresh` endpoint wires up.

Rust

```
use firefly_config::ReloadableConfig;

let cfg: ReloadableConfig<AppCfg> = ReloadableConfig::load(sources)?;
let snapshot = cfg.get();           // Arc<AppCfg>
let mut rx = cfg.subscribe();       // tokio watch receiver
let changed: Vec<String> = cfg.reload()?; // sorted, changed top-level keys
```

`Arc<ReloadableConfig<T>>` coerces to `Arc<dyn Refresher>` — the object-safe trait the actuator refresh endpoint depends on.

Property-source introspection and masking

`Layered::property_sources()` returns ordered, origin-attributed `PropertySourceView`s (highest precedence first, Spring `/actuator/env` style), with secrets masked: keys naming secrets (`password`, `secret`, `token`, `credential`, `*key`) mask as `*****`, and URI userinfo passwords are redacted (`postgresql://user:*****@host`). The `mask` module exposes `mask_value`, `is_sensitive_key`, and `sanitize_uri` directly.

Profile expressions

`accepts_profiles(&active, &exprs)` evaluates the Spring Boot 2.4+ profile-expression grammar against an active-profile list:

Rust

```
use firefly_config::{accepts_profiles, active_profiles};

let active = active_profiles("dev");           // e.g. ["prod", "cloud"]
accepts_profiles(&active, &["prod & cloud"]); // AND
accepts_profiles(&active, &["prod | qa"]);      // OR
accepts_profiles(&active, &["!test"]);          // negation
accepts_profiles(&active, &["(prod & cloud) | qa"]); // grouping
```

It returns `true` when any expression matches; a malformed expression evaluates to `false` (it never panics).

In-process application events

`ApplicationEventBus` is a synchronous, in-process, `TypeId`-dispatched, `@order`-sorted pub/sub — Spring's `ApplicationEventPublisher` model. This is distinct from the asynchronous `firefly-eda` broker (no transport, no topics; listeners run on the publishing thread):

Rust

```
use firefly_config::{ApplicationEventBus, ApplicationReadyEvent};

let bus = ApplicationEventBus::new();
bus.subscribe::<ApplicationReadyEvent, _>(|_e| { /* on ready */ });
bus.publish(&ApplicationReadyEvent);
```

Lifecycle events ship: `ContextRefreshedEvent`, `ApplicationReadyEvent`, `ContextClosedEvent`. Any `'static` type can be published as a domain event.

Pulling config from a config server

`ConfigClient` fetches a Spring-Cloud-Config document and flattens it into a `StaticSource` you slot into your chain above the defaults:

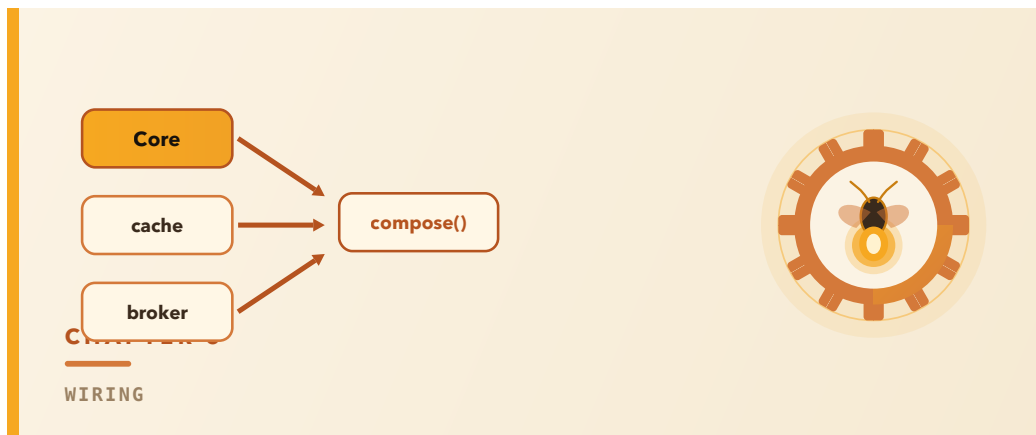
Rust

```
use firefly_config::ConfigClient;

let remote = ConfigClient::new("http://config:8888", "orders")
    .with_profile("prod")
    .with_label("main")
    .with_basic_auth("user", "pass")
    .fetch_source()           // fail-fast; .fetch_source_or_empty() = soft
                              fallback
    .await?;
sources.insert(1, Box::new(remote)); // above defaults, below env/flags
```

A non-2xx response from the server logs a warning and yields an empty map (soft miss); transport or decode failures raise `ConfigError::Remote`. The standalone config server itself lives in `firefly-config-server`.

The next chapter covers how to wire your application's components together — both explicitly and with the optional DI container. See [Dependency Wiring](#).



CHAPTER 5

Dependency Wiring & Composition

Firefly's default style is **explicit construction**: you build your components with plain constructors and pass them where they are needed, exactly as you would in idiomatic Rust. For teams that want a Spring-style service locator, `firefly-container` provides an opt-in DI container. This chapter covers both, and when to reach for each.

Explicit construction (the default)

Most services never need a container. You build the infrastructure with `Core::new`, build your own services with constructors, share them as `Arc<T>`, and capture them in your handler closures.

Rust

```

use std::sync::Arc;
use firefly_starter_core::{Core, CoreConfig};

// Your own service, built by hand.
struct OrderService {
    bus: Arc<firefly_cqrs::Bus>,
    cache: Arc<dyn firefly_cache::Adapter>,
}

impl OrderService {
    fn new(bus: Arc<firefly_cqrs::Bus>, cache: Arc<dyn firefly_cache::Adapter>)
    -> Self {
        Self { bus, cache }
    }
}

let core = Core::new(CoreConfig { app_name:
"orders".into(), ..Default::default() });

// Reuse the Core's wired components – no duplication, no globals.
let svc = Arc::new(OrderService::new(Arc::clone(&core.bus),
Arc::clone(&core.cache)));

```

Because every Firefly port is an object-safe trait, "depend on the interface, inject the implementation" is just an `Arc<dyn Trait>` field:

Rust

```

struct NotificationSink {
    channel: Arc<dyn firefly_notifications::Channel>, // swap at construction
}

```

TIP

Explicit wiring keeps the dependency graph visible in code and checked by the compiler. There is no reflection and no startup-time magic; if it compiles, it is wired. Prefer this for application code.

The **Core** as a composition root

Core is a wired bundle of the components a typical service needs. You read them straight off the struct, or via accessors that a downstream admin dashboard can also consume:

Field / accessor	Type
<code>core.bus</code>	<code>Arc<cqrs::Bus></code> (validation pre-installed)
<code>core.cache</code>	<code>Arc<dyn cache::Adapter></code> (Memory by default)
<code>core.broker</code>	<code>Arc<dyn eda::Broker></code> (InMemory by default)
<code>core.scheduler</code>	<code>Arc<scheduling::Scheduler></code>
<code>core.metrics</code>	<code>Arc<actuator::MetricRegistry></code>
<code>core.health</code>	<code>Arc<actuator::HealthComposite></code>
<code>core.health_composite()</code>	accessor for the same

You can override any of them in `CoreConfig` — drop in a Redis cache, a Kafka broker, your own bus — and everything downstream uses your choice:

Rust

```

use std::sync::Arc;
use firefly_starter_core::{Core, CoreConfig};

let core = Core::new(CoreConfig {
    app_name: "orders".into(),
    cache: Some(Arc::new(my_redis_adapter)), // Arc<dyn cache::Adapter>
    broker: Some(Arc::new(my_kafka_broker)), // Arc<dyn eda::Broker>
    ..CoreConfig::default()
});

```

The opt-in container – **firefly-container**

When you want a Spring-style **service locator** — register everything once, resolve by type, support scopes and trait-object bindings — reach for **firefly-container**. It is **TypeId**-keyed, **Send + Sync**, and shareable as **Arc<Container>**.

Rust

```

use firefly_container::{Container, Scope};
use std::sync::Arc;

struct Greeter;
impl Greeter {
    fn greet(&self) -> &'static str { "hello" }
}
struct UserService {
    greeter: Arc<Greeter>,
}

let c = Container::new();
c.register_factory::<Greeter, _>(Scope::Singleton, |_| Ok(Greeter));
c.register_factory::<UserService, _>(Scope::Singleton, |c| {
    Ok(UserService { greeter: c.resolve::<Greeter>()? })
});

let svc = c.resolve::<UserService>().unwrap();
assert_eq!(svc.greeter.greet(), "hello");

```

A factory closure resolves its own dependencies by calling `resolve` — the Rust analog of constructor injection. There is no reflective autowiring (Rust has no runtime reflection), so you wire dependencies explicitly inside the closure.

Scopes

`Scope` controls instance lifecycle:

Scope	Behaviour
<code>Scope::Singleton</code>	One instance, cached after first resolve
<code>Scope::Prototype</code>	A fresh instance on every <code>resolve</code>
<code>Scope::Request</code>	One per request (backed by <code>register_request_scope</code>)
<code>Scope::Session</code>	One per session (backed by <code>register_session_scope</code>)
custom	Your own <code>ScopeHandler</code> via <code>register_scope</code>

Trait-object bindings

Bind an interface to an implementation, then resolve the trait object. This is how you express "depend on the port, get the adapter":

Rust

```
use firefly_container::{Container, Scope};

trait Clock: Send + Sync { fn now(&self) -> u64; }
struct SystemClock;
impl Clock for SystemClock { fn now(&self) -> u64 { 0 } }

let c = Container::new();
c.register_factory::<SystemClock, _>(Scope::Singleton, |_| Ok(SystemClock));
c.bind::<dyn Clock, SystemClock>(|impl_arc| impl_arc); // view Arc<SystemClock>
as Arc<dyn Clock>

let clock = c.resolve::<dyn Clock>().unwrap();
```

`primary` / `order` metadata (`register_factory_with`) and `resolve_all::<dyn Trait>()` let you register many implementations and select or list them, exactly as Spring's `@Primary` / `@Order` and `List<T>` injection do.

Errors and introspection

The error taxonomy mirrors Spring's: `ContainerError::NoSuchBean`, `NoUniqueBean`, `CircularDependency`. Circular dependencies are detected with a thread-local resolution stack. For diagnostics, `fuzzy_suggestions(name)` returns similar registered names, `registered_types()` lists everything, and `bean_metrics::<T>()` exposes per-bean resolution stats.

NOTE

The container is **opt-in**. None of the Go-parity core or the starters require it; it exists for teams that prefer a central registry. For most application code, explicit construction (above) is simpler, faster to compile, and fully checked by the borrow checker.

Aspect-oriented advice – `firefly-aop`

For cross-cutting behaviour (timing, auditing, retry) without threading it through every call, `firefly-aop` ports Spring's aspect model: a `Pointcut` glob matcher, a `JoinPoint`, an `Aspect` with five hooks (`before` / `after` / `after_returning` / `after_throwing` / `around`), and an `intercept` chain executor with `around` / `Proceed`. Weaving is explicit at the call site (there is no class-path proxying), which keeps the control flow visible.

With the wiring in place, you are ready to build. The reactive model underpins everything that follows — read [The Reactive Model](#) next.



CHAPTER 6

The Reactive Model – Mono & Flux

This is the keystone chapter. `firefly-reactive` is a faithful, production-grade **Reactor / WebFlux-style reactive core** — the Rust analog of Project Reactor's `Mono` and `Flux`, the engine behind Spring WebFlux and the Java Firefly framework. Every reactive surface in the framework — reactive HTTP endpoints, reactive repositories, the reactive `WebClient`, reactive EDA and CQRS — is built on the two types you will learn here. Read this before the service-building chapters.

Mono and Flux

Two publishers, mirroring Reactor exactly:

- `Mono<T>` — a producer of *at most one* value (0-or-1, plus a terminal error). The reactive analog of "an async function that returns a `T`."
- `Flux<T>` — a producer of *0..N* values plus a terminal completion-or-error. The reactive analog of "an async stream of `T`."

Both are **lazy**: building a pipeline does nothing; work runs only when you subscribe, block, or await. Both are `Send + 'static'`, so a `Mono` or `Flux` drops directly into an axum handler.

The error type is fixed to `firefly_kernel::FireflyError`, exactly as WebFlux models everything as a `Throwable`. Fixing the error keeps the operator surface ergonomic (no error type parameter) and wires straight into the framework's RFC 7807 problem responses.

Rust

```
use firefly_reactive::{Flux, Mono};

# async fn ex() {
  // Mono: one value, lazily transformed, then awaited.
  let n = Mono::just(20)
    .map(|x| x + 1)
    .filter(|x| *x > 10)
    .default_if_empty(0)
    .block()
    .await
    .unwrap();
  assert_eq!(n, Some(21));

  // Flux: a stream of values, filtered + mapped, collected to a Vec.
  let xs = Flux::range(1, 5)
    .filter(|x| x % 2 == 1)
    .map(|x| x * 10)
    .collect_list()
    .block()
    .await
    .unwrap() // Result -> Option
    .unwrap(); // Option -> Vec (collect_list always yields a list)
  assert_eq!(xs, vec![10, 30, 50]);
# }
```

 REACTOR PARITY

`Mono::block()` here is *not* the thread-parking `Mono.block()` of the JVM. It is `async` and never parks a Tokio worker; it resolves the publisher in place and returns `Result<Option<T>, FireflyError>`.

Reactor → firefly-reactive concept map

Project Reactor	firefly-reactive
<code>Mono<T></code>	<code>Mono<T></code>
<code>Flux<T></code>	<code>Flux<T></code>
<code>Throwable</code> (error signal)	<code>firefly_kernel::FireflyError</code> (fixed)
<code>Mono.empty()</code> / <code>onComplete</code>	<code>Ok(None)</code> from a <code>Mono</code>
<code>onError(t)</code>	<code>Err(FireflyError)</code> (terminal)
<code>Mono.block()</code>	<code>Mono::block</code> (async — never parks a thread)
<code>Mono.subscribe(..)</code> / <code>Flux.subscribe(..)</code>	<code>Mono::subscribe</code> / <code>Flux::subscribe</code>
<code>Schedulers.immediate()</code>	<code>Scheduler::Immediate</code>
<code>Schedulers.parallel()</code>	<code>Scheduler::Parallel</code>
<code>Schedulers.boundedElastic()</code>	<code>Scheduler::BoundedElastic</code>
<code>subscribeOn</code> / <code>publishOn</code>	<code>subscribe_on</code> / <code>publish_on</code>
<code>FluxSink</code> / <code>Flux.create</code>	<code>FluxSink</code> / <code>Flux::create</code>
<code>Retry.backoff(..)</code>	<code>Backoff</code> + <code>*::retry_backoff</code>
<code>Mono.toFuture()</code> / <code>await</code>	<code>Mono::into_future</code> / <code>.await</code>
<code>Flux.toStream()</code> (escape hatch)	<code>Flux::to_stream</code> / <code>Flux::into_stream</code>

Creating publishers

Mono constructors:

Constructor	Produces
<code>Mono::just(v)</code>	exactly <code>v</code>
<code>Mono::just_or_empty(opt)</code>	<code>v</code> if <code>Some</code> , empty if <code>None</code>
<code>Mono::empty()</code>	completes with no value (<code>Ok(None)</code>)
<code>Mono::error(e)</code>	terminal error
<code>Mono::from_future(fut)</code>	awaits a <code>Future<Output = T></code>
<code>Mono::from_result_future(fut)</code>	awaits a <code>Future<Output = Result<T, _>></code>
<code>Mono::from_callable(f)</code>	runs a <code>FnOnce() -> Result<Option<T>, _></code> on subscribe
<code>Mono::defer(factory)</code>	builds the <code>Mono</code> fresh per subscription

Flux constructors:

Constructor	Produces
<code>Flux::just(vec)</code>	each element of the <code>Vec</code>
<code>Flux::from_iter(iter)</code>	each element of an iterator
<code>Flux::range(start, count)</code>	<code>start, start+1, ...</code> (count items)
<code>Flux::empty()</code> / <code>Flux::never()</code>	completes immediately / never emits
<code>Flux::error(e)</code>	terminal error
<code>Flux::from_stream(s)</code>	a <code>Stream<Item = Result<T, FireflyError>></code>
<code>Flux::from_value_stream(s)</code>	a <code>Stream<Item = T></code>
<code>Flux::create(producer)</code>	imperative push via a <code>FluxSink</code> (see below)
<code>Flux::interval(period)</code>	<code>0, 1, 2, ...</code> on a timer
<code>Flux::generate(seed, step)</code>	stateful generation

Operator quick reference

The operator surface mirrors Reactor. `Mono` and `Flux` share most names; the differences reflect cardinality.

Category	Mono	Flux
transform	<code>map</code> <code>map_async</code> <code>flat_map</code> <code>flat_map_many</code> <code>filter</code>	<code>map</code> <code>map_async</code> <code>flat_map(n)</code> <code>concat_map</code> <code>filter</code> <code>scan</code> <code>index</code> <code>flat_map_iterable</code>
reduce/term	<code>then</code> <code>then_return</code> <code>zip_with</code>	<code>reduce</code> <code>collect_list</code> <code>collect_map</code> <code>count</code> <code>all</code> <code>any</code> <code>then</code> <code>last</code> <code>next</code> <code>single</code> <code>element_at</code>
limit/slice	—	<code>take</code> <code>take_while</code> <code>take_last</code> <code>skip</code> <code>skip_while</code> <code>distinct</code> <code>distinct_until_changed</code>
combine	<code>when</code> <code>zip</code>	<code>merge_with</code> <code>concat_with</code> <code>zip_with</code> <code>combine_latest</code> <code>start_with</code> <code>switch_if_empty</code> <code>default_if_empty</code>
error	<code>on_error_return</code> <code>on_error_resume</code> <code>on_error_map</code> <code>retry</code> <code>retry_backoff</code>	<code>on_error_resume</code> <code>on_error_continue</code> <code>retry</code> <code>retry_backoff</code>
time	<code>timeout</code> <code>delay_element</code>	<code>timeout</code> <code>delay_elements</code> <code>sample</code> <code>debounce</code> <code>interval</code>
backpressure	—	<code>on_backpressure_buffer</code> <code>on_backpressure_drop</code> <code>on_backpressure_latest</code> <code>limit_rate</code>
window	—	<code>buffer</code> <code>window</code> <code>group_by</code>
side-effect	<code>do_on_next</code> <code>do_on_success</code> <code>do_on_error</code> <code>do_on_finally</code>	<code>do_on_next</code> <code>do_on_complete</code> <code>do_on_error</code> <code>do_on_finally</code>
schedule	<code>subscribe_on</code> <code>publish_on</code>	<code>subscribe_on</code> <code>publish_on</code>
cache/view	<code>cache</code> <code>as_flux</code>	—

Transforming and chaining

Rust

```
use firefly_reactive::{Flux, Mono};

# async fn ex() {
    // flat_map: chain a Mono onto the result of another (sequential dependency).
    let total = Mono::just(3)
        .flat_map(|seed| Mono::just(seed * 10))
        .map(|x| x + 1)
        .block()
        .await
        .unwrap();
    assert_eq!(total, Some(31));

    // flat_map on a Flux runs up to N inner publishers concurrently.
    let doubled = Flux::range(1, 3)
        .flat_map(2, |n| Mono::just(n * 2).as_flux())
        .collect_list()
        .block()
        .await
        .unwrap()
        .unwrap();
    assert_eq!(doubled.len(), 3);
# }
```

Combining

Rust

```
use firefly_reactive::{zip, Mono};

# async fn ex() {
  // zip two Monos into a tuple – both run, then combine.
  let pair = zip(Mono::just("alice"), Mono::just(42))
    .block()
    .await
    .unwrap();
  assert_eq!(pair, Some(("alice", 42)));
# }
```

Error semantics

An **Err** item is **terminal** in a **Flux**: every operator short-circuits on the first error and propagates it downstream — there is no per-element error channel. To recover:

- **Mono::on_error_return(fallback)** — substitute a value;
- **Mono::on_error_resume(f)** / **Flux::on_error_resume(f)** — switch to a fallback publisher, keeping items emitted before the error;
- **Flux::on_error_continue(handler)** — drop the failing element and keep the rest (for operators that re-signal per item);
- **Mono::on_error_map(f)** — translate the error.

retry and **retry_backoff** re-subscribe to a **factory closure**, since Rust streams and futures are single-use:

Rust

```

use std::time::Duration;
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use firefly_reactive::{Backoff, Mono};
use firefly_kernel::FireflyError;

# async fn ex() {
let calls = Arc::new(AtomicUsize::new(0));
let c = calls.clone();
let value = Mono::retry_backoff(
    move || {
        let c = c.clone();
        Mono::from_callable(move || {
            let n = c.fetch_add(1, Ordering::SeqCst);
            if n < 2 { Err(FireflyError::internal("flaky")) } else {
Ok(Some(n)) }
        })
    },
    Backoff::new(5, Duration::from_millis(10)),
)
.block()
.await
.unwrap();
assert_eq!(value, Some(2));
# }

```

`Mono::timeout` / `Flux::timeout` map a missed deadline to a 504 `FireflyError` (code `REACTIVE_TIMEOUT`), which renders as an RFC 7807 problem response.

Imperative emission with `FluxSink`

When values arrive from a callback or an imperative loop, push them into a `Flux` with `Flux::create`:

Rust

```
use firefly_reactive::Flux;

# async fn ex() {
let flux = Flux::create(|sink| {
    for i in 1..=3 {
        sink.next(i);
    }
    sink.complete();
});
let out = flux.collect_list().block().await.unwrap();
assert_eq!(out, Some(vec![1, 2, 3]));
# }
```

Schedulers

A `Scheduler` decides *where* work runs. `subscribe_on` hops the source onto a scheduler; `publish_on` switches the thread for everything downstream.

Rust

```
use firefly_reactive::{Flux, Scheduler};

# async fn ex() {
let out = Flux::range(1, 3)
    .subscribe_on(Scheduler::Parallel) // run the source on the Tokio worker
    pool
    .map(|x| x * 2)
    .collect_list()
    .block()
    .await
    .unwrap();
assert_eq!(out, Some(vec![2, 4, 6]));
# }
```

`Scheduler::Immediate` runs inline, `Scheduler::Parallel` uses the Tokio worker pool for CPU-bound work, and `Scheduler::BoundedElastic` is for blocking calls.

Reactive HTTP endpoints

`firefly-web` ships responders that turn a `Mono` / `Flux` into an axum response — the Rust analog of returning `Mono<T>` / `Flux<T>` from a WebFlux `@RestController`. They reuse `firefly-sse`'s wire format, so every reactive response is byte-compatible across the ports.

Spring WebFlux	firefly-web
<code>Mono<T></code> handler return	<code>MonoJson(Mono<T>)</code>
<code>Mono<T></code> empty → <code>404</code>	<code>Ok(None)</code> → 404 problem+json
<code>Mono<T></code> error → problem	<code>Err(FireflyError)</code> → that error's RFC 7807 response
<code>Flux<T></code> + <code>APPLICATION_NDJSON_VALUE</code>	<code>NdJson(Flux<T>)</code>
<code>Flux<ServerSentEvent<T>></code>	<code>Sse(Flux<T>)</code> / <code>SseEvents(Flux<Event>)</code>

Rust

```

use axum::{routing::get, response::IntoResponse, Router};
use firefly_reactive::{Flux, Mono};
use firefly_web::{MonoJson, NdJson, Sse};

async fn one_order() -> impl IntoResponse {
    // Ok(Some) -> 200 application/json; Ok(None) -> 404 problem+json;
    // Err -> that error's problem response.
    MonoJson(Mono::just(serde_json::json!({ "id": "o1" })))
}

async fn stream_orders() -> impl IntoResponse {
    // application/x-ndjson, one line per element, backpressured.
    NdJson(Flux::just(vec![1, 2, 3]))
}

async fn live_orders() -> impl IntoResponse {
    // text/event-stream, one `data:` frame per element.
    Sse(Flux::just(vec![1, 2, 3]))
}

let app: Router = Router::new()
    .route("/orders/one", get(one_order))
    .route("/orders", get(stream_orders))
    .route("/orders/live", get(live_orders));

```

The responders, precisely:

- **MonoJson(Mono<T>)** resolves the **Mono** : **Ok(Some)** → **200 application/json** ; **Ok(None)** → **404 application/problem+json** ; **Err** → that error's problem response.
- **NdJson(Flux<T>)** streams **application/x-ndjson** — one compact JSON doc + **'\n'** per element, flushed incrementally with real backpressure. The **Flux**'s **Stream** is bridged straight into an axum streaming **Body** ; the whole stream is **never** buffered. An **Err** item mid-stream terminates the body cleanly.

- `Sse(Flux<T>)` streams `text/event-stream` — each element serialized to a bare `data: <json>\n\n` frame, byte-identical to the `firefly-sse` writer.
- `SseEvents(Flux<Event>)` streams pre-built `firefly_sse::Event` values — use it when you need `id` / `event` / `retry` fields.

⚠ WARNING

Backpressure is real, not cosmetic. A slow client throttles the producer; nothing is buffered up front. This is what lets a `NdJson` endpoint stream a million rows without the response landing fully in memory.

The reactive `WebClient`

The reactive HTTP client — the Rust analog of WebFlux's `WebClient` — hands its terminal operators back as `Mono` / `Flux`, so an outbound call drops straight into a reactive pipeline and composes end-to-end with the `NdJson` / `Sse` responders above. Full treatment in [HTTP Clients](#); the shape:

Rust

```

use firefly_client::WebClientBuilder;
use http::Method;
use serde::Deserialize;

#[derive(Deserialize)]
struct Order { id: String }
#[derive(Deserialize)]
struct Tick { seq: u64 }

# async fn ex() {
let client = WebClientBuilder::new("https://api.example.com").build();

// body_to_mono – the whole body decoded as one T.
let _order: firefly_reactive::Mono<Order> =
    client.get().uri("/orders/o1").retrieve().body_to_mono::<Order>();

// body_to_flux – a streamed NDJSON/SSE body decoded element-by-element,
// lazily and with backpressure.
let _ticks: firefly_reactive::Flux<Tick> = client
    .get()
    .uri("/ticks")
    .header("Accept", "application/x-ndjson")
    .retrieve()
    .body_to_flux::<Tick>();
# }

```

 REACTOR PARITY

Like WebFlux, the `WebClient` has no baked-in retry. Compose `Mono::retry` / `Mono::retry_backoff` on the returned publisher, just as WebFlux composes `retryWhen(..)` rather than baking retry into the client.

Reactive repositories, EDA, and CQRS

The same two types thread through the rest of the framework:

- **Repositories** — `ReactiveCrudRepository<T, ID>` returns `Mono` / `Flux`; the Postgres adapter streams rows out of `find_all()` as a `Flux` so a huge table never lands fully in memory. See [Persistence](#).
- **EDA** — `InMemoryBroker::subscribe_reactive(topic)` yields a `Flux<Event>`, and `publish_mono(event)` is a cold reactive publish. See [EDA](#).
- **CQRS** — `Bus::send_mono` / `Bus::query_mono` wrap the dispatch in a lazy `Mono<R>`, running the same handler lookup and middleware chain. See [CQRS](#).

Interop with raw `Stream` / `Future`

The reactive types are not a walled garden. Convert in and out at the edges:

- **In:** `Flux::from_stream` (a `Stream<Item = Result<T, FireflyError>>`), `Flux::from_value_stream` (a `Stream<Item = T>`), `Mono::from_future`, `Mono::from_result_future`.
- **Out:** `Flux::to_stream` / `Flux::into_stream`, `Mono::into_future` (or just `.await` the `Mono`).

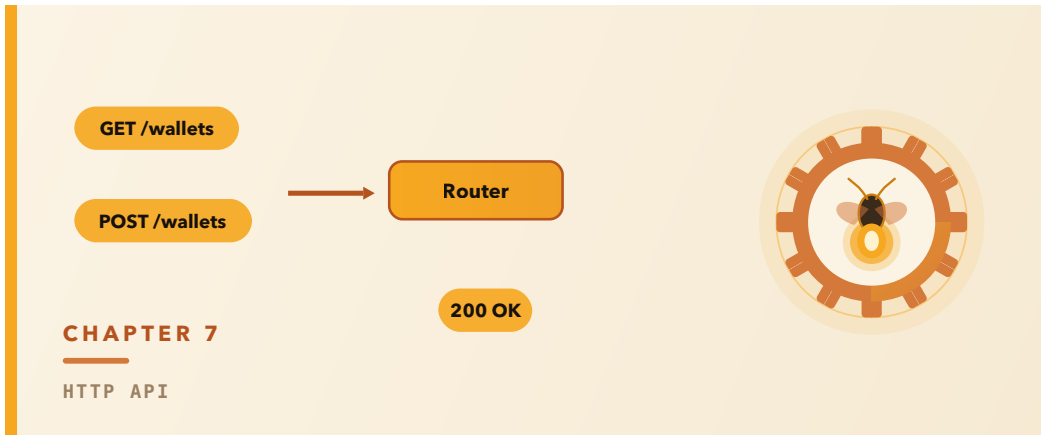
You now have the vocabulary the rest of the book builds on. Next, put it to work in [Your First HTTP API](#).



PART II

Modeling & Persisting the Domain





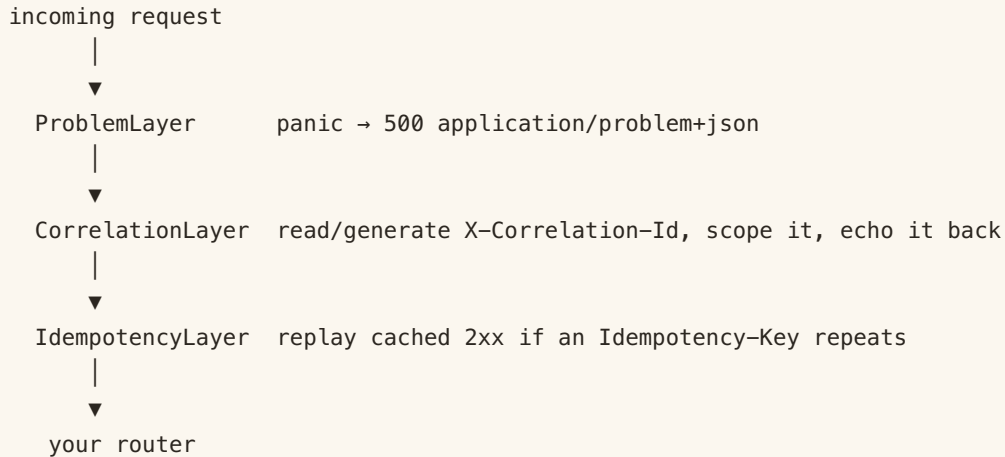
CHAPTER 7

Your First HTTP API

This chapter builds a real HTTP API: routes, JSON bodies, typed errors that render as RFC 7807 problems, idempotent writes, correlation propagation, and a reactive streaming endpoint. The HTTP layer is axum; Firefly contributes the middleware and the problem/idempotency/correlation behaviour through `firefly-web`, all composed for you by `Core::apply_middleware`.

The middleware chain

`Core::apply_middleware(router)` wraps your router in the canonical outermost chain. The default chain (matching the Go-parity core) is:

Text

Optional pyfly-parity layers (CORS, security headers, CSRF, request logging, request metrics, HTTP-exchange recording) weave in at their canonical filter order when you set the matching `CoreConfig` knob — all OFF by default. See [Production & Deployment](#).

Routes and JSON

A handler is a plain axum handler. Use axum's `Json` extractor/responder for bodies and `Path` for path parameters:

Rust

```

use axum::{extract::Path, routing::{get, post}, Json, Router};
use serde::{Deserialize, Serialize};

#[derive(Deserialize)]
struct CreateOrder {
    customer: String,
}

#[derive(Serialize)]
struct Order {
    id: String,
    customer: String,
}

async fn create_order(Json(body): Json<CreateOrder>) -> Json<Order> {
    Json(Order {
        id: "o1".into(),
        customer: body.customer,
    })
}

async fn get_order(Path(id): Path<String>) -> Json<Order> {
    Json(Order { id, customer: "alice".into() })
}

let router = Router::new()
    .route("/orders", post(create_order))
    .route("/orders/{id}", get(get_order));

```

Typed errors → RFC 7807 problems

Firefly errors are `firefly_kernel::FireflyError`, and `firefly-web` renders them as `application/problem+json`. Use `WebResult<T>` (an alias whose error arm is a `FireflyError`) so `?` turns any error into the right problem response:

Rust

```

use axum::{extract::Path, Json};
use firefly_kernel::FireflyError;
use firefly_web::WebResult;
use serde::Serialize;

#[derive(Serialize)]
struct Order { id: String }

async fn get_order(Path(id): Path<String>) -> WebResult<Json<Order>> {
    if id.is_empty() {
        // 400 problem+json, type .../bad-request.
        return Err(FireflyError::bad_request("id must not be empty"));
    }
    if id == "missing" {
        // 404 problem+json.
        return Err(FireflyError::not_found("no such order"));
    }
    Ok(Json(Order { id }))
}

```

The `FireflyError` constructors map straight to HTTP status:

Constructor	Status	Use
<code>FireflyError::bad_request(detail)</code>	400	malformed input
<code>FireflyError::unauthorized(detail)</code>	401	missing/invalid credentials
<code>FireflyError::forbidden(detail)</code>	403	authenticated but not allowed
<code>FireflyError::not_found(detail)</code>	404	absent resource
<code>FireflyError::conflict(detail)</code>	409	state conflict
<code>FireflyError::validation(detail)</code>	422	semantic validation failure
<code>FireflyError::business_rule(rule, detail)</code>	422	domain rule violated
<code>FireflyError::internal(detail)</code>	500	server fault

A rendered problem looks like:

JSON

```
{
  "type": "https://fireflyframework.org/problems/not-found",
  "title": "Not Found",
  "status": 404,
  "detail": "no such order"
}
```

Correlation IDs

Every response carries an `X-Correlation-Id`. An incoming one is honoured; otherwise one is generated. The id is scoped through `firefly_kernel::with_correlation_id` for the whole request, so every log line, every emitted event, and every outbound client call inherits it automatically. Read it in a handler:

Rust

```
use axum::Extension;
use firefly_web::CorrelationId;

async fn handler(Extension(cid): Extension<CorrelationId>) -> String {
    format!("correlation: {}", cid.0)
}
```

The extended `CorrelationLayer` also mints/echoes `X-Request-Id`, propagates `X-Tenant-Id` and `X-Transaction-Id` into the kernel task-locals, and echoes W3C `traceparent` / `tracestate`.

Idempotent writes

Any `POST` / `PUT` / `PATCH` carrying an `Idempotency-Key` header is recorded. A repeat of the same key replays the stored response with `Idempotent-Replay: true`; reusing the key with a *different* body is a `409`. This is on by default through the middleware chain — you write the handler once and get exactly-once-from-the-client-perspective semantics for free.

Shell

```
# First call: executes and stores.
curl -X POST localhost:8080/orders \
  -H 'Idempotency-Key: abc-123' \
  -H 'Content-Type: application/json' \
  -d '{"customer":"alice"}'

# Same key, same body: replays the stored response.
curl -i -X POST localhost:8080/orders \
  -H 'Idempotency-Key: abc-123' \
  -H 'Content-Type: application/json' \
  -d '{"customer":"alice"}'

# Idempotent-Replay: true
```

The default store is in-process (`MemoryIdempotencyStore`); for a multi-replica deployment, implement the `IdempotencyStore` trait over Redis or Postgres and pass it via `IdempotencyConfig`.

A reactive endpoint

Mount a streaming endpoint with the reactive responders from [The Reactive Model](#). The `Flux` is bridged straight into the response body with backpressure:

Rust

```
use axum::{routing::get, response::IntoResponse, Router};
use firefly_reactive::{Flux, Mono};
use firefly_web::{MonoJson, NdJson};
use serde::Serialize;

#[derive(Serialize, Clone)]
struct Order { id: String }

async fn one_order() -> impl IntoResponse {
    MonoJson(Mono::just(Order { id: "o1".into() }))
}

async fn stream_orders() -> impl IntoResponse {
    // Emits one application/x-ndjson line per order, flushed as produced.
    NdJson(Flux::just(vec![
        Order { id: "o1".into() },
        Order { id: "o2".into() },
    ]))
}

let router = Router::new()
    .route("/orders/one", get(one_order))
    .route("/orders/stream", get(stream_orders));
```

Putting it together

A complete service mounts the routes through `apply_middleware`, serves the public API and actuator on separate ports, and runs under the lifecycle application for graceful shutdown:

Rust

```

use axum::{routing::{get, post}, Json, Router};
use firefly_starter_core::{Core, CoreConfig};
use serde::{Deserialize, Serialize};

#[derive(Deserialize)]
struct CreateOrder { customer: String }
#[derive(Serialize)]
struct Order { id: String, customer: String }

async fn create_order(Json(b): Json<CreateOrder>) -> Json<Order> {
    Json(Order { id: "o1".into(), customer: b.customer })
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let core = Core::new(CoreConfig { app_name:
"orders".into(), ..Default::default() });
    core.init_logging()?;

    let api = core.apply_middleware(
        Router::new().route("/orders", post(create_order)),
    );
    let admin = core.actuator_router(Vec::new());

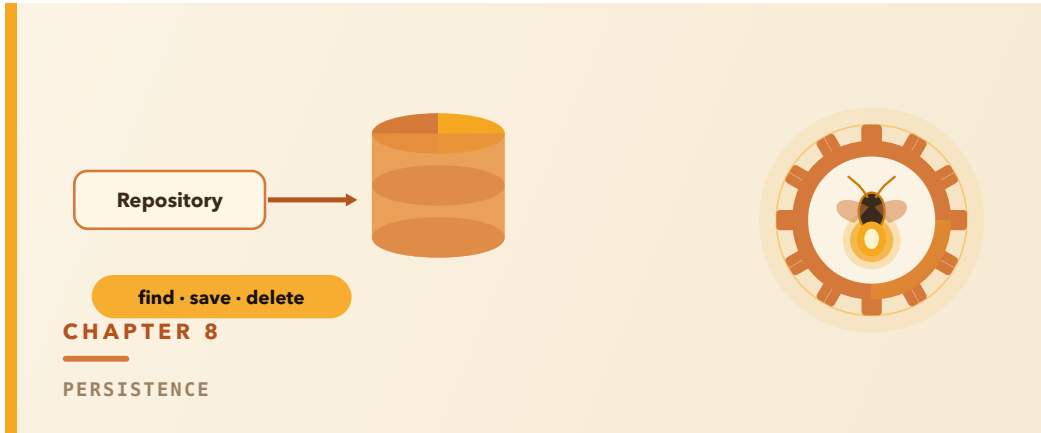
    let app = core
        .new_application()
        .on_server("api", move |sd| async move {
            let l = tokio::net::TcpListener::bind("0.0.0.0:8080").await?;
            axum::serve(l, api).with_graceful_shutdown(sd.wait()).await?;
            Ok(())
        })
        .on_server("admin", move |sd| async move {
            let l = tokio::net::TcpListener::bind("0.0.0.0:8081").await?;
            axum::serve(l, admin).with_graceful_shutdown(sd.wait()).await?;
            Ok(())
        });
    app.run().await?;
}

```



```
    ok(())  
}
```

Next, give your service a database with reactive repositories in [Persistence & Reactive Repositories](#).



CHAPTER 8

Persistence & Reactive Repositories

`firefly-data` provides the framework's **persistence vocabulary**: a composable `Filter` query DSL, a `Page<T>` paged-result envelope, a blocking `Repository<T, K>` contract, and — built on `firefly-reactive` — a **reactive CRUD surface** that is the Rust analog of Spring Data R2DBC. This chapter covers both, with a real streaming Postgres repository.

The `Page<T>` envelope

`Page<T>` is the canonical paged result, wire-identical to the Java/.NET/Go `Page<T>` so SDK clients deserialize it uniformly:

Rust

```
pub struct Page<T> {
    pub content: Vec<T>,
    pub number: usize,      // zero-based page index
    pub size: usize,
    pub total_elements: u64,
    pub total_pages: usize, // derived
}
```

The **Filter** DSL

A **Filter** composes predicates, sorts, and a page window, and renders to a parameterised **WHERE** clause via **to_sql()**:

Rust

```
use firefly_data::{Direction, Filter, Op, Predicate};
use serde_json::json;

let filter = Filter::default()
    .where_eq("status", json!("OPEN"))
    .add(Predicate { field: "total".into(), op: Op::Gte, value: json!(100) })
    .order_by("created_at", Direction::Desc)
    .paged(0, 20);

let (where_clause, args) = filter.to_sql();
// where_clause: a parameter-indexed " WHERE ..." fragment
// args:          the bound values, in order
assert!(where_clause.contains("WHERE"));
assert_eq!(args.len(), 2);
```

The operators (**Op**) cover **Eq**, **Ne**, **Lt**, **Lte**, **Gt**, **Gte**, **Like**, **ILike**, **In**, and **IsNil** — the last skips an argument slot, so a predicate list and its argument list stay aligned.

The blocking **Repository** contract

Repository<T, K> is the object-safe **async_trait** port; **MemoryRepository** implements it for tests, and you back it with your driver in production:

Rust

```
#[async_trait]
pub trait Repository<T, K>: Send + Sync {
    async fn find_by_id(&self, id: &K) -> Result<T, DataError>;
    async fn find(&self, filter: &Filter) -> Result<Page<T>, DataError>;
    // save / delete / count ...
}
```

The reactive **CRUD** surface

On top of the blocking contract, **firefly-data** adds a **reactive** surface — the Spring Data R2DBC analog — built on **Mono** / **Flux**. It is purely additive: nothing about the existing **Repository** API changes.

Spring Data R2DBC	firefly-data reactive
<code>ReactiveCrudRepository<T, ID></code>	<code>ReactiveCrudRepository<T, ID></code>
<code>Flux<T> findAll()</code>	<code>find_all() -> Flux<T></code>
<code>Flux<T> findAllById(ids)</code>	<code>find_all_by_id(ids) -> Flux<T></code>
<code>Mono<T> findById(id)</code>	<code>find_by_id(id) -> Mono<T></code>
<code>Mono<Boolean> existsById(id)</code>	<code>exists_by_id(id) -> Mono<bool></code>
<code>Mono<T> save(e)</code>	<code>save(e) -> Mono<T></code>
<code>Flux<T> saveAll(es)</code>	<code>save_all(es) -> Flux<T></code>
<code>Mono<Void> deleteById(id)</code>	<code>delete_by_id(id) -> Mono<()></code>
<code>Mono<Void> deleteAll()</code>	<code>delete_all() -> Mono<()></code>
<code>Mono<Long> count()</code>	<code>count() -> Mono<u64></code>
<code>findAll(Specification, Pageable)</code>	<code>ReactiveSpecificationRepository</code>

A "no row" `find_by_id` maps to an **empty** `Mono` (Reactor's `Mono.empty()`), exactly as Spring Data signals a missing `findById`.

In-memory, for tests

`ReactiveMemoryRepository` is the reactive twin of `MemoryRepository`. Drive the publishers with `block()` / `collect_list()`:

Rust

```

use firefly_data::{ReactiveCrudRepository, ReactiveMemoryRepository};

#[derive(Clone, PartialEq, Debug)]
struct User { id: String, name: String }

#[tokio::main]
async fn main() {
    let repo = ReactiveMemoryRepository::new(|u: &User| u.id.clone());

    // save -> Mono<T>
    repo.save(User { id: "u1".into(), name: "alice".into() })
        .block().await.unwrap();

    // find_all -> Flux<T>, collected to a Vec
    let all = repo.find_all().collect_list().block().await.unwrap().unwrap();
    assert_eq!(all.len(), 1);

    // find_by_id miss -> empty Mono
    assert_eq!(repo.find_by_id("ghost".into()).block().await.unwrap(), None);
    assert_eq!(repo.count().block().await.unwrap(), Some(1));
}

```

Real Postgres, streaming rows as a Flux

`PostgresReactiveRepository` is the production repository over `tokio-postgres`. Reads drive the driver's `query_raw` row stream, so each row is decoded by a `RowMapper` and emitted the moment it arrives over the wire — a million-row table never lands fully in memory. Writes use a per-entity `inserter` closure that renders a `T` to an upsert whose `RETURNING` projects the configured columns.

Rust

```

use std::sync::Arc;
use firefly_data::{PostgresReactiveRepository, ReactiveCrudRepository,
TableConfig};
use firefly_kernel::FireflyError;
use tokio_postgres::{Row, types::ToSql, NoTls};

#[derive(Clone, PartialEq, Debug)]
struct User { id: String, name: String }

# async fn ex() -> Result<(), Box<dyn std::error::Error>> {
let (client, conn) =
    tokio_postgres::connect("postgres://localhost/app", NoTls).await?;
tokio::spawn(async move { let _ = conn.await; });
let client = Arc::new(client);

let repo: PostgresReactiveRepository<User, String> =
PostgresReactiveRepository::new(
    Arc::clone(&client),
    TableConfig::new("users", "id", ["id", "name"]),
    // RowMapper: decode (id, name) from each streamed row.
    |row: &Row| Ok(User {
        id: row.try_get("id").map_err(|e|
FireflyError::internal(e.to_string()))?,
        name: row.try_get("name").map_err(|e|
FireflyError::internal(e.to_string()))?,
    }),
    // inserter: upsert RETURNING the projected columns.
    |u: &User| (
        "INSERT INTO \"users\" (\"id\", \"name\") VALUES ($1, $2) \
        ON CONFLICT (\"id\") DO UPDATE SET \"name\" = EXCLUDED.\"name\" \
        RETURNING \"id\", \"name\"".to_string(),
        vec![
            Box::new(u.id.clone()) as Box<dyn ToSql + Sync + Send>,
            Box::new(u.name.clone()) as Box<dyn ToSql + Sync + Send>,
        ],
    ),
);

```

```
// Rows stream lazily out of find_all() as a Flux.
let all = repo.find_all().collect_list().block().await?.unwrap();
# Ok(())
# }
```

Use `stream_query(sql, params)` for custom derived queries: any `SELECT` projecting the configured columns is streamed row-by-row through the same `RowMapper`. This `Flux` plugs directly into a `NdJson` / `Sse` endpoint, so a database read streams to the client end-to-end with backpressure — no collect-then-emit step anywhere in the path.

Reactive specification / paging

`ReactiveSpecificationRepository` runs a composable `Specification` predicate with an optional `Pageable` window and streams the matches as a `Flux` — the reactive analog of `findAll(Specification, Pageable)`, but with no intermediate `Page<T>` envelope, so it plugs straight into an NDJSON / SSE endpoint with backpressure.

Pluggable databases – a new DB is a new adapter

`firefly-data` is the `ports` crate: it owns no driver and implies no SQL engine. The `Filter` DSL, the composable `Specification`, the repository traits, and the auditing / soft-delete policies are all storage-agnostic. The lowering surface is what makes this hexagonal:

- a `SqlDialect` trait with three shipped impls — `PostgresDialect`, `MySqlDialect`, `SqliteDialect` — so `Filter::to_sql_with(&dialect)` and `Specification::to_sql_with(&dialect)` render the *same* query tree per backend, getting placeholder style (`$1` vs `?`), identifier quoting (`"id"` vs ``id``), `IN`-list shape, and case-insensitive `LIKE` right for each. (`Filter::to_sql` / `Specification::to_sql` stay the PostgreSQL default.)

- a `Specification::to_mongo()` / `Filter::to_mongo()` that lowers the same tree to a MongoDB `$`-operator filter document.

Two adapter crates implement those ports so you code once and swap backends.

Relational – `firefly-data-sqlx` (Postgres / MySQL / SQLite)

`SqlxReactiveRepository` (and the blocking `SqlxRepository`) serve all three relational backends from one codebase. A `Db` enum tags a `PgPool` / `MySqlPool` / `SqlitePool` with its `Backend`, and the repository picks the matching `SqlDialect` at runtime — so "new relational DB = new pool", not "new adapter". `UPSERT` is dialect-aware (`ON CONFLICT ... DO UPDATE` for Postgres/SQLite, `ON DUPLICATE KEY UPDATE` for MySQL), reads stream off sqlx's row stream into a `Flux`, and an optional `Auditor` / `SoftDeletePolicy` auto-stamps and hides rows on every write/read.

Rust

```

use firefly_data::{ReactiveCrudRepository, TableConfig};
use firefly_data_sqlx::{AnyRow, ColumnValue, Db, SqlxReactiveRepository};
use firefly_kernel::FireflyError;

#[derive(Debug, Clone, PartialEq)]
struct User { id: String, name: String }

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async
{
    let pool = sqlx::SqlitePool::connect("sqlite::memory:").await.unwrap();
    sqlx::query(r#"CREATE TABLE "users" ("id" TEXT PRIMARY KEY, "name" TEXT NOT
NULL)"#)
        .execute(&pool).await.unwrap();

    let repo: SqlxReactiveRepository<User, String> = SqlxReactiveRepository::new(
        Db::Sqlite(pool),
        TableConfig::new("users", "id", ["id", "name"]),
        // RowMapper: decode by column name – backend-agnostic via AnyRow.
        |row: &AnyRow| Ok:<_, FireflyError>(User {
            id: row.get_str("id")?,
            name: row.get_str("name")?,
        }),
        // RowWriter: the entity's (column, value) pairs.
        |u: &User| vec![
            ColumnValue::new("id", u.id.clone()),
            ColumnValue::new("name", u.name.clone()),
        ],
    );
    repo.save(User { id: "u1".into(), name:
    "alice".into() }).block().await.unwrap();
    # });

```

Switching to Postgres or MySQL is `Db::Postgres(pg_pool)` / `Db::MySql(my_pool)` — the repository call sites do not change.

Document – firefly-data-mongodb (MongoDB)

`MongoRepository<T, ID>` puts a MongoDB collection behind the same `ReactiveCrudRepository` + `ReactiveSpecificationRepository` traits, lowering a `Specification` via `Specification::to_mongo()` exactly as the relational adapters lower it via `to_sql`. A `BaseDocument` mixin (embedded with `#[serde(flatten)]`) carries the audit stamps and soft-delete column, and `with_soft_delete(policy)` makes every read inject a `{"<column>": null}` guard while `delete_by_id` becomes a logical delete. Reads stream lazily off the driver cursor as a `Flux`.

Rust

```
use firefly_data::{ReactiveCrudRepository, Specification};
use firefly_data_mongodb::{BaseDocument, MongoRepository};
use mongodb::bson::{Bson, Document};
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize)]
struct UserDocument {
    #[serde(rename = "_id")] id: String,
    name: String,
    #[serde(flatten)] base: BaseDocument,
}

# async fn run() -> Result<(), Box<dyn std::error::Error>> {
let client = mongodb::Client::with_uri_str("mongodb://localhost:27017").await?;
let collection = client.database("app").collection::<Document>("users");
let repo: MongoRepository<UserDocument, String> =
    MongoRepository::new(collection, |u: &UserDocument|
        Bson::String(u.id.clone()));

repo.save(UserDocument {
    id: "u1".into(), name: "alice".into(), base: BaseDocument::new(),
}).block().await?;
# Ok(())
# }
```

Because all four backends sit behind the same ports, a service that codes against `Repository` / `ReactiveCrudRepository` / `Specification` can move from Postgres to MySQL, SQLite, or MongoDB by swapping the adapter constructor — and adding a *new* database is "write a `firefly-data-<tech>` crate that implements the ports", not "rewrite the data layer". Both adapters are tested against real Postgres, MySQL, SQLite, and MongoDB.

Schema migrations

`firefly-migrations` is a forward-only SQL migration runner. Migration files are named `V{version}__{description}.sql` (e.g. `V001__init.sql`); each runs once, in version order, inside a transaction. The runner works against any store behind the small synchronous `Database` port:

Rust

```
use firefly_migrations::{run, DirSource};

let src = DirSource { dir: "migrations".into() };
run(&mut db, &src)?; // applies pending migrations in order
let status = firefly_migrations::inspect(&mut db, &src)?; // applied + pending
```

The `CLI` wraps this: `firefly db init`, `firefly db migrate -m "create users"`, `firefly db upgrade --url sqlite://app.db`, and `firefly db status`.

Transactions

`firefly-transactional` provides `with_tx(ctx, db, f)` over pluggable `Database` / `Transaction` ports, so a unit of work commits or rolls back as a whole. Bind the ports to your driver and run your repository calls inside the closure.

With persistence in place, the next chapter models the business itself. See [Domain-Driven Design](#).



CHAPTER 9

Domain-Driven Design

`firefly-kernel::ddd` is a zero-dependency DDD building-block kit: a **Specification** predicate algebra, an **Entity** trait with identity equality, transient domain events, and an aggregate's pending-events buffer. This chapter models a small **Wallet** aggregate to show how the pieces fit, and how they relate to repositories and events.

🌿 SPRING PARITY

This is the same vocabulary you know from DDD on the JVM — entities, value objects, aggregates, specifications, domain events — expressed as idiomatic Rust traits. There is no framework lock-in: it is plain types.

Value objects

A value object is identity-free and compared by value. In Rust that is just a **Clone** struct (use struct-update syntax to "modify" it by producing a new one):

Rust

```
#[derive(Clone, PartialEq, Debug)]
struct Money {
    amount_cents: i64,
    currency: String,
}

impl Money {
    fn add(&self, other: &Money) -> Money {
        Money {
            amount_cents: self.amount_cents + other.amount_cents,
            currency: self.currency.clone(),
        }
    }
}
```

Entities and identity

An `Entity` has identity — two entities are "the same" if their ids match, even if their fields differ. Implement the `Entity` trait to get `is_transient` and `same_identity` for free:

Rust

```

use firefly_kernel::ddd::Entity;

struct Wallet {
    id: Option<String>,    // None while transient (not yet persisted)
    balance_cents: i64,
}

impl Entity for Wallet {
    type Id = String;
    fn id(&self) -> Option<&String> {
        self.id.as_ref()
    }
}

let a = Wallet { id: Some("w1".into()), balance_cents: 0 };
let b = Wallet { id: Some("w1".into()), balance_cents: 9999 };
assert!(a.same_identity(&b));    // same id => same entity
assert!(!a.is_transient());

let fresh = Wallet { id: None, balance_cents: 0 };
assert!(fresh.is_transient());    // no id yet

```

Specifications – a predicate algebra

A `Specification<T>` is a reusable, composable business rule. Implement `is_satisfied_by`, then combine rules with `.and()`, `.or()`, and `.not()`. Any `Fn(&T) -> bool` closure is already a `Specification`, so you can mix named rules with ad-hoc ones:

Rust

```

use firefly_kernel::ddd::Specification;

struct Customer { age: u32, premium: bool }

struct IsAdult;
impl Specification<Customer> for IsAdult {
    fn is_satisfied_by(&self, c: &Customer) -> bool {
        c.age >= 18
    }
}

// Combine a named spec with an inline closure spec.
let premium_adult = IsAdult.and(|c: &Customer| c.premium);

assert!(premium_adult.is_satisfied_by(&Customer { age: 30, premium: true }));
assert!(!premium_adult.is_satisfied_by(&Customer { age: 30, premium: false }));

```

Specifications shine for query filters and guard clauses: define the rule once, reuse it in validation, in repository filtering, and in tests.

Domain events

A transient domain event records "something happened" — it is collected during a transaction and dispatched after the unit of work commits. Each event embeds an `EventMeta` (a fresh UUID v4 + a UTC timestamp); `event_type` defaults to the short type name:

Rust

```

use firefly_kernel::ddd::{EventMeta, TransientDomainEvent};

struct WalletCredited {
    meta: EventMeta,
    wallet_id: String,
    amount_cents: i64,
}

impl WalletCredited {
    fn new(wallet_id: String, amount_cents: i64) -> Self {
        Self { meta: EventMeta::new(), wallet_id, amount_cents }
    }
}

impl TransientDomainEvent for WalletCredited {
    fn meta(&self) -> &EventMeta {
        &self.meta
    }
}

let ev = WalletCredited::new("w1".into(), 500);
assert_eq!(ev.event_type(), "WalletCredited"); // defaults to the type name

```

 NOTE

This is the **non-event-sourced** event: state persists through repositories and events are merely collected for post-commit publication. The event-sourced variant (versioned, wire-formatted, `EventStore`-coupled) lives in `firefly-eventsourcing` — see [Event Sourcing](#).

The aggregate root

An aggregate enforces invariants and accumulates the events it raises.

`PendingEvents<E>` is the buffer — `raise` to record, `pending` to inspect, `drain` to take-and-clear after committing:

Rust

```

use firefly_kernel::ddd::{EventMeta, PendingEvents, TransientDomainEvent};

struct WalletCredited { meta: EventMeta, amount_cents: i64 }
impl TransientDomainEvent for WalletCredited {
    fn meta(&self) -> &EventMeta { &self.meta }
}

struct Wallet {
    id: String,
    balance_cents: i64,
    events: PendingEvents<WalletCredited>,
}

impl Wallet {
    fn new(id: String) -> Self {
        Self { id, balance_cents: 0, events: PendingEvents::new() }
    }

    // A behaviour method: enforce the invariant, mutate state, raise an event.
    fn credit(&mut self, amount_cents: i64) -> Result<(), String> {
        if amount_cents <= 0 {
            return Err("credit must be positive".into());
        }
        self.balance_cents += amount_cents;
        self.events.raise(WalletCredited { meta: EventMeta::new(),
amount_cents });
        Ok(())
    }
}

let mut w = Wallet::new("w1".into());
w.credit(500).unwrap();
w.credit(250).unwrap();
assert_eq!(w.balance_cents, 750);
assert_eq!(w.events.len(), 2);

// After persisting, drain the events to publish them on the broker.
let to_publish = w.events.drain();

```

```
assert_eq!(to_publish.len(), 2);
assert!(w.events.is_empty());
```

The application-service workflow

The shape of a write use case in a DDD service:

1. **Load** the aggregate from a repository (`Repository<T, K>::find_by_id`).
2. **Invoke** a behaviour method that enforces invariants and `raise`s events.
3. **Save** the aggregate (`Repository::save` — or `next_id` from a fresh `Uuid::new_v4()` for a brand-new one).
4. **Drain** the pending events and **publish** them on the [EDA broker](#) after the unit of work commits.

Rust

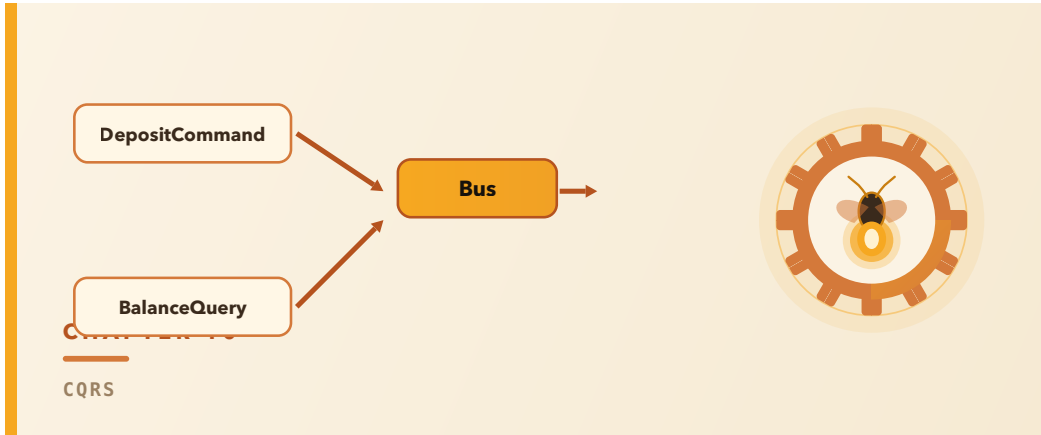
```
// Pseudocode for the application service.
let mut wallet = repo.find_by_id(&id).await?; // 1. load
wallet.credit(500)?; // 2. behaviour + raise
repo.save(wallet.clone()).await?; // 3. save
for ev in wallet.events.drain() { // 4. publish post-commit
    broker.publish(to_event(&ev)).await?;
}
```

This keeps the domain pure (no I/O in the aggregate), the application service thin (orchestration only), and events published exactly once after commit.

Mapping the rest of the DDD glossary to Firefly:

DDD concept	Firefly
Value object	a <code>Clone</code> struct + struct-update syntax
Entity	<code>firefly_kernel::ddd::Entity</code>
Aggregate root	a struct holding <code>PendingEvents<E></code>
Specification	<code>firefly_kernel::ddd::Specification<T></code>
Domain event	<code>firefly_kernel::ddd::TransientDomainEvent</code>
Domain repository	<code>firefly_data::Repository<T, K></code>
Business-rule error	<code>FireflyError::business_rule(rule, detail)</code> (422)
Aggregate-not-found	<code>FireflyError::aggregate_not_found(type, id)</code> (404)

Next, separate reads from writes with CQRS.



CHAPTER 10

CQRS: Commands & Queries

`firefly-cqrs` provides the framework's **type-dispatched command/query bus**: typed handlers registered at startup, dispatched through `Bus::send` / `Bus::query`, matched by `std::any::TypeId`. On top of that sits pluggable middleware for validation, query caching, and authorization, plus a reactive `Mono`-returning surface. This chapter wires a real bus end-to-end.

🌿 SPRING PARITY

The bus is the framework's command/query dispatcher. `register` ~ a `@CommandHandler` / `@QueryHandler`, `send` / `query` ~ dispatching through the gateway, and the middleware chain ~ Spring's handler interceptors.

Commands, queries, and the `Message` trait

Every command and query implements `Message`. For a plain message that is one line; the trait's optional methods (`validate`, `cache_ttl`, `authorize`) are overridable defaults that the matching middleware picks up automatically:

Rust

```
pub trait Message: Clone + Serialize + Send + Sync + 'static {
    fn validate(&self) -> Result<(), CqrsError> { Ok(()) } //
    ValidationMiddleware
    fn cache_ttl(&self) -> Option<Duration> { None } // QueryCache
}
```

`Clone` stands in for pass-by-value handler invocation; `Serialize` seeds the cache key. A message with no special behaviour is `impl Message for MyCommand {}`.

A bus, end-to-end

Register typed async handlers, then dispatch. The handler's input type is the dispatch key; its output is the typed result:

Rust

```

use std::time::Duration;
use firefly_cqrs::{Bus, CqrsError, Message, QueryCache, ValidationMiddleware};
use serde::Serialize;

#[derive(Clone, Serialize)]
struct CreateUser { name: String }

impl Message for CreateUser {
    fn validate(&self) -> Result<(), CqrsError> {
        if self.name.is_empty() {
            return Err(CqrsError::validation("name required"));
        }
        Ok(())
    }
}

#[derive(Clone, Serialize)]
struct GetUser { id: String }

impl Message for GetUser {
    fn cache_ttl(&self) -> Option<Duration> { Some(Duration::from_secs(60)) }
}

#[derive(Clone, Debug)]
struct UserCreated { id: String, name: String }

#[tokio::main]
async fn main() {
    let bus = Bus::new();
    bus.use_middleware(ValidationMiddleware::new());
    let cache = QueryCache::new();
    bus.use_middleware(cache.middleware());

    bus.register(|c: CreateUser| async move {
        Ok:::<_, CqrsError>(UserCreated { id: "u1".into(), name: c.name })
    });
    bus.register(|q: GetUser| async move {
        Ok:::<_, CqrsError>(UserCreated { id: q.id, name: "alice".into() })
    });
}

```



```
});

let created: UserCreated = bus.send(CreateUser { name:
"alice".into() }).await.unwrap();
let view: UserCreated = bus.query(GetUser { id:
created.id }).await.unwrap();
assert_eq!(view.name, "alice");

cache.invalidate_type::<GetUser>(); // after a mutation
}
```

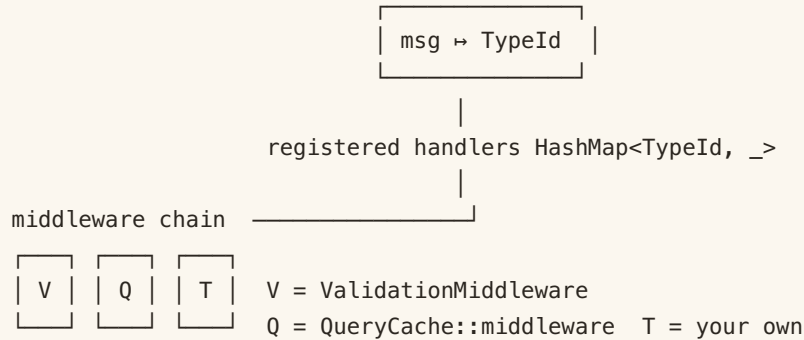
`Bus::query` is a readability synonym for `Bus::send`. An unrouted message is a `CqrsError::NoHandler`.

The middleware chain

Middleware runs first-registered = outermost. Three ship in the box:

Middleware	Behaviour
<code>ValidationMiddleware</code>	calls <code>Message::validate</code> before dispatch, short-circuits on error
<code>QueryCache::middleware()</code>	memoises results for messages whose <code>cache_ttl</code> is <code>Some</code>
<code>AuthorizationMiddleware</code>	calls <code>Message::authorize(ctx)</code> , denies before the handler runs

Text



Query caching and invalidation

`QueryCache` memoises a query result under a `<type name>:<sha-256 of JSON>` key. After a mutation, evict precisely:

- `cache.invalidate_type::<GetUser>()` — every cached result for exactly that query type;
- `cache.invalidate(prefix)` — every key starting with `prefix`.

For event-driven invalidation, `EdaCacheInvalidationBridge::new(cache)` evicts entries when domain events arrive on a `firefly-eda` broker: `register(event_type, "order:{order_id}")` maps an event type to cache-key patterns whose `{field}` placeholders are resolved from the event payload, and `subscribe(&broker, topic)` wires it in.

Authorization

A message can gate itself. Implement `authorize` (default: always authorized), wire the `AuthorizationMiddleware`, and a denial short-circuits dispatch before the handler runs:

Rust

```
use firefly_cqrs::{AuthorizationMiddleware, Bus};

let bus = Bus::new();
bus.use_middleware(AuthorizationMiddleware::new()); // or ::disabled() to allow
all
```

The hook receives the dispatch's `ExecutionContext` when one is attached. A denial becomes `CqrsError::Authorization(result)`, which maps to a 403.

ExecutionContext

`ExecutionContext` carries the request's identity — user, tenant, organization, session, request id, source, client IP, user agent, timestamp, arbitrary properties, and feature flags. Build one with the fluent builder and attach it to a dispatch; it reaches `authorize`, any middleware reading `Envelope::context`, and context-aware handlers:

Rust

```
use firefly_cqrs::{Bus, ExecutionContext};

let ctx = ExecutionContext::builder()
    .user("u1")
    .tenant("acme")
    .build();

let result: UserCreated = bus
    .send_with_context(CreateUser { name: "alice".into() }, ctx)
    .await?;
```

Fluent builders

`CommandBuilder::create(cmd)` / `QueryBuilder::create(q)` accumulate the identity fields the message carries — a fresh UUID `message_id`, `correlated_by`, `initiated_by`, a timestamp, free-form metadata, an optional context — and dispatch via `execute_with(&bus)`. `QueryBuilder` adds cache control: `cached_for(ttl)` / `uncached()` override `cache_ttl` for the dispatch, and `with_cache_key(key)` replaces the derived key.

The reactive bus

The bus exposes a Reactor / WebFlux-style surface that wraps the eventual result in a lazy `Mono<R>` — the same handler lookup, the same middleware chain, run only when the `Mono` is subscribed, blocked, or awaited.

Method	Returns	Reactor analog
<code>Bus::send_mono(cmd)</code>	<code>Mono<R></code>	<code>Mono<R> bus.send(cmd)</code>
<code>Bus::query_mono(q)</code>	<code>Mono<R></code>	<code>Mono<R> bus.query(q)</code>
<code>Bus::send_mono_with_context</code>	<code>Mono<R></code>	context-carrying send
<code>Bus::query_mono_with_context</code>	<code>Mono<R></code>	context-carrying query

The reactive methods take `&Arc<Bus>` (so the lazy `Mono` can own the bus); register handlers on the `Arc<Bus>` exactly as on a `Bus`:

Rust

```

use std::sync::Arc;
use firefly_cqrs::{Bus, CqrsError, Message};

let bus = Arc::new(Bus::new());
bus.register(|c: CreateUser| async move {
    Ok:::<_, CqrsError>(UserCreated { id: "u1".into(), name: c.name })
});

// Compose with Reactor operators, then block / subscribe / await.
let id = bus
    .send_mono:::<_, UserCreated>(CreateUser { name: "alice".into() })
    .map(|u| u.id)
    .block()
    .await?; // Ok(Some("u1"))

```

Because `firefly-reactive` fixes its error channel to `FireflyError`, a failed dispatch is mapped from `CqrsError` into a status-faithful `FireflyError` (a validation failure → 422, an authorization denial → 403, a missing handler → 500), with the original `CqrsError` preserved as the error's `source()`. So a reactive command flows straight into the RFC 7807 problem stack while staying inspectable.

Listing handlers

`Bus::handler_names()` returns the sorted, fully-qualified type names of every registered handler — consumed by the admin actuator to show the CQRS map.

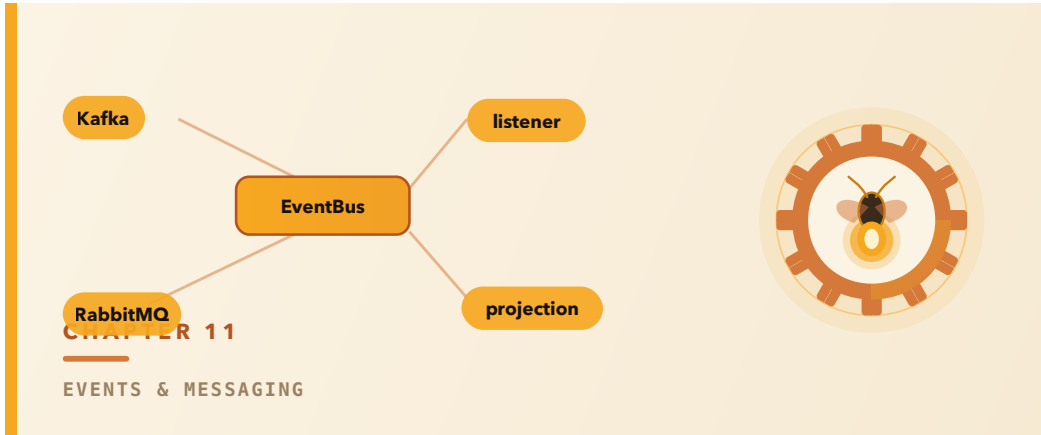
The bus dispatches commands and queries within a service. To communicate *between* services, fan out domain events. Continue to [Event-Driven Architecture & Messaging](#).



PART III

Event-Driven Architecture





CHAPTER 11

Event-Driven Architecture & Messaging

`firefly-eda` is the framework's **event-driven architecture port**. It defines the **Event** envelope every Firefly event flows through, the **Publisher** / **Subscriber** / **Broker** ports, an in-process **InMemoryBroker**, and the messaging machinery — glob topics, consumer groups, retry/DLQ, event filters, and a reactive **Flux** subscription surface. The production transports (Kafka, RabbitMQ, Postgres outbox, Redis Streams) implement the same ports and slot in at wiring time.

🌿 SPRING PARITY

The **Broker** port is the Spring Cloud Stream binder abstraction. `wrap_listener`'s retry/DLQ is `@RetryableTopic`; the glob subscription is `bus.subscribe("user.*", ...)`.

The Event envelope

Event is wire-compatible across the Java/.NET/Go/Python/Rust ports — the same JSON field names and omission rules. Construct one with **Event::new**, which also stamps **correlationId** from the kernel's task-local correlation scope:

Rust

```
use firefly_eda::Event;

let ev = Event::new(
    "orders.created",    // topic
    "OrderCreated",     // event type
    "orders-svc",       // source
    Some(br#"{"id":"o1"}"#.to_vec()), // payload (base64 on the wire)
)
.with_header("x-tenant", "acme")
.with_key(b"customer-42".to_vec()); // partition / routing key
```

The **key** is what Kafka uses for partitioning and RabbitMQ for routing; it is omitted from the wire when absent, so older events stay byte-for-byte identical.

The in-process broker

InMemoryBroker is the default — fan-out delivery, glob topic matching, and per-**(topic, group)** round-robin, with no external dependency. Subscribe a handler, publish an event:

Rust

```

use firefly_eda::{handler, Event, InMemoryBroker};

#[tokio::main]
async fn main() {
    let broker = InMemoryBroker::new();

    broker
        .subscribe(
            "orders.created",
            handler(|ev: Event| async move {
                println!("got order {}", ev.id);
                Ok(())
            }),
        )
        .unwrap();

    let ev = Event::new("orders.created", "OrderCreated", "orders-svc",
        Some(br#"{"id":"o1"}"#.to_vec()));
    broker.publish(ev).await.unwrap();
    broker.close().unwrap();
}

```

`InMemoryBroker::publish` awaits each subscribed handler sequentially on the publisher's task; the first handler error short-circuits and is returned to the publisher. After `close()`, publish and subscribe fail with `EdaError::Closed`.

Glob topics and consumer groups

A subscription topic is a glob pattern (`*`, `?`, `[..]`, `{a,b}`); a published event is delivered to every subscription whose pattern matches its topic:

Rust

```
broker.subscribe("orders.*", handler(|ev| async move { Ok(()) })).unwrap();
// matches orders.created, orders.shipped, ...
```

Consumer groups give competing-consumer delivery: within a group each matching event goes to exactly **one** member (round-robin); distinct groups each get their own copy:

Rust

```
broker.subscribe_group("orders.*", "fulfillment", handler1).unwrap();
broker.subscribe_group("orders.*", "fulfillment", handler2).unwrap();
// each orders.* event reaches exactly one of handler1/handler2
```

Retry and dead-letter

`wrap_listener(handler, publisher, policy)` is the adapter-agnostic retry/DLQ wrapper. A failing delivery is retried up to `retries` times with linear backoff (`retry_delay * attempt`); on exhaustion the event is republished to the dead-letter topic (when set), carrying the original payload/key/headers plus `x-original-topic` and `x-exception` diagnostic headers:

Rust

```

use std::sync::Arc;
use std::time::Duration;
use firefly_eda::{handler, wrap_listener, InMemoryBroker, ListenerPolicy};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async {
    let broker = Arc::new(InMemoryBroker::new());
    let inner = handler(|_ev| async {
        Err(firefly_kernel::FireflyError::internal("boom")) });
    let wrapped = wrap_listener(
        inner,
        broker.clone(),
        ListenerPolicy::with_retries(3)
            .retry_delay(Duration::from_millis(50))
            .dead_letter_topic("orders.DLT"),
    );
    broker.subscribe("orders", wrapped).unwrap();
# });

```

For an inspectable record of failures (rather than a routing topic), wire an `EdaDeadLetterStore` via `ListenerPolicy::dead_letter_store`: an exhausted event is captured into the store (queryable with `list` / `get` / `remove`).

Event filters

`EventFilter` is a per-envelope delivery gate layered over topic matching. Where the broker decides *which* subscriptions a topic reaches, a filter decides whether a reached subscription actually *runs*. Two ship — a header regex filter and an arbitrary predicate filter:

Rust

```

use firefly_eda::{handler, with_filters, Event, HeaderEventFilter,
InMemoryBroker};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async
{
    let broker = InMemoryBroker::new();
    let inner = handler(|_ev: Event| async { Ok(()) });
    let gated = with_filters(inner, [HeaderEventFilter::new("x-tenant", r"^acme-.+
$").unwrap()]);
    broker.subscribe("orders", gated).unwrap();
    # });

```

An event must pass *every* filter to be delivered; a non-matching event is dropped before the handler body runs.

The reactive subscription surface

`InMemoryBroker::subscribe_reactive(topic)` is the reactive twin of `subscribe_channel` — a `Flux<Event>` that emits every event delivered to the topic, composing with the whole Reactor operator set. `publish_mono(event)` is the cold reactive publish (nothing happens until the `Mono` is subscribed):

Rust

```

use std::sync::Arc;
use firefly_eda::{Event, InMemoryBroker};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async
{
    let broker = Arc::new(InMemoryBroker::new());
    let flux = broker.subscribe_reactive("orders.*").unwrap();

    broker
        .publish_mono(Event::new("orders.created", "OrderCreated", "svc", None))
        .block()
        .await
        .unwrap();
    broker.close().unwrap(); // terminates the Flux

    let events = flux.take(1).collect_list().block().await.unwrap().unwrap();
    assert_eq!(events[0].topic, "orders.created");
# });

```

Deliveries are buffered through a bounded channel; when the downstream consumer falls behind, the newest events are dropped (`onBackpressureDrop`) rather than blocking or failing the publisher — extending "a slow consumer never fails publishers" to the reactive surface.

Production transports

Each transport crate implements the same `Broker` port; swap the constructor and keep every handler. Code against `firefly_eda::Broker` and select the adapter at wiring time.

Crate	Backend	Constructor
<code>firefly-eda-kafka</code>	Apache Kafka	<code>new_kafka_broker(KafkaConfig)?</code>
<code>firefly-eda-rabbitmq</code>	RabbitMQ	<code>RabbitMqBroker::new(RabbitMqBrokerConfig)</code>
<code>firefly-eda-postgres</code>	Postgres outbox	<code>PostgresBroker::new(PostgresConfig::new(dsn))</code>
<code>firefly-eda-redis</code>	Redis Streams	<code>RedisStreamsBroker::connect(RedisConfig::new(url))?</code>

Kafka, for example:

Rust

```

use firefly_eda::{handler, Event};
use firefly_eda_kafka::{new_kafka_broker, KafkaConfig};

# async fn ex() -> firefly_eda::EdaResult<()> {
let broker = new_kafka_broker(KafkaConfig {
    brokers: vec!["kafka:9092".into()],
    client_id: "orders".into(),
    consumer_group: "orders-svc".into(),
    ..Default::default()
})?;

broker
    .subscribe("orders.created", handler(|ev: Event| async move {
        println!("got order {}", ev.id);
        Ok(())
    }))
    .await?;

let ev = Event::new("orders.created", "OrderCreated", "orders-svc", None);
broker.publish(ev).await?;
# Ok(())
# }

```

Redis Streams uses a connect-then-start lifecycle:

Rust

```

use firefly_eda::{handler, Event};
use firefly_eda_redis::{RedisConfig, RedisStreamsBroker};

# async fn ex() -> firefly_eda::EdaResult<()> {
let broker = RedisStreamsBroker::connect(
    RedisConfig::new("redis://localhost:6379/0")
        .with_streams(["orders"])
        .with_group("orders-svc"),
    )?;
broker.subscribe("orders.*", handler(|ev: Event| async move {
    println!("got {}", ev.event_type);
    Ok(())
})).await?;
broker.start().await?;
# Ok(())
# }

```

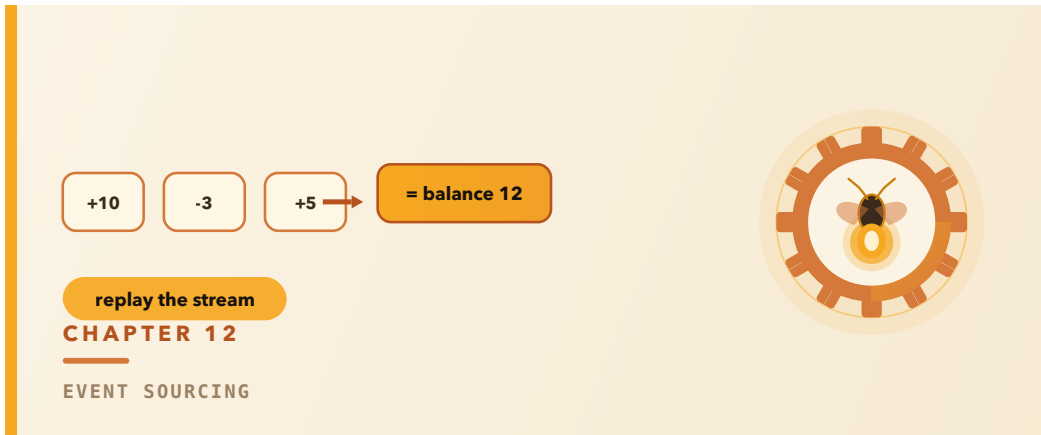
NOTE

The Postgres broker is a **transactional outbox**: events are written in the same transaction as your state change and drained to consumers via **LISTEN / NOTIFY**, giving at-least-once delivery without a separate broker.

Broker health

EventPublisherHealthIndicator adapts any broker implementing the **BrokerHealth** ping probe to a **firefly_observability::Indicator**, surfacing broker liveness on **/ actuator/health** under the **eventPublisher** id.

For durable, replayable event history (rather than transient pub/sub), the next chapter covers [Event Sourcing](#).



CHAPTER 12

Event Sourcing the Ledger

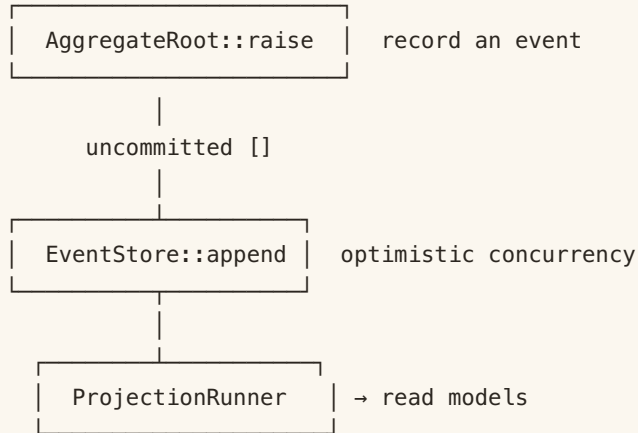
`firefly-eventsourcing` provides the framework's event-sourced aggregate primitives: an `AggregateRoot` that tracks uncommitted events, an `EventStore` with optimistic concurrency, snapshots, projections, a global cross-aggregate stream, a transactional outbox, and multi-tenancy. Where the [DDD chapter](#)'s events were transient (collected for post-commit publication), event-sourced events are the *source of truth* — state is rebuilt by replaying them.

🌿 SPRING PARITY

This is the `firefly-event-sourcing` starter / Axon-style model: `AggregateRoot.raise`, an `EventStore` with optimistic concurrency, and projections that build read models. The `DomainEvent` JSON is wire-compatible across every port.

The mental model

Text



Raising and appending events

An `AggregateRoot` accumulates `DomainEvent`s as you `raise` them, then you `take_uncommitted` and `append` them to the store. `append` enforces optimistic concurrency: pass the version you loaded, and a concurrent writer's append fails with `EventSourcingError::Concurrency`:

Rust

```

use firefly_event sourcing::{AggregateRoot, EventStore, MemoryEventStore};

#[tokio::main]
async fn main() {
    let store = MemoryEventStore::new();

    let mut user = AggregateRoot::new("u1", "User");
    user.raise("UserCreated", br#"{"name":"alice"}"#);
    user.raise("UserRenamed", br#"{"name":"bob"}"#);

    let events = user.take_uncommitted();
    // expected_version 0 -> this is a brand-new aggregate.
    if let Err(err) = store.append(&user.id, 0, events).await {
        eprintln!("append failed (raced): {err}");
    }

    assert_eq!(store.load("u1").await.unwrap().len(), 2);
}

```

The `EventStore` port:

Rust

```

#[async_trait]
pub trait EventStore: Send + Sync {
    async fn append(&self, aggregate_id: &str, expected_version: i64,
        events: Vec<DomainEvent>) -> Result<(), EventSourcingError>;
    async fn load(&self, aggregate_id: &str) -> Result<Vec<DomainEvent>,
        EventSourcingError>;
    async fn load_after(&self, aggregate_id: &str, since_version: i64)
        -> Result<Vec<DomainEvent>, EventSourcingError>;
    async fn stream_all(&self, after_event_id: Option<&str>, limit: usize,
        tenant: Option<&str>)
        -> Result<Vec<StreamedEvent>, EventSourcingError>;
}

```

The default is `MemoryEventStore`; `SqlEventStore` backs it with a SQL store over the `firefly-transactional` `Database` port for production.

Typed aggregates and the repository

For real aggregates, implement `EventSourcedAggregate` — a typed `apply_event` that mutates state per event, plus optional snapshot serialization — and let `EventSourcedRepository` tie `load` (snapshot + replay) and `save` (append + snapshot policy) together:

Rust

```

use firefly_event sourcing::{
    AggregateRoot, DomainEvent, EventSourcedAggregate, EventSourcedRepository,
    EventSourcingError, MemoryEventStore,
};
use std::sync::Arc;

#[derive(Default)]
struct Wallet { root: AggregateRoot, balance: i64 }

impl EventSourcedAggregate for Wallet {
    const AGGREGATE_TYPE: &'static str = "Wallet";
    fn root(&self) -> &AggregateRoot { &self.root }
    fn root_mut(&mut self) -> &mut AggregateRoot { &mut self.root }
    fn apply_event(&mut self, event: &DomainEvent) -> Result<(),
EventSourcingError> {
        if event.event_type == "Credited" {
            let amount: i64 = serde_json::from_slice(&event.payload)
                .map_err(|e| EventSourcingError::Projection(e.to_string()))?;
            self.balance += amount;
        }
        Ok(())
    }
}

# async fn ex() -> Result<(), EventSourcingError> {
    let repo =
EventSourcedRepository::<Wallet>::new(Arc::new(MemoryEventStore::new()));

    let mut w = Wallet::default();
    w.root_mut().raise("Credited", b"500");
    repo.save(&mut w).await?; // append uncommitted

    let reloaded = repo.load(&w.root.id).await?; // snapshot + replay
    assert!(reloaded.is_some());
    # Ok(())
    # }

```

`EventSourcedRepository::with_snapshots(store, snapshots, interval)` enables periodic state captures so rehydration does not replay the entire history.

Projections – building read models

A `Projection` is a read-side handler. Register projections on a `ProjectionRunner` and replay an aggregate's events through them:

Rust

```
use std::sync::Arc;
use firefly_event sourcing::{FunctionProjection, ProjectionRunner};

let runner = ProjectionRunner::new();
runner.register(Arc::new(FunctionProjection::new("balances", |event| async move {
    // update a read-model row from the event ...
    Ok(())
})));

runner.replay(&store, "u1").await?; // replay one aggregate's stream
```

The global stream

`EventStore::stream_all` exposes the global, cross-aggregate, ordered event stream with a resumable cursor — the engine for read models that span many aggregates. The runner consumes it in batches, at-least-once and in-order:

Rust

```
// Drive one batch; returns the next cursor + any per-event error.
let (next_cursor, err) = runner
    .drive_once(&store, None, 100, None)
    .await?;

// Or replay the whole global stream from a start cursor.
let cursor = runner.replay_all(&store, None, 100, None).await?;
```

The transactional outbox

`TransactionalOutbox` gives at-least-once delivery of stored events to a broker. A writer `enqueue`s a `DomainEvent`; a background relay polls and forwards each pending record to an `OutboxSink`, retrying up to `max_attempts`. The default `EdaSink` bridges each `DomainEvent` to a `firefly_eda::Event` and publishes it:

Rust

```
use std::sync::Arc;
use firefly_eventsourcing::{EdaSink, TransactionalOutbox};

let outbox = TransactionalOutbox::new(Arc::new(EdaSink::new(broker)))
    .with_max_attempts(5);

outbox.enqueue(some_event).await;           // a writer enqueues
outbox.start().await;                       // background relay forwards + retries
// ... later
let dead = outbox.dead_letters().await;     // exhausted records, for inspection
outbox.stop().await;
```

Exhausted records become dead letters — excluded from the publish loop and surfaced for inspection or manual retry.

Schema evolution – upcasters

`EventUpcaster` migrates events on the `read` paths only, so consumers always observe current-schema events while the stored history stays untouched:

Rust

```
use std::sync::Arc;
use firefly_event sourcing::{EventUpcaster, MemoryEventStore};

let store = MemoryEventStore::with_upcasters(vec![Arc::new(MyUpcaster)]);
// every event returned by load / load_after passes through applicable upcasters
```

Multi-tenancy

An optional `DomainEvent::tenant_id` (stamped from `AggregateRoot::with_tenant`, persisted and filterable, omitted from JSON when `None`) is threaded through `append` / `load` / `stream_all`, so one store serves many tenants with per-tenant isolation on the global stream.

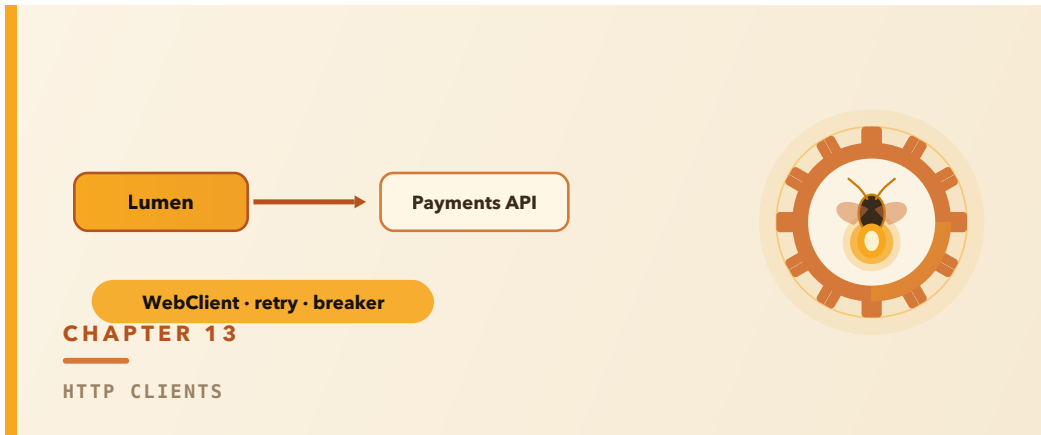
When a business process spans multiple aggregates or services and needs compensation, reach for the orchestration engines. Continue to [Sagas, Workflows & TCC](#).



PART IV

Into Microservices





CHAPTER 13

HTTP Clients & Calling Other Services

`firefly-client` provides two HTTP clients that share the same automatics — default `Accept` / `Content-Type`, correlation-id and W3C trace-context propagation, and RFC 7807 problem decode into a typed `FireflyError`:

- the eager `RestClient` (built with `RestBuilder`) — an `async fn` that awaits a `Result`, with a built-in retry budget;
- the reactive `WebClient` — the Rust analog of WebFlux's `WebClient`, whose terminal operators hand back `Mono` / `Flux`.

The crate also ships scaffolds for SOAP, gRPC, GraphQL, and WebSocket clients.

🌿 SPRING PARITY

`RestClient` is `RestTemplate` / `RestClient`; `WebClient` is the WebFlux `WebClient` — fluent builder, `body_to_mono` / `body_to_flux` terminals, `exchange` for the raw response.

The eager `RestClient`

Build a client with `RestBuilder`, then call `request` with a method, path, and optional body. The retry budget, timeout, and default headers are configured on the builder:

Rust

```
use std::time::Duration;
use firefly_client::RestBuilder;
use http::Method;
use serde::{Deserialize, Serialize};

#[derive(Serialize)]
struct CreateOrder { customer: String }
#[derive(Deserialize)]
struct Order { id: String, customer: String }

#[tokio::main]
async fn main() {
    let client = RestBuilder::new("https://api.example.com")
        .with_header("X-Tenant", "acme")
        .with_timeout(Duration::from_secs(5))
        .with_retries(3)
        .build();

    let req = CreateOrder { customer: "acme".into() };
    match client.request:::<_, Order>(Method::POST, "/orders", Some(&req)).await
    {
        Ok(order) => println!("created {} for {}", order.id, order.customer),
        Err(err) => {
            // Upstream RFC 7807 problems are decoded into a typed FireflyError.
            if let Some(fe) = err.as_firefly() {
                eprintln!("upstream {}: {}", fe.status, fe.detail);
            }
        }
    }
}
```

A non-2xx `application/problem+json` response is decoded into a `FireflyError`, so an upstream failure carries the upstream's status and detail straight through your service's own error stack.

The reactive `WebClient`

The reactive client returns `Mono` / `Flux`, so an outbound call drops straight into a reactive pipeline and composes end-to-end with the `NdJson` / `Sse responders`. The fluent chain is the `WebFlux` spelling:

Rust

```

use firefly_client::WebClientBuilder;
use http::Method;
use serde::{Deserialize, Serialize};

#[derive(Serialize)]
struct CreateOrder { customer: String }
#[derive(Deserialize)]
struct Order { id: String }
#[derive(Deserialize)]
struct Tick { seq: u64 }

#[tokio::main]
async fn main() {
    let client = WebClientBuilder::new("https://api.example.com")
        .with_header("X-Tenant", "acme")
        .build();

    // body_to_mono - a single value -> Mono<Order>.
    let _order = client
        .method(Method::POST)
        .uri("/orders")
        .body(&CreateOrder { customer: "acme".into() })
        .retrieve()
        .body_to_mono:::<Order>();

    // body_to_flux - a streamed NDJSON OR SSE body, decoded lazily
    // element-by-element with backpressure.
    let _ticks = client
        .get()
        .uri("/ticks")
        .header("Accept", "application/x-ndjson")
        .retrieve()
        .body_to_flux:::<Tick>();

    // exchange - raw status + headers without raising on a non-2xx.
    let _resp = client.get().uri("/health").retrieve().exchange();
}

```

The terminal operators:

Operator	Returns	Behaviour
<code>body_to_mono::<T>()</code>	<code>Mono<T></code>	the whole body decoded as one <code>T</code>
<code>body_to_flux::<T>()</code>	<code>Flux<T></code>	a streamed NDJSON/SSE body, element-by-element
<code>exchange()</code>	<code>Mono<WebClientResponse></code>	the raw status + headers + body, no raise

Streaming semantics

`body_to_flux` consumes the byte stream chunk-by-chunk and decodes one element per frame, lazily and with backpressure — a slow downstream throttles the producer, and `.take(n)` stops pulling early. The decoder is chosen from the response `ContentType`:

- `application/x-ndjson` (and any non-SSE type) → one JSON document per newline-terminated line;
- `text/event-stream` → SSE frames separated by a blank line; the `data:` lines are concatenated and comment / `event:` / `id:` lines are ignored.

A malformed element terminates the stream with a decode `FireflyError` (Reactor's first-error-is-terminal semantics).

Inspecting the raw response

`exchange()` hands back a `WebClientResponse` without raising on a non-2xx, so you can inspect it and decide:

Rust

```

let resp = client.get().uri("/
health").retrieve().exchange().block().await?.unwrap();
if resp.is_success() {
    let body: serde_json::Value = resp.body_json()?;
} else if let Some(problem) = resp.problem() {
    // a decoded RFC 7807 FireflyError, if the body was a problem document
}

```

No baked-in retry **REACTOR PARITY**

Unlike `RestBuilder::with_retries`, the `WebClient` has **no** retry budget. Compose retries on the returned publisher with `Mono::retry` / `Mono::retry_backoff`, exactly as WebFlux composes `retryWhen(..)` rather than baking retry into the client:

Rust

```

use firefly_reactive::{Backoff, Mono};
use std::time::Duration;

let order = Mono::retry_backoff(
    || client.get().uri("/orders/o1").retrieve().body_to_mono::<Order>(),
    Backoff::new(3, Duration::from_millis(100)),
);

```

Composing with resilience

Both clients are deliberately small. For circuit breaking, rate limiting, or bulkheads, wrap calls in `firefly-resilience` decorators (covered in [Caching](#) and applied the same way to outbound calls):

Rust

```

use firefly_resilience::{CircuitBreaker, CircuitConfig};

// CircuitBreaker::execute returns the operation's value (Result<T, _>), so the
// guarded call still yields the Order. (Chain::execute is for guarded ops whose
// value you discard – it returns Result<(), _>.)
let breaker = CircuitBreaker::new(CircuitConfig::default());

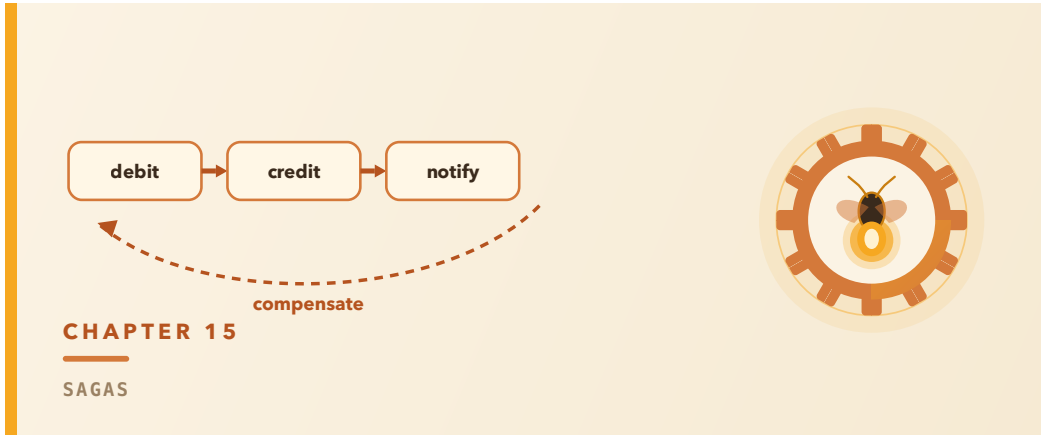
let order = breaker.execute(|| async {
    client.request::<_, Order>(Method::POST, "/charge", Some(&req)).await
}).await?;

```

Other protocols

The crate ships builders/scaffolds for the protocols a back-office platform needs — SOAP (CXF-style envelope), gRPC, GraphQL, and WebSocket — selected by feature so heavy dependencies stay out of services that do not use them. The REST and GraphQL surfaces are fully wired; SOAP and the streaming protocols are feature-gated.

Outbound calls inherit the caller's correlation id automatically, so a request that fans out to three upstreams stitches together in your traces. The next chapter secures the inbound side. Continue to [Security](#).



CHAPTER 15

Distributed Transactions: Sagas, Workflows & TCC

`firefly-orchestration` ships the three classic distributed-transaction engines every Firefly platform agrees on. Each composes async steps, runs as a plain future on the caller's task, applies a per-step retry policy, threads a typed context blackboard, and respects cooperative cancellation.

Engine	Topology	Compensation
<code>Saga</code>	Sequential steps	Reverse-order, configurable policy
<code>Workflow</code>	DAG with parallel branches	Reverse-order, configurable policy
<code>Tcc</code>	Try-all then Confirm-all	Cancel-tried-on-Try-failure

 SPRING PARITY

This is the `firefly-common-domain` orchestration model: orchestrated multi-step processes with compensation, the same step/compensation shapes you know from the JVM, expressed as async closures.

Saga – sequential with compensation

A `Saga` runs steps in order; on any step failure it compensates the completed steps in reverse order. Attach a compensation to each step with `with_compensation`:

Rust

```
use firefly_orchestration::{CompensationPolicy, Saga, SagaStatus, Step};

let saga = Saga::new("checkout")
    .policy(CompensationPolicy::BestEffort) // or StopOnError
    .step(
        Step::new("reserve", || async { Ok(()) })
            .with_compensation(|| async { Ok(()) }),
    )
    .step(
        Step::new("charge", || async { Ok(()) })
            .with_compensation(|| async { Ok(()) }),
    )
    .step(Step::new("ship", || async { Ok(()) }));

let outcome = tokio::runtime::Runtime::new()
    .unwrap()
    .block_on(saga.run())
    .expect("saga completes");

assert_eq!(outcome.status, SagaStatus::Completed);
assert_eq!(outcome.steps_executed, ["reserve", "charge", "ship"]);
```

On success the outcome's `status` is `Completed` and `steps_executed` lists the steps that ran. On failure, `run` returns a `SagaFailure` carrying both the error and the fully-populated `Outcome` (with `steps_rolled` naming the compensations that ran, reverse order). The status is then `Compensated` (rollback succeeded) or `Failed`.

`CompensationPolicy`:

- `BestEffort` (default) — log and continue compensating remaining steps even if one compensation fails.
- `StopOnError` — abort rollback at the first compensation failure and surface a `SagaError::Compensation` wrapping the original.

Passing data between steps

A step built with `Step::with_context(name, |ctx| ...)` reads the typed `StepContext` blackboard so a later step can consume earlier outputs. Run the saga with `run_with_context(&ctx)`. Per-step `with_retry(policy)` adds max attempts, exponential backoff, jitter, and a per-attempt timeout.

Workflow – a DAG

A `Workflow` is a DAG of `Node`s with `depends_on` declarations. Independent nodes run concurrently within a wave; the first node error short-circuits the run:

Rust

```

use firefly_orchestration::{Node, Workflow};

let workflow = Workflow::new("approval")
    .node(Node::new("credit-check", || async { Ok(()) }))
    .node(Node::new("fraud-scan", || async { Ok(()) }))
    .node(
        Node::new("approve", || async { Ok(()) })
            .depends_on(["credit-check", "fraud-scan"]),
    );

let result = tokio::runtime::Runtime::new()
    .unwrap()
    .block_on(workflow.run());
assert!(result.is_ok());

```

`credit-check` and `fraud-scan` run in parallel; `approve` waits for both. Duplicate node names and unknown dependencies are rejected up-front, and an unreachable node aborts the run (`"no progress (dependency cycle?)"`).

Nodes get the same superpowers as saga steps: `Node::with_compensation` rolls back in reverse completion order under the configurable `CompensationPolicy`, `Node::with_context` reads prior results, `Node::when(expr)` skips a node on a false predicate, and `Node::fire_and_forget` schedules it without blocking the wave.

TCC – Try / Confirm / Cancel

`Tcc` runs a two-phase protocol: Try all participants, then Confirm all on success; on any Try failure, Cancel the tried participants in reverse order (best-effort):

Rust

```

use firefly_orchestration::{Tcc, TccParticipant};

let tcc = Tcc::new("transfer")
    .participant(
        TccParticipant::new("debit", || async { Ok(()) }, || async { Ok(()) })
            .with_cancel(|| async { Ok(()) }),
    )
    .participant(
        TccParticipant::new("credit", || async { Ok(()) }, || async { Ok(()) })
            .with_cancel(|| async { Ok(()) }),
    );

let result = tokio::runtime::Runtime::new()
    .unwrap()
    .block_on(tcc.run());
assert!(result.is_ok());

```

`TccParticipant::new(name, try_fn, confirm_fn)` takes the Try and Confirm phases; `with_cancel` adds the compensation for a failed Try.

Cancellation

All three engines respect a `CancellationToken` — the Rust analog of the Go port's `context.Context` cancellation. Pass one to `run_cancellable(&token)` and cancel it from elsewhere to drain the run:

Rust

```

use firefly_orchestration::CancellationToken;

let token = CancellationToken::new();
let handle = token.clone();
// ... cancel from a timeout or a shutdown signal:
handle.cancel();

let outcome = saga.run_cancellable(&token).await;

```

The engines depend only on `futures`, so any executor (Tokio included) drives them.

Choosing an engine

Need	Engine
Linear process, undo on failure	<code>Saga</code>
Parallel branches that join	<code>Workflow</code>
Reserve-then-commit across resources	<code>Tcc</code>

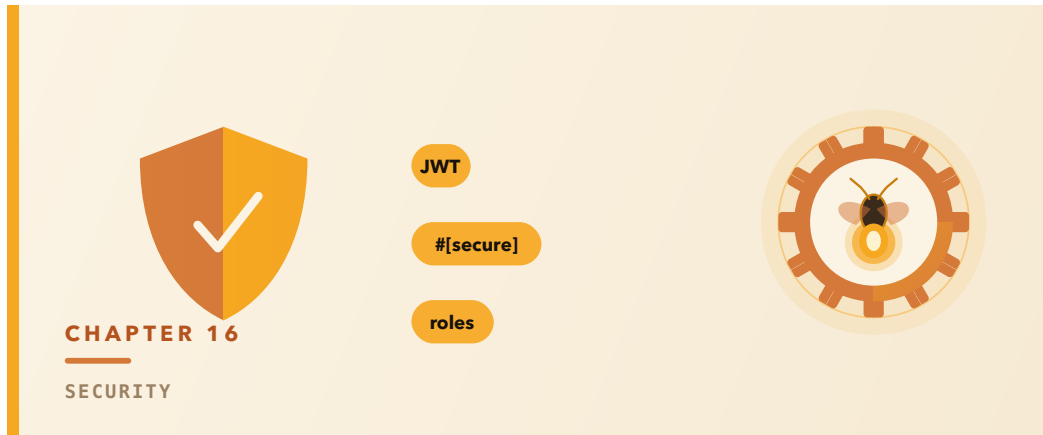
A typical money-transfer is a `Saga` (debit → credit → notify, with debit/credit compensations) or a `Tcc` (try-debit + try-credit, confirm both, cancel on any try failure). A multi-check approval (credit + fraud + KYC, then decision) is a `Workflow`.

To call the external services these steps coordinate, you need an HTTP client. Continue to [HTTP Clients](#).



PART V

Secure, Observe & Ship



CHAPTER 16

Security, Sessions & Identity

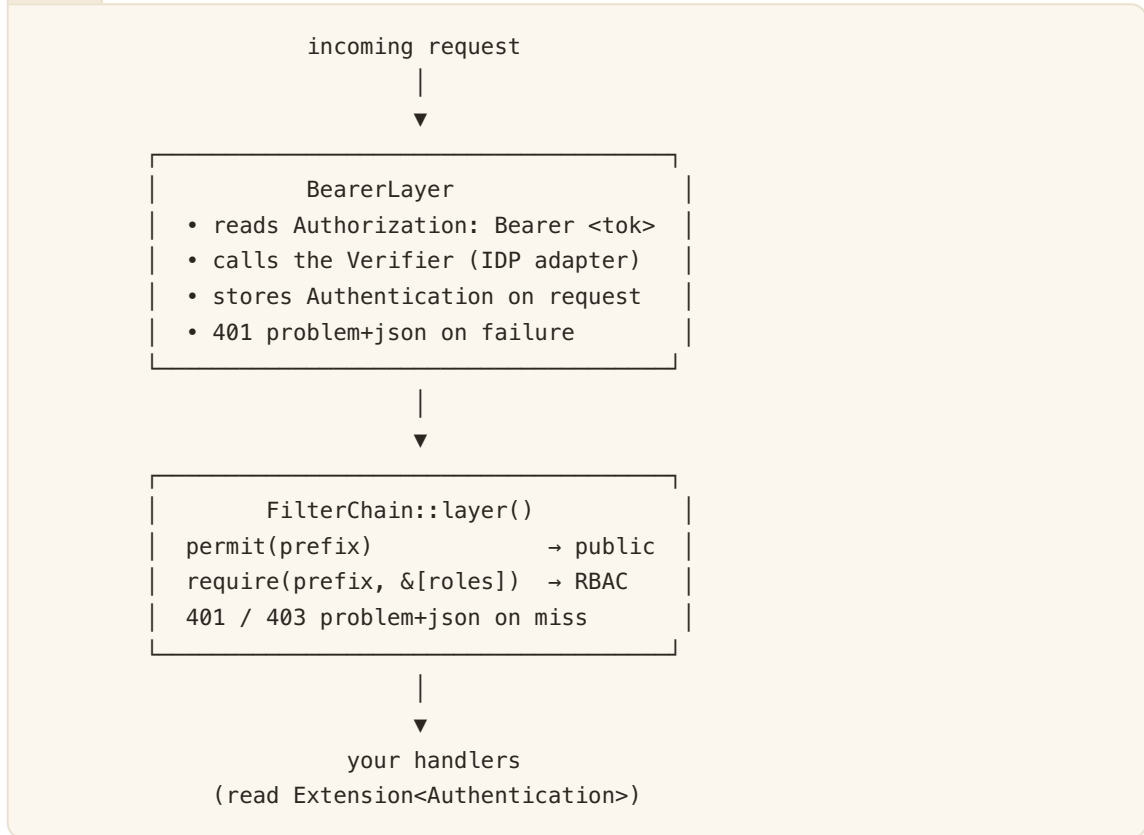
`firefly-security` is the framework's HTTP-layer authentication and authorization tier: a `Verifier` port for token validators, a `BearerLayer` that authenticates requests, a path-prefix RBAC `FilterChain`, JWKS verification, OAuth2 (client registrations, PKCE/OIDC login, an authorization server), a role hierarchy, CSRF, and a bcrypt password encoder.

🌿 SPRING PARITY

This is Spring Security: a resource server (`BearerLayer` + JWKS), URL-based authorization (`FilterChain` ~ `HttpSecurity`), method guards, and an OAuth2 client + authorization server. Wire formats are RFC 7807 problems byte-identical across the ports.

The mental model

Text



Authentication and the Verifier

A **Verifier** validates a bearer token and returns an **Authentication** (principal + username + roles + claims). The **BearerLayer** extracts `Authorization: Bearer <token>`, calls the verifier, and stores the result on the request extensions.

VerifierFn adapts a plain async closure:

Rust

```

use axum::{routing::get, Extension, Router};
use firefly_security::{
    Authentication, BearerConfig, BearerLayer, FilterChain, SecurityError,
    VerifierFn,
};

let verifier = VerifierFn(|token: String| async move {
    if token == "letmein" {
        Ok(Authentication {
            principal: "u1".into(),
            username: "alice".into(),
            roles: vec!["ADMIN".into()],
            ..Default::default()
        })
    } else {
        Err(SecurityError::verification("unknown token"))
    }
});

let chain = FilterChain::new()
    .permit("/actuator/health")
    .permit("/actuator/info")
    .require("/admin/", &["ADMIN"])
    .require("/api/", &["USER", "ADMIN"]);

let app: Router = Router::new()
    .route(
        "/admin/users",
        get(|Extension(auth): Extension<Authentication>| async move {
            format!("hello, {}", auth.username)
        }),
    )
    // axum runs the last-added layer first: bearer authenticates, then the
    // chain authorizes.
    .layer(chain.layer())
    .layer(BearerLayer::new(BearerConfig::new(verifier)));

```

Handlers read the authenticated principal with axum's `Extension`:

Rust

```
use axum::Extension;
use firefly_security::Authentication;

async fn admin_only(Extension(auth): Extension<Authentication>) -> String {
    if !auth.has_any_role(&["ADMIN", "OPERATOR"]) {
        return "forbidden".into();
    }
    format!("welcome {}", auth.username)
}
```

`Authentication::has_role`, `has_any_role`, and `has_authority` (which accepts a role name or a fine-grained permission/scope) cover the common checks;

`Authentication::anonymous()` is the unauthenticated principal.

Wiring an IDP adapter as the Verifier

An IDP adapter (`firefly_idp::Adapter`) exposes `validate(token) -> User`. Adapt it to a `Verifier`:

Rust

```

use firefly_security::{Authentication, SecurityError, VerifierFn};

let verifier = VerifierFn(move |token: String| {
    let idp = idp_adapter.clone();
    async move {
        let u = idp.validate(&token).await
            .map_err(|e| SecurityError::verification(e.to_string()))?;
        Ok(Authentication {
            principal: u.id,
            username: u.username,
            roles: u.roles,
            ..Default::default()
        })
    }
});

```

The IDP vendor adapters — Keycloak (OIDC + admin REST), Azure AD (Microsoft Graph), AWS Cognito (JSON API + SigV4), and the self-hosted `firefly-idp-internal-db` (bcrypt + HS256 JWT) — all satisfy the same `Adapter` port, so this wiring is identical regardless of provider.

URL authorization – the FilterChain

`FilterChain` matches request paths against rules. The Go-parity surface is prefix-based (`permit` / `permit_method` / `require`); the pyfly-parity surface adds an fnmatch-style glob DSL with `deny-by-default`:

Rust

```
use firefly_security::FilterChain;

let chain = FilterChain::new()
    .permit_pattern("/actuator/health")
    .permit_pattern("/public/**")
    .require_pattern("/api/admin/**", &["ADMIN"])
    .require_authority("/api/reports/**", "reports:read")
    .authenticated("/api/**")
    .any_request_authenticated(); // re-open the unmatched tail (or deny it)
```

⚠ WARNING

Once any rule is declared, the chain is **fail-closed**: a request matching no rule is rejected with 403 (Spring Security 6 semantics). Re-open the unmatched tail explicitly with `any_request_permit` / `any_request_authenticated` / `any_request_deny`. A chain with *no* rules at all is a no-op and passes everything through (never a blanket lockout).

Role hierarchies and method guards

`RoleHierarchy` parses `"ADMIN > USER"` specs (transitive `expand`), consulted by the `FilterChain` via `with_role_hierarchy` so granting `ADMIN` implies `USER`.

For method-level checks (the replacement for `@PreAuthorize` SpEL strings), the `guards` module composes typed `AuthorizationGuard` predicates with `and` / `or` / `not`:

Rust

```
use firefly_security::guards::{has_authority, has_role};

let guard = has_role("ADMIN").or(has_authority("orders:write"));
// authorize is a method on the guard; pass the optional Authentication.
guard.authorize(Some(&auth))?; // 401 if no principal, 403 if predicate false
```

JWKS verification

`JwksVerifier` is a JWKS resource-server `Verifier` (RS256, kid cache, `iss` / `aud` validation, `exp` required). It maps claims to an `Authentication`: `sub` → principal, `preferred_username` / `name` → username, a flat `roles` claim or Keycloak `realm_access.roles` → roles, and `permissions` or a space-separated `scope` → authorities. Drop it in wherever a `Verifier` is expected.

OAuth2

The `oauth2` module covers both sides:

- **Client / login.** `ClientRegistration` with `google` / `github` / `keycloak` presets and an `InMemoryClientRegistrationRepository`. `OAuth2LoginHandler::router()` mounts the authorization-code flow — `state` + `nonce` + PKCE S256, OIDC id-token validation against the provider JWKS, userinfo fallback, and session-fixation-safe id rotation.
- **Authorization server.** `AuthorizationServer` issues HS256 JWTs for the `client_credentials` and `refresh_token` grants with refresh-token rotation and constant-time client authentication. The RFC-6749 error codes (`INVALID_CLIENT`, `UNAUTHORIZED_CLIENT`, `INVALID_GRANT`, ...) match the other ports exactly.
- **Token storage.** A `TokenStore` port with `InMemoryTokenStore`, `RedisTokenStore` (`SET ... EX`), and `PostgresTokenStore` implementations.

CSRF

For cookie-session browser flows, `CsrfLayer` implements the double-submit cookie pattern: an `XSRF-TOKEN` cookie compared against an `X-XSRF-TOKEN` header on unsafe methods, safe-method pass-through with cookie refresh, a `Authorization: Bearer`

bypass (token clients do not need CSRF), and a 403 problem on mismatch. Mint and validate tokens directly with `generate_csrf_token()` / `validate_csrf_token(cookie, header)`.

Password hashing

`BcryptPasswordEncoder` is a standalone credential hash/verify primitive, usable independently of any IDP (a worker, a custom user store, a rotation job):

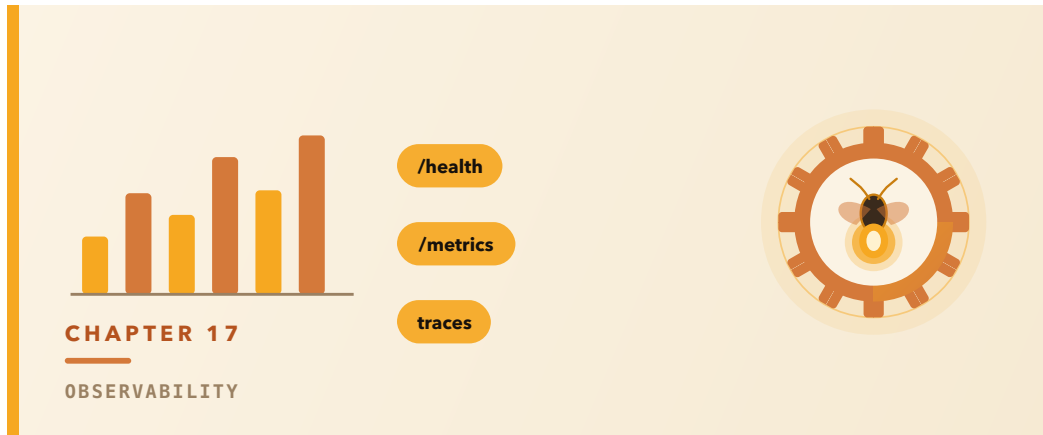
Rust

```
use firefly_security::BcryptPasswordEncoder;

let enc = BcryptPasswordEncoder::new(); // work factor 12 (the default)
let hash = enc.hash("s3cret").unwrap();
assert!(enc.verify("s3cret", &hash).unwrap());
assert!(!enc.verify("wrong", &hash).unwrap());
```

The `$2b$` hashes interchange with the `firefly-idp-internal-db` adapter and the Java/.NET/Go ports.

A secure service is also an observable one. The next chapter covers logging, metrics, tracing, health, and the admin dashboard. Continue to [Observability](#).



CHAPTER 17

Observability & Health

Firefly is **observable by default**: `Core::new` wires structured logging with correlation enrichment, a health composite, a metrics registry, and the actuator endpoints. This chapter covers the three orthogonal concerns of `firefly-observability` — logging, health, and the banner — plus the `firefly-actuator` management surface and the `firefly-admin` dashboard.

🌿 SPRING PARITY

This is `firefly-otel-spring-boot-starter` + Spring Boot Actuator: structured logs with MDC-style correlation, `/actuator/health` + `/actuator/metrics`, and a Spring-Boot-Admin-style dashboard.

Structured logging

`firefly-observability` provides a `tracing` layer (`CorrelationLayer`) that formats every event as one JSON (or text/console) line and auto-enriches it with the correlation id from the kernel's task-local scope. `Core::init_logging` installs it globally; the log field names (`time` , `level` , `msg` , `service` , `correlationId`) are identical across the ports, so one pipeline parses everything.

Rust

```
use firefly_observability::{init_logging, LogConfig, LogFormat};
use tracing::Level;

let cfg = LogConfig::new()
    .with_service("orders")
    .with_level(Level::INFO)
    .with_format(LogFormat::Json); // Json | Text | Console (dev renderer)
init_logging(cfg);
```

After that, plain `tracing` macros produce enriched lines — and any fields you record on an enclosing span are merged into each event:

Rust

```
use tracing::info;

let span = tracing::info_span!("create_order", order_id = "o1");
let _g = span.enter();
info!(customer = "alice", "order created"); // carries order_id + correlationId
```

The `Core` typically owns the `LogConfig` (preset to the app name); you usually just call `core.init_logging()` and use `tracing` macros throughout.

Correlation propagation

Because the correlation id lives in a task-local scope (set by the `CorrelationLayer` HTTP middleware — see [Your First HTTP API](#)), it flows automatically into:

- every log line (via the logging layer above);
- every event you publish (`Event::new` stamps `correlationId`);
- every outbound client call (`RestClient` / `WebClient` propagate it as a header plus W3C `traceparent` / `tracestate`).

So a request that fans out to events and upstream calls stitches together under one correlation id with no manual threading.

Health

A health `Indicator` is an async probe returning a `HealthResult` (status + message + details + duration + time). A `Composite` aggregates indicators into the canonical rollup:

Rust

```
use firefly_observability::{Composite, HealthResult, IndicatorFn, Status};

let composite = Composite::new();
composite.add(IndicatorFn::new("db", || async {
    if db_reachable().await { HealthResult::up() } else {
        HealthResult::down("unreachable") }
}));

let (status, details) = composite.check_all().await;
assert!(matches!(status, Status::Up | Status::Degraded | Status::Down));
```

The rollup is `DOWN` if any indicator is `DOWN`, else `DEGRADED` if any is `DEGRADED`, else `UP` (`UNKNOWN` is neutral). `Core` carries an actuator `HealthComposite` with a default cache indicator; bridge an observability `Indicator` onto it with `core.add_observability_indicator(..)`, and broker liveness with the `EventPublisherHealthIndicator`.

The startup banner

`Core::print_banner()` emits the ASCII Firefly banner — the red script-figlet, a `::` `Firefly Framework for Rust :: (v26.6.3)` tagline, the license line, and the app / starter / runtime / active-profiles metadata. `core.banner()` returns it as a `String` for tests.

The actuator surface

`firefly-actuator` exposes the management endpoints, mounted by `core.actuator_router(info_contributors)` — typically on a **separate listener** so `/actuator/*` never leaks onto the public network:

Endpoint	Returns
<code>/actuator/health</code>	the composite rollup (+ liveness/readiness probes)
<code>/actuator/info</code>	app metadata + your info contributors
<code>/actuator/metrics</code>	labeled Micrometer-parity metrics
<code>/actuator/env</code>	masked, origin-attributed property sources
<code>/actuator/tasks</code>	scheduled-task descriptors
<code>/actuator/version</code>	the running version
<code>/actuator/loggers [{name}]</code>	runtime log-level control (when wired)
<code>/actuator/httpexchanges</code>	recent HTTP exchanges (when wired)
<code>/actuator/threaddump</code>	a thread/task dump
<code>/actuator/refresh</code>	reload config (the <code>ReloadableConfig</code> hook)

An info contributor adds a section to `/actuator/info`:

Rust

```
use firefly_starter_core::InfoContributor;

let contributor: InfoContributor = Box::new(|| {
    let mut m = serde_json::Map::new();
    m.insert("orders".into(), serde_json::json!({ "store": "in-memory" }));
    m
});
let admin = core.actuator_router(vec![contributor]);
```

HTTP server metrics

When you set the `request_metrics` knob on `CoreConfig`, the `MetricsLayer` records the labeled `http_server_requests_seconds` timer and a companion `_max` gauge per request, tagged `method` / templated `uri` (the axum matched path) / `status` / `outcome` / `exception`, and bridges them onto the actuator `MetricRegistry` for `/actuator/metrics`. A clean request carries `exception="None"`.

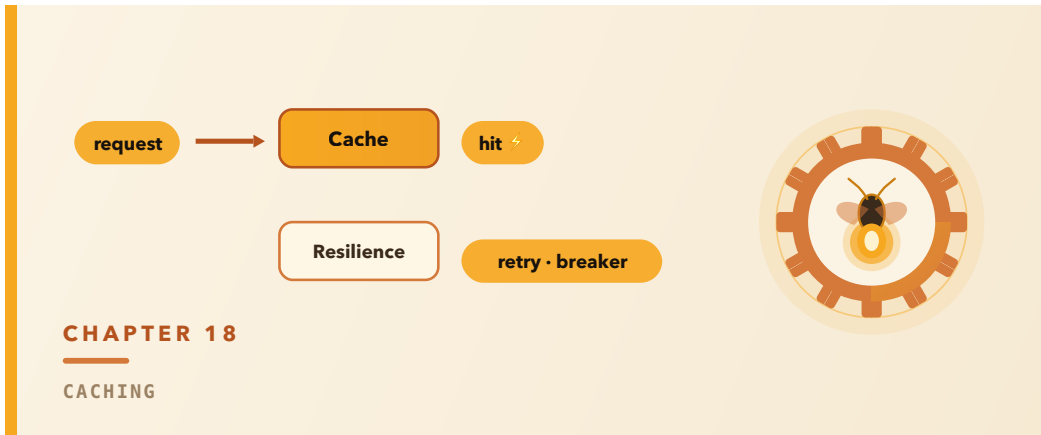
Tracing / OpenTelemetry

The crate exposes the building blocks that compose with the `tracing` ecosystem and propagates W3C trace-context (`traceparent` / `tracestate`) on the HTTP edges and outbound client calls. OpenTelemetry SDK wiring — exporters, sampling, resource attributes — is left to your `main.rs`, where you add your preferred OTel `tracing` layer alongside the `CorrelationLayer`.

The admin dashboard

`firefly-admin` mounts a Spring-Boot-Admin-style embedded dashboard: a single-page UI (overview / health / metrics / loggers / mappings / caches / scheduled tasks / traces / CQRS / transactions / beans / config / instances), a JSON API over `firefly-actuator`, and SSE live streams. It builds its `AdminDeps` from the public `Core` accessors (`cqrs_bus()`, `scheduler()`, `health_composite()`, `metric_registry()`, `http_exchanges()`, `loggers()`), so wiring it is a few lines once you have a `Core`.

With your service observable, the next chapter adds background work and outbound messaging. Continue to [Scheduling & Notifications](#).



CHAPTER 18

Caching & Resilience

`firefly-cache` exposes a single port — `Adapter` — and ships in-process, no-op, and fallback implementations plus a typed memoisation wrapper. Code against the port and select the backend (in-memory, Redis, Postgres) at wiring time. This chapter also covers `firefly-resilience`, the decorators that protect the calls a cache miss falls back to.

🌿 SPRING PARITY

`Adapter` is the `Cache` abstraction; `Typed::get_or_set` is `@Cacheable`; the resilience decorators are `Resilience4j` (circuit breaker, rate limiter, bulkhead, timeout) composed for reactive Rust.

The cache port

Every consumer — CQRS query cache, session store, OAuth2 token store — depends on `Arc<dyn cache::Adapter>`, never on a concrete client. The port:

Rust

```
#[async_trait]
pub trait Adapter: Send + Sync {
    async fn get(&self, key: &str) -> Result<Vec<u8>, CacheError>;
    async fn set(&self, key: &str, value: &[u8], ttl: Option<Duration>) ->
    Result<(), CacheError>;
    async fn delete(&self, key: &str) -> Result<(), CacheError>;
    async fn clear(&self) -> Result<(), CacheError>;
    async fn set_if_absent(&self, key, value, ttl) -> Result<bool, CacheError>;
    async fn exists(&self, key: &str) -> Result<bool, CacheError>;
    async fn delete_prefix(&self, prefix: &str) -> Result<u64, CacheError>;
    async fn health_check(&self) -> Result<(), CacheError>;
}
```

A miss is `CacheError::NotFound`; `ttl: None` (or zero) means no expiry.

Implementation	Backing	Use
<code>MemoryAdapter</code>	<code>HashMap</code> + <code>RwLock</code>	in-process, TTL-aware, the default
<code>NoOpAdapter</code>	none	tests / disabled-cache configs
<code>FallbackAdapter { primary, secondary }</code>	composite	primary-then-secondary, writes to both
<code>RedisAdapter (firefly-cache-redis)</code>	Redis (RESP)	distributed cache

Typed memoisation

The `Typed<T>` wrapper serializes values as `serde_json` bytes (wire-compatible with the other ports) and gives you `get_or_set` — the memoisation primitive that consults the cache, calls the loader on a miss, persists, and returns the value. A caching error never masks a successful loader result:

Rust

```

use std::sync::Arc;
use std::time::Duration;
use firefly_cache::{MemoryAdapter, Typed};

#[derive(serde::Serialize, serde::Deserialize)]
struct Order { id: String }

#[tokio::main(flavor = "current_thread")]
async fn main() -> Result<(), firefly_cache::CacheError> {
    let cache = Arc::new(MemoryAdapter::new());
    let typed: Typed<Order> = Typed::new(cache);

    let order = typed
        .get_or_set("order:42", Some(Duration::from_secs(60))), || async {
            // loaded from the repository on a miss
            Ok(Order { id: "42".into() })
        }
        .await?;
    assert_eq!(order.id, "42");
    Ok(())
}

```

Choosing and composing backends

`Core::new` uses `MemoryAdapter` by default; pass an `Arc<dyn cache::Adapter>` in `CoreConfig.cache` to swap it. For high availability, compose Redis with an in-process fallback so a Redis blip degrades to local caching instead of failing:

Rust

```

use std::sync::Arc;
use firefly_cache::{FallbackAdapter, MemoryAdapter};

// Tries Redis first; on a transport error or miss, falls through to memory
// and writes to both.
let cache = FallbackAdapter::new(redis_adapter, Arc::new(MemoryAdapter::new()));

```

Because everything downstream depends on the port, swapping the backend changes one constructor — your handlers, the CQRS query cache, and the session store are untouched.

Resilience decorators

`firefly-resilience` guards the calls a cache miss falls through to (and any outbound call). Four primitives, composable into a `Chain`:

Decorator	Guards against	Error on trip
<code>CircuitBreaker</code>	cascading failure of a slow / failing dep	<code>ResilienceError::CircuitOpen</code>
<code>RateLimiter</code>	outbound rate overrun (token bucket)	<code>ResilienceError::RateLimited</code>
<code>Bulkhead</code>	resource exhaustion from runaway concurrency	<code>ResilienceError::BulkheadFull</code>
<code>Timeout</code>	stuck calls	<code>ResilienceError::Timeout</code>

A `Chain` composes them into a single guarded call:

Rust

```

use std::{sync::Arc, time::Duration};
use firefly_resilience::{Bulkhead, Chain, CircuitBreaker, CircuitConfig,
Timeout};

# async fn ex() -> Result<(), firefly_resilience::ResilienceError> {
let breaker = Arc::new(CircuitBreaker::new(CircuitConfig::default()));

let guarded = Chain::new()
    .with(Timeout::new(Duration::from_secs(2))) // per-call deadline
    (outermost)
    .with(breaker) // open the circuit on
    repeated failures
    .with(Bulkhead::new(20)); // cap concurrent in-flight
    calls

guarded.execute(|| async {
    // the protected operation – a cache loader, an upstream call, ...
    Ok(())
}).await?;
# Ok(())
# }

```

Each decorator also stands alone:

Rust

```

use std::time::Duration;
use firefly_resilience::{Bulkhead, CircuitBreaker, CircuitConfig, RateLimiter,
Timeout};

let cb = CircuitBreaker::new(CircuitConfig::default());
let _ = cb.execute(|| async { charge().await }).await;

let rl = RateLimiter::new(100.0, 200);           // 100 rps, burst 200
let _ = rl.execute(|| async { call().await }).await;

let bh = Bulkhead::new(20);
let _ = bh.try_execute(|| async { call().await }).await; // non-blocking;
BulkheadFull if full

let to = Timeout::new(Duration::from_secs(2));
let _ = to.execute(|| async { slow_call().await }).await;

```

A complete read path

The pieces fit together as a cache-aside read protected by a circuit breaker:

Rust

```

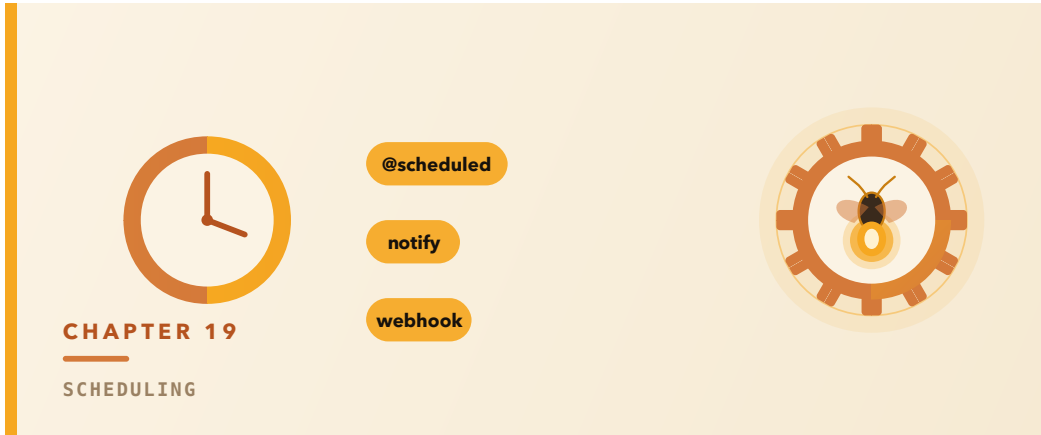
use std::sync::Arc;
use std::time::Duration;
use firefly_cache::{MemoryAdapter, Typed};
use firefly_resilience::{CircuitBreaker, CircuitConfig};

let typed: Typed<Order> = Typed::new(Arc::new(MemoryAdapter::new()));
let breaker = CircuitBreaker::new(CircuitConfig::default());

let order = typed
    .get_or_set("order:42", Some(Duration::from_secs(60)), || async {
        // the loader is what the circuit protects: an upstream / DB call.
        breaker.execute(|| async { fetch_order_from_db("42").await }).await
            .map_err(|e| firefly_cache::CacheError::Backend(e.to_string()))
    })
    .await?;

```

You now have a fast, resilient read path. The remaining chapters cover shipping: testing against real infrastructure, the CLI, and production deployment. Continue to [Testing](#).



CHAPTER 19

Scheduling, Notifications & Webhooks

Two everyday back-office needs: running work on a timer, and delivering messages (email, SMS, push) through swappable providers. `firefly-scheduling` and `firefly-notifications` cover them with the same port-and-adapter discipline as the rest of the framework.

Scheduling

`firefly-scheduling` is a `Scheduler` that owns Cron, FixedRate, and FixedDelay triggers, runs each task on its own tokio task with panic recovery, and shuts down gracefully on `stop()`.

🌿 SPRING PARITY

This is `@Scheduled`: cron expressions, fixed rate, fixed delay. The `Core` carries a `Scheduler` so you can register tasks without wiring one yourself.

Rust

```

use std::{sync::Arc, time::Duration};
use firefly_scheduling::Scheduler;

# async fn demo() {
let s = Arc::new(Scheduler::new());

// Cron: a 5-field expression (or Spring 6-field with leading seconds).
s.cron("nightly-rollup", "0 2 * * *", || async { Ok(()) }).unwrap();

// FixedRate: fire every period from a fixed anchor (slips on slow runs).
s.fixed_rate("metrics-emit", Duration::from_secs(30), || async { Ok(()) });

// FixedDelay: fire `delay` after the previous run finishes.
s.fixed_delay("cleanup", Duration::from_secs(300), || async { Ok(()) });

// Stop on a signal, then start (blocks until stop()).
let handle = Arc::clone(&s);
tokio::spawn(async move {
    tokio::signal::ctrl_c().await.ok();
    handle.stop();
});
s.start().await; // in-flight runs finish before shutdown
# }

```

Triggers

Trigger	Behaviour
<code>CronTrigger</code>	fires when the local wall clock matches the expression
<code>ZonedCronTrigger</code>	fires per the expression in an IANA time zone
<code>FixedRateTrigger</code>	fires every period from a fixed start anchor (slips)
<code>FixedDelayTrigger</code>	fires the delay after the previous run finished

Cron syntax

The canonical 5-field `minute hour day-of-month month day-of-week`, plus the Spring 6-field form with a leading seconds field (`sec min hour dom month dow`), the Quartz `?` placeholder, and the `@hourly` / `@daily` / `@weekly` / `@monthly` / `@yearly` macros. Each field accepts a literal (`15`), a list (`0,15,30`), a range (`9-17`), a wildcard (`*`), or a step (`*/15` , `9-17/2`). When both day-of-month and day-of-week are restricted, the rule fires when **either** matches (Vixie cron behaviour).

For a specific time zone, use `ZonedCronTrigger` :

Rust

```
use firefly_scheduling::{parse_cron, ZonedCronTrigger};

let expr = parse_cron("0 9 * * MON-FRI").unwrap();
let trigger = ZonedCronTrigger::in_zone(expr, "America/New_York").unwrap();
```

Scheduled tasks surface on `/actuator/tasks` , and you can introspect cron schedules (`CronExpr::next` , `next_n` , `seconds_until_next`) for diagnostics.

Notifications

`firefly-notifications` defines a `Notification` envelope, a `Channel` transport trait, and a `Dispatcher` that routes messages to channels keyed on their `Kind` . The default `MemoryChannel` records every message (for tests); real provider adapters slot in behind the same trait.

🌿 SPRING PARITY

The `Channel` port + `Dispatcher` is the notification sender abstraction; swapping `MemoryChannel` for a real provider is changing one registration, not your code.

Rust

```

use std::sync::Arc;
use firefly_notifications::{Dispatcher, Kind, MemoryChannel, Notification};

#[tokio::main(flavor = "current_thread")]
async fn main() {
    let dispatcher = Dispatcher::new();
    dispatcher.register(Arc::new(MemoryChannel::new(Kind::EMAIL)));
    dispatcher.register(Arc::new(MemoryChannel::new(Kind::SMS)));

    dispatcher
        .dispatch(Notification {
            channel: Kind::EMAIL,
            to: "alice@example.com".into(),
            subject: "Welcome".into(),
            body: "Welcome to Firefly!".into(),
            ..Notification::default()
        })
        .await
        .unwrap();
}

```

The `Dispatcher` routes by the notification's `Kind`; a message for an unregistered kind is a `NotificationError::NoChannel`.

Provider adapters

For production, register a real channel in place of `MemoryChannel` — same trait, real delivery:

Crate	Channel	Backing
<code>firefly-notifications-smtp</code>	email	<code>lettre</code> (real MIME, STARTTLS)
<code>firefly-notifications-twilio</code>	SMS	Twilio (real provider)
<code>firefly-notifications-firebase</code>	push	Firebase Cloud Messaging
<code>firefly-notifications-sendgrid</code>	email	SendGrid (locked port, pending)
<code>firefly-notifications-resend</code>	email	Resend (locked port, pending)

Rust

```

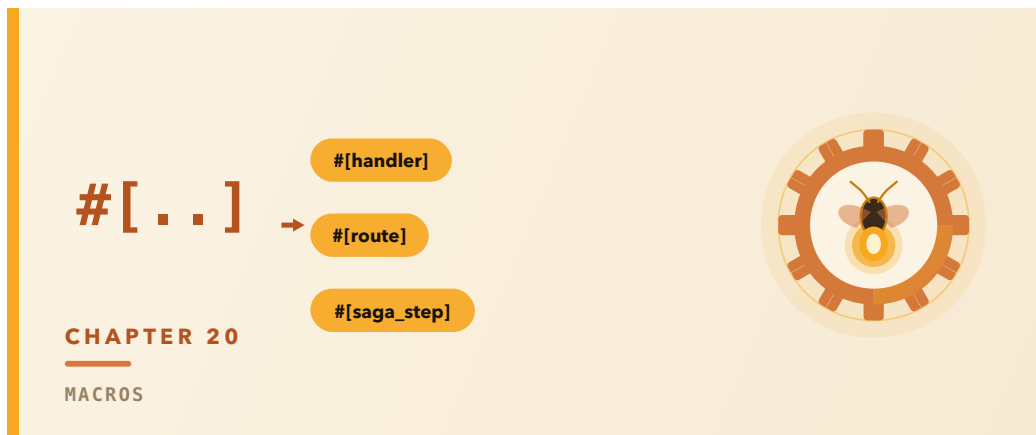
use std::sync::Arc;
use firefly_notifications::{Dispatcher, Kind};

let dispatcher = Dispatcher::new();
dispatcher.register(Arc::new(smtp_email_channel)); // SmtEmailProvider,
Kind::EMAIL
dispatcher.register(Arc::new(twilio_sms_channel)); // Kind::SMS

```

Because every channel is an `Arc<dyn Channel>`, the heavy provider SDKs stay out of services that do not select that channel — code against the `Channel` port, register the concrete adapter at wiring time.

The next chapter covers the cache abstraction and the resilience decorators that protect calls to the providers above. Continue to [Caching](#).



CHAPTER 20

Declarative Services with Macros

Every chapter so far wired the framework by hand — `impl Message for ...`, `bus.register(move |req| ...)`, `Router::new().route(...)`, `Scheduler::fixed_rate(...)`. That is explicit and fully type-safe, and it is exactly what the framework runs. But for the common cases it is also *mechanical*. Version 26.6.3 adds the **declarative layer** — the Rust answer to Spring annotations and the pyfly decorator set — so that the common cases become a *declaration sitting next to the code it describes*, with the wiring generated for you.

It is two crates:

- `firefly` — the one-dependency facade, and
- `firefly-macros` — the `#[derive(...)]` / `#[...]` macros (re-exported through the facade).

One dependency, one prelude

Before, a service listed ten-to-fifteen `firefly-*` crates and imported from each. With the facade you add one dependency:

TOML

```
[dependencies]
firefly = "26.6.3"           # the whole framework + every macro
axum    = "0.7"              # you author axum handlers
serde   = { version = "1", features = ["derive"] }
tokio   = { version = "1", features = ["rt-multi-thread", "macros"] }
```

and glob-import one prelude:

Rust

```
use firefly::prelude::*;
```

That glob brings the whole framework into scope — `Bus`, `Container`, `Scheduler`, `Saga / Step`, `Application / ShutdownHandle`, `Core / CoreConfig`, `WebResult / WebError / problem_response`, `FireflyError / FireflyResult`, `Mono / Flux` — and every macro. There are also ergonomic per-crate aliases (`firefly::cqrs::Bus` instead of `firefly_cqrs::Bus`, one for every runtime crate). `serde` and `axum` are the only two ecosystem crates a Firefly service still writes against directly.

Staying lean

The default `firefly` build pulls only the framework's lean *port* crates — no heavy third-party drivers. Heavy adapters are opt-in cargo features:

TOML

```
firefly = { version = "26.6.3", features = ["data-sqlx", "eda-kafka"] }
```

Feature	Pulls in
<code>data-sqlx</code>	relational repository adapter (Postgres / MySQL / SQLite)
<code>data-mongodb</code>	document repository adapter (MongoDB)
<code>eda-kafka</code> / <code>eda-rabbitmq</code> / <code>eda-redis</code> / <code>eda-postgres</code>	event-broker transports
<code>cache-redis</code> / <code>cache-postgres</code>	cache backends
<code>admin</code>	the admin dashboard
<code>full</code>	all of the above

A minimal `firefly` dependency compiles none of them.

The macros

Macro	On	Generates
<code>#[derive(Command)] /</code> <code>#[derive(Query)]</code>	a message struct	<code>impl Message</code> <code>(#[firefly(validate)] /</code> <code>#[firefly(cache_ttl = "...")])</code>
<code>#[command_handler] /</code> <code>#[query_handler]</code>	<code>async fn(Msg) -></code> <code>Result<R,</code> <code>CqrsError></code>	a <code>register_<fn>(bus)</code> helper
<code>#[derive(Component)] /</code> <code>#[derive(Service)] /</code> <code>#[derive(Repository)]</code>	a struct with <code>#[autowired]</code> fields	a <code>firefly_register(container)</code> method
<code>register_all!(&container,</code> <code>[A, B, ...])</code>	—	calls each type's <code>firefly_register</code>
<code>#[scheduled]</code>	a zero-arg <code>async fn</code>	a <code>schedule_<fn>(scheduler)</code> helper
<code>#[rest_controller] +</code> <code>#[get/post/put/delete/</code> <code>patch]</code>	an <code>impl</code> block	a <code>routes(state) -></code> <code>axum::Router</code>
<code>#[derive(DomainEvent)] /</code> <code>#[derive(AggregateRoot)]</code>	a struct	event-type / aggregate ergonomics
<code>#[event_listener]</code>	<code>async fn(Event) -></code> <code>FireflyResult<()></code>	a <code>subscribe_<fn>(broker)</code> helper

CQRS – messages and handlers

`#[derive(Command)] / #[derive(Query)]` generate the `Message` impl;
`#[firefly(validate)]` makes an empty / zero field fail validation before the handler runs, and `#[firefly(cache_ttl = "...")]` feeds the query cache:

Rust

```

use firefly::prelude::*;
use serde::Serialize;

#[derive(Clone, Serialize, Command)]
pub struct PlaceOrder {
    #[firefly(validate)] pub customer: String,
    #[firefly(validate)] pub sku: String,
    #[firefly(validate)] pub quantity: u32,
}

#[derive(Clone, Serialize, Query)]
#[firefly(cache_ttl = "30s")]
pub struct GetOrder { pub id: String }

```

`#[command_handler]` / `#[query_handler]` mark a free `async fn` and generate a `register_<fn>(bus)` helper that installs it on a `Bus`:

Rust

```

#[command_handler]
pub async fn place_order(cmd: PlaceOrder) -> Result<OrderView, CqrsError>
{ /* ... */ }

#[query_handler]
pub async fn get_order(q: GetOrder) -> Result<OrderView, CqrsError> { /* ... */ }
// generated: register_place_order(&bus); register_get_order(&bus);

```

Components – dependency injection

`#[derive(Component)]` (and the `Service` / `Repository` aliases) generates a `firefly_register(container)` method; `register_all!` installs a list of them in the DI `Container`:

Rust

```
#[derive(Component, Default)]
#[firefly(scope = "singleton")]
pub struct OrderStore { /* ... */ }

let container = Container::new();
register_all!(&container, [OrderStore]);
let store = container.resolve::<OrderStore>().unwrap();
```

REST controllers

`#[rest_controller(path = "...")]` turns an `impl` block into a generated `routes(state) -> axum::Router`. Methods carry `#[get]` / `#[post]` / ... and use ordinary axum extractors, returning `WebResult<T>` so errors render as RFC 9457 problems:

Rust

```
#[rest_controller(path = "/api/v1/orders")]
impl OrderApi {
    #[post("")]
    async fn create(State(api): State<OrderApi>, Json(body): Json<PlaceOrder>)
        -> WebResult<Json<OrderView>> { /* dispatch through api.bus ... */ }

    #[get("/:id")]
    async fn fetch(State(api): State<OrderApi>, Path(id): Path<String>)
        -> WebResult<Json<OrderView>> { /* ... */ }
}

// generated: OrderApi::routes(OrderApi { bus }) -> axum::Router
```

Scheduling and events

`#[scheduled]` generates a `schedule_<fn>(scheduler)` helper, and `#[event_listener]` a `subscribe_<fn>(broker)` helper:

Rust

```
#[scheduled(fixed_rate = "60s", initial_delay = "5s")]
pub async fn sweep_stale_orders() -> Result<(), std::io::Error> { /* ... */ }

#[event_listener("orders.created")]
async fn on_order_created(ev: Event) -> FireflyResult<()> { Ok(()) }
```

How it works: the `__rt` contract

A `proc-macro` crate cannot re-export runtime types, so macro-generated code references every runtime type through the facade's hidden `__rt` contract path — e.g. `::firefly::__rt::firefly_cqrs::Bus`. That is why a service that depends only on `firefly` (plus the `axum` / `serde` it writes against anyway) compiles whatever a macro expands to without listing the underlying `firefly-*` crates. You never write `__rt` yourself. If you rename or shim the facade, pass `#[firefly(crate = "my_firefly")]` to any macro to override the leading segment.

Rust has no package scanning or reflective autowiring, so the declarative layer removes the *mechanical* boilerplate, not the explicitness: you still list the components in `register_all!` and still register the generated handlers on the bus — the macros just generate the `impl`s, the routers, and the helper functions you would otherwise hand-write.

Before / after

The `macro-quickstart` sample re-implements the `orders` reference service through the declarative layer. The same behaviour, far less code:

Measure (src/*.rs)	orders (builder)	macro-quickstart (macros)	Win
Total lines	1022	376	−63%
Code lines	587	182	−69%
Source modules	7	2	−5 files

Rust

```

// BEFORE – builder wiring: a Message impl + manual bus registration + router
impl Message for GetOrderQuery {
    fn cache_ttl(&self) -> Option<Duration> { Some(Duration::from_secs(30)) }
}

pub fn register(bus: &Bus, repo: Arc<dyn Repository>) {
    bus.register(move |q: GetOrderQuery| { /* ... */ });
}

pub fn api_router(bus: Arc<Bus>) -> Router {
    Router::new()
        .route("/api/v1/orders", post(place_order))
        .route("/api/v1/orders/:id", get(get_order))
        .with_state(bus)
}

// AFTER – macro declarations: the wiring is generated
#[derive(Clone, Serialize, Query)]
#[firefly(cache_ttl = "30s")]
pub struct GetOrder { pub id: String }

#[query_handler]
pub async fn get_order(q: GetOrder) -> Result<OrderView, CqrsError> { /* ... */ }

#[rest_controller(path = "/api/v1/orders")]
impl OrderApi {
    #[post("")] async fn create(/* ... */) -> WebResult<Json<OrderView>>
    { /* ... */ }
    #[get("/:id")] async fn fetch(/* ... */) -> WebResult<Json<OrderView>>
    { /* ... */ }
}

// register_get_order(&bus); and OrderApi::routes(state) are generated.

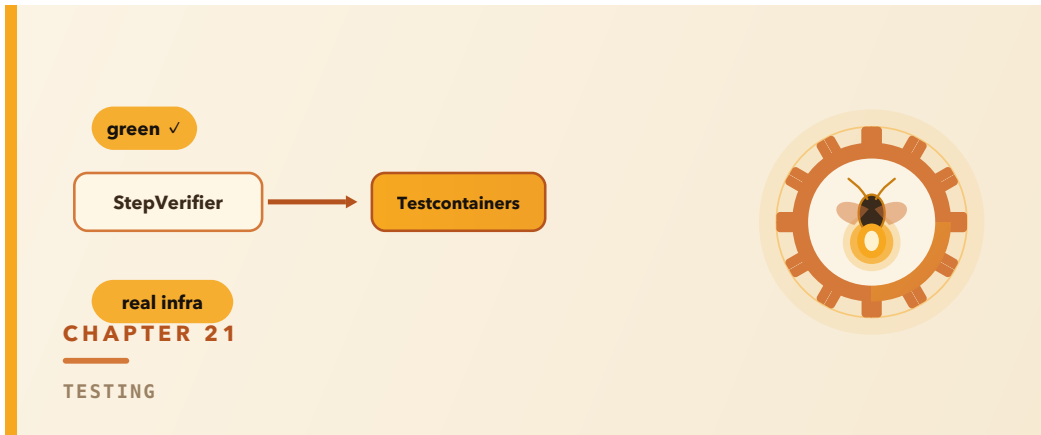
```

Run and test the sample:

Shell

```
cargo run -p firefly-sample-macro-quickstart  
cargo test -p firefly-sample-macro-quickstart
```

The tests drive the macro-generated `OrderApi::routes(...)` router in-process through `tower::ServiceExt::oneshot` (no socket bound), proving the generated routes, CQRS handlers, validation (422), the not-found path (404), and the `#[scheduled]` registration all work end to end.



CHAPTER 21

Testing Firefly Applications

Firefly services are designed to be testable at every level: in-process unit tests with no I/O, in-process integration tests that exercise the real middleware and routers via `tower::oneshot`, reactive-pipeline tests, and real-infrastructure integration tests against a `docker-compose` stack. This chapter walks each tier, with `firefly-testkit` providing the shared helpers.

Unit tests with no infrastructure

The default stack — `MemoryAdapter` cache, `InMemoryBroker` broker, `ReactiveMemoryRepository` — needs nothing external, so most logic tests run as plain `#[tokio::test]` s. Dispatch a command through a real bus, assert the result:

Rust

```
#[tokio::test]
async fn create_user_returns_id() {
    let bus = firefly_cqrs::Bus::new();
    bus.register(|c: CreateUser| async move {
        Ok:::<_, firefly_cqrs::CqrsError>(UserCreated { id: "u1".into(), name:
c.name })
    });

    let out: UserCreated = bus.send(CreateUser { name:
"alice".into() }).await.unwrap();
    assert_eq!(out.id, "u1");
}
```

In-process HTTP tests

Test the full middleware chain and your routes without binding a socket: build the router with `Core::apply_middleware` and drive it with `tower::ServiceExt::oneshot`. This exercises the real problem renderer, correlation, and idempotency layers:

Rust

```

use axum::{body::Body, http::{Request, StatusCode}};
use tower::ServiceExt;

#[tokio::test]
async fn unknown_order_is_404_problem() {
    let core = firefly_starter_core::Core::new(Default::default());
    let app = core.apply_middleware(api_router());

    let res = app
        .oneshot(Request::get("/orders/missing").body(Body::empty()).unwrap())
        .await
        .unwrap();

    assert_eq!(res.status(), StatusCode::NOT_FOUND);
    // body is application/problem+json
}

```

Asserting emitted events

`firefly-testkit`'s `SpyBroker` records what a handler published so you can assert on it. `must_encode` / `must_decode` are panic-on-failure JSON helpers (a panic fails the test, matching `t.Fatalf` semantics):

Rust

```

use firefly_testkit::{must_encode, SpyBroker};

#[derive(serde::Serialize)]
struct Order { id: String }

#[test]
fn place_order_emits() {
    let spy = SpyBroker::new();
    let place_order = |order: &Order| {
        let body = must_encode(order);
        spy.record("orders", "OrderPlaced", &body);
    };
    place_order(&Order { id: "o1".into() });
    assert_eq!(spy.find_by_type("OrderPlaced").len(), 1);
}

```

Testing webhook receivers

The testkit ships HMAC signers that match each inbound validator byte-for-byte, so a signed request validates exactly as a real Stripe / GitHub / Twilio webhook would:

Rust

```

use axum::{body::Body, http::{Request, StatusCode}};
use firefly_testkit::sign_stripe;
use tower::ServiceExt;

# fn app() -> axum::Router { axum::Router::new() }
#[tokio::test]
async fn stripe_webhook_accepts_signed_body() {
    let secret = b"whsec_test";
    let body = br#"{"type":"charge.succeeded"}"#;

    let req = Request::post("/api/webhooks/stripe")
        .header("Stripe-Signature", sign_stripe(secret, body, 1_700_000_000))
        .body(Body::from(&body[..]))
        .unwrap();

    let _res = app().oneshot(req).await.unwrap();
    // assert ACCEPTED against a real webhook router
}

```

`sign_hmac`, `sign_github`, and `sign_twilio` cover the other validators.

Testing reactive pipelines

A reactive pipeline is tested by driving it to a terminal — `block()`, `collect_list()`, or `count()` — and asserting the resolved value. This is the firefly-reactive analog of Reactor's `StepVerifier`:

Rust

```

use firefly_reactive::Flux;

#[tokio::test]
async fn pipeline_filters_and_maps() {
    let out = Flux::range(1, 5)
        .filter(|x| x % 2 == 1)
        .map(|x| x * 10)
        .collect_list()
        .block()
        .await
        .unwrap()
        .unwrap();
    assert_eq!(out, vec![10, 30, 50]);
}

```

For error paths, assert the terminal `Err`:

Rust

```

use firefly_reactive::Mono;
use firefly_kernel::FireflyError;

#[tokio::test]
async fn mono_error_propagates() {
    let result =
        Mono::<i32>::error(FireflyError::internal("boom")).block().await;
    assert!(result.is_err());
}

```

Reactive repositories are tested the same way — drive `ReactiveMemoryRepository` with `block()` / `collect_list()` (see [Persistence](#)).

Real-infrastructure integration tests

For tests that need a real Postgres, Redis, RabbitMQ, Kafka, Keycloak, S3, or SMTP, the repository ships a `docker-compose.yml` with the full stack:

YAML

```
# docker-compose.yml (excerpt)
services:
  postgres: # postgres:16-alpine           :5432
  redis:    # redis:7-alpine                :6379
  rabbitmq: # rabbitmq:3-management        :5672 / :15672
  redpanda: # Kafka-compatible             :9092
  keycloak: # quay.io/keycloak/keycloak    :8080
  localstack: # AWS S3 emulation
  azurite:   # Azure Blob emulation
  mailhog:   # SMTP capture
```

Bring it up, then run the env-gated tests against it:

Shell

```
docker compose up -d                                # start the backing services

# Each integration test reads a connection URL from the environment and SKIPS
# when it is unset, so `cargo test` on a bare machine stays green. Provide the
# URLs to exercise the real services:
DATABASE_URL=postgres://firefly:firefly@localhost:5432/firefly \
REDIS_URL=redis://localhost:6379/0 \
  cargo test --workspace -- --ignored

docker compose down                                # tear it all down
```

The pattern, in a test: read the connection URL, skip if absent, otherwise exercise the real adapter. This keeps the default `cargo test` hermetic while letting CI flip on the full real-infrastructure suite:

Rust

```
#[tokio::test]
#[ignore = "requires postgres (DATABASE_URL)"]
async fn postgres_repo_round_trips() {
    let Ok(url) = std::env::var("DATABASE_URL") else { return }; // skip on a
    bare machine

    let (client, conn) = tokio_postgres::connect(&url,
    tokio_postgres::NoTls).await.unwrap();
    tokio::spawn(async move { let _ = conn.await; });
    // ... drive PostgresReactiveRepository against the live database
}
```

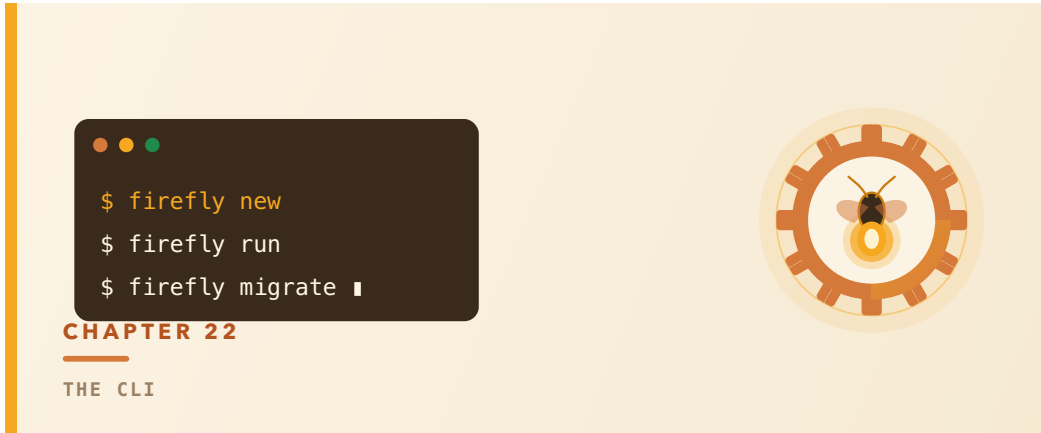
The adapter crates follow this convention throughout — `firefly-data`'s Postgres repository, `firefly-cache-redis`, the `firefly-eda-*` transports, and the IDP / ECM vendor adapters each carry live round-trip tests gated behind `#[ignore]` and an environment variable.

Running the suites

Shell

```
cargo test --workspace           # the whole 67-member workspace, hermetic
cargo test -p firefly-cqrs       # one crate
cargo test --workspace -- --ignored up) # the real-infra suite (with the stack
make ci                          # fmt-check + clippy -D warnings + build
+ test
```

With the tests green, the next chapters cover the developer CLI and shipping to production. Continue to [The CLI](#).



CHAPTER 22

The firefly CLI

The `firefly` developer CLI scaffolds projects, generates code artifacts, manages migrations, exports OpenAPI, introspects a running app's actuator, and diagnoses your toolchain. It is the Rust port of pyfly's `pyfly` CLI, adapted to a compiled Cargo workspace.

Installing

Shell

```
cargo install --path crates/cli # installs the `firefly` binary
firefly --help
```

Command overview

Command	Purpose
<code>firefly new <name></code>	scaffold a new firefly-rust project
<code>firefly generate <kind> <name> (alias <code>g</code>)</code>	generate a code artifact
<code>firefly info</code>	framework + environment information
<code>firefly doctor</code>	toolchain checks (rustc, cargo, git, ...)
<code>firefly db <init\ migrate\ upgrade\ status></code>	migration management
<code>firefly openapi --format json\ yaml [-o file]</code>	export an OpenAPI 3.1 document
<code>firefly actuator <endpoint> --url <base></code>	query a running app's <code>/actuator/*</code>
<code>firefly routes\ env\ health\ metrics --url <base></code>	remote introspection of a running app

Scaffolding a project

`firefly new` generates a workspace-less Cargo crate with a `src/` tree, a `firefly.yaml`, a `.gitignore`, a `README.md`, a `Dockerfile`, and `tests/`:

Shell

```
firefly new my-service --archetype web-api --features web,data,cqrs --git
firefly new my-lib --archetype library --dep-path ../../crates # local dev
deps
firefly new --list # archetypes +
features
firefly new svc --dry-run # plan without
writing
```

Archetypes: `core`, `web-api`, `web`, `hexagonal`, `library`, `cli`. The `firefly-*` dependencies are git / path / version configurable (`--dep-path` / `--dep-version`, defaulting to the canonical GitHub repo). `--git` initializes a repository with an initial commit; `--force` overwrites.

Generating artifacts

`firefly generate` writes a code artifact into the current project, detecting the package, archetype, and feature flags from `Cargo.toml` + `firefly.yaml`:

Shell

```
firefly generate handler Order
firefly generate entity Product      # data-aware when relational data is
enabled
firefly generate command OpenWallet  # command + handler in src/cqrs/
firefly generate migration AddUsers   # V###__add_users.sql in migrations/
firefly g saga MoneyTransfer --dry-run
```

Artifact kinds: `handler`, `route`, `entity`, `repository`, `dto`, `aggregate`, `command`, `query`, `saga`, `migration`. Names are accepted in any case and converted as needed; `--force` overwrites, `--dry-run` plans.

Database migrations

`firefly db` wraps the `firefly-migrations` forward-only runner:

Shell

```
firefly db init                    # migrations/ + starter V001__init.sql
firefly db migrate -m "create users" # writes V002__create_users.sql
firefly db upgrade --url sqlite://app.db # apply pending migrations
firefly db status --url sqlite://app.db # show applied + pending
```

The database URL resolves from `--url`, then `$DATABASE_URL`, then `firefly.datasource.url` in `firefly.yaml`, defaulting to `sqlite://firefly.db`.

NOTE

The CLI backend is **SQLite** via `rusqlite`; a `postgres://` / `mysql://` URL returns a clear "not wired into the CLI" error — adapt the `firefly_migrations::Database` port to your driver and call `run` directly. The runner is forward-only, so `db downgrade` is unsupported (write a corrective migration instead).

Exporting OpenAPI

Shell

```
firefly openapi                                # OpenAPI 3.1 JSON to stdout
firefly openapi --format yaml -o openapi.yaml
```

The document metadata (`info.title` / `info.version` / `description`) is read from `firefly.yaml` then `Cargo.toml`. A compiled binary cannot boot an arbitrary app to enumerate live routes, so the CLI emits a metadata-stamped **skeleton**; to emit real routes, build them with `firefly_openapi::Builder` and serve them with `Builder::router()`.

Introspecting a running app

These commands query a running app's actuator over HTTP — a compiled binary has no offline DI context to boot, so `--url` is required:

Shell

```

firefly actuator health --url http://localhost:8080
firefly actuator metrics requests --url http://localhost:8080 --json
firefly routes --url http://localhost:8080 # -> /actuator/mappings
firefly env --url http://localhost:8080 # -> /actuator/env
firefly health --url http://localhost:8080 # -> /actuator/health
firefly metrics requests --url http://localhost:8080

```

Diagnosing your environment

Shell

```

firefly info      # framework + environment information
firefly doctor    # checks rustc, cargo, git, clippy, rustfmt, docker

```

Running through cargo

If you have not installed the binary, drive the CLI through cargo from a framework checkout:

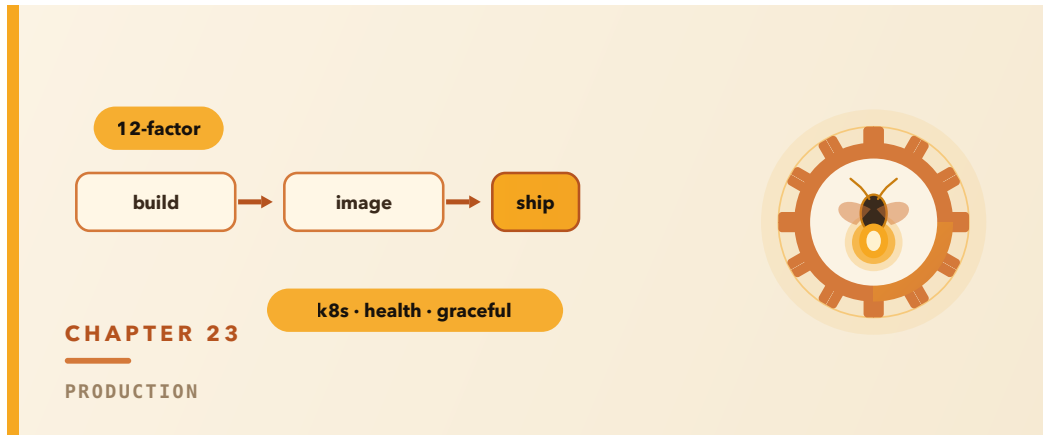
Shell

```

make cli ARGS="doctor"
make cli ARGS="new orders --archetype web-api"
cargo run -p firefly-cli --bin firefly -- info

```

With a project scaffolded and code generated, the last chapter takes it to production. Continue to [Production & Deployment](#).



CHAPTER 23

Extending Firefly & Going to Production

This chapter covers everything between "it works on my machine" and a service running reliably in production: the lifecycle and graceful shutdown, the management surface, TLS, the optional hardening middleware, container packaging, and a deployment checklist.

The lifecycle and graceful shutdown

`firefly-lifecycle`'s `Application` is the Rust analog of `SpringApplication.run()`: it traps `SIGINT/SIGTERM`, runs each server task with its own drain signal, and grants a drain budget (default 30 s) before exiting. `Core::new_application()` gives you one named after your app.

Rust

```

use firefly_starter_core::{Core, CoreConfig};

# async fn ex() -> Result<(), Box<dyn std::error::Error>> {
let core = Core::new(CoreConfig { app_name:
"orders".into(), ..Default::default() });

let app = core
    .new_application()
    .on_server("api", move |shutdown| async move {
        let l = tokio::net::TcpListener::bind("0.0.0.0:8080").await?;
        axum::serve(l, build_api())
            .with_graceful_shutdown(shutdown.wait()) // drain on signal
            .await?;
        Ok(())
    })
    .on_server("admin", move |shutdown| async move {
        let l = tokio::net::TcpListener::bind("0.0.0.0:8081").await?;
        axum::serve(l, build_admin())
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    });

app.run().await?; // blocks until ctrl-c / SIGTERM, then drains
# Ok(())
# }
# fn build_api() -> axum::Router { axum::Router::new() }
# fn build_admin() -> axum::Router { axum::Router::new() }

```

Multiple servers are allowed (each gets its own task and drain signal), and `on_start` / `on_stop` hooks let you run setup and teardown. A `ShutdownHandle` (`app.shutdown_handle()`) triggers shutdown programmatically, so integration tests never need to send a real signal.

The management surface on a separate port

Always mount `core.actuator_router(..)` on a **different listener** from your public API so `/actuator/*` (health, info, metrics, env, threaddump, ...) is reachable by your orchestrator and ops tooling but never exposed to the public network. The Quickstart and the chapters above use `:8080` for the API and `:8081` for the actuator — keep that split in production and firewall the admin port.

Health endpoints feed your orchestrator's probes:

Probe	Endpoint
liveness	<code>/actuator/health/liveness</code>
readiness	<code>/actuator/health/readiness</code>
overall	<code>/actuator/health</code>

Production hardening middleware

The pyfly-parity middleware is OFF by default; turn on what you need via `CoreConfig` and it weaves into `apply_middleware` at the correct filter order:

Rust

```

use firefly_starter_core::{Core, CoreConfig};
use firefly_web::{CorsConfig, SecurityHeadersConfig};

let core = Core::new(CoreConfig {
    app_name: "orders".into(),
    cors: Some(CorsConfig::permit_defaults()),           // CORS at the edge
    security_headers: Some(SecurityHeadersConfig::default()), // OWASP headers
    request_log: Some(Default::default()),              // one access-log event/
    request
        request_metrics:
Some(Default::default()),          // http_server_requests_* metrics
    ..CoreConfig::default()
});

```

Knob	Adds
<code>cors</code>	<code>CorsLayer</code> (preflight + simple-request decoration)
<code>security_headers</code>	OWASP response headers (<code>nosniff</code> , <code>DENY</code> , HSTS, ...)
<code>csrf</code>	double-submit-cookie CSRF
<code>request_log</code>	one structured access-log event per request
<code>request_metrics</code>	<code>http_server_requests_seconds</code> + <code>_max</code> (actuator)
<code>http_exchanges</code>	recent-exchange recorder + <code>/actuator/httpexchanges</code>
<code>loggers</code>	<code>/actuator/loggers</code> runtime log-level control
<code>redaction</code>	PII scrubbing on the log writer

The effective chain (outermost → innermost) is CORS → Problem → SecurityHeaders → Correlation → Metrics → HttpExchanges → RequestLog → CSRF → Idempotency → your router. Idempotency always stays innermost so a replayed request still passes through every outer concern.

TLS

`firefly-web`'s `Server` terminates TLS via `axum-server` + `rustls`. Bind under `ServerProperties` with a `TlsConfig`:

Rust

```
use firefly_web::{Server, ServerProperties, TlsConfig};

let props = ServerProperties {
    host: "0.0.0.0".into(),
    port: 8443,
    tls: Some(TlsConfig { cert_file: "tls/cert.pem".into(), key_file: "tls/
key.pem".into() }),
    ..Default::default()
};
// Server::serve(router, props, shutdown) honours the lifecycle drain.
```

Most deployments terminate TLS at an ingress/load balancer and run the service plain-HTTP behind it; the built-in TLS is there for when you need end-to-end encryption to the process.

Configuration in production

Bind configuration from layered sources with environment overrides on top, so a container reads its settings from the environment (see [Configuration](#)):

Shell

```
FIREFLY_PROFILE=prod \
FIREFLY_WEB_PORT=8080 \
DATABASE_URL=postgres://... \
./orders
```

`FIREFLY_*` variables beat the YAML files, secrets are masked in `/actuator/env`, and `${...}` placeholders resolve env-then-config-then-default. For dynamic reconfiguration, wire `ReloadableConfig` to `POST /actuator/refresh`.

Container packaging

`firefly new` generates a `Dockerfile`. A typical multi-stage build:

Dockerfile

```
FROM rust:1.85 AS build
WORKDIR /app
COPY . .
RUN cargo build --release

FROM debian:bookworm-slim
COPY --from=build /app/target/release/orders /usr/local/bin/orders
EXPOSE 8080 8081
ENTRYPOINT ["/usr/local/bin/orders"]
```

Because the framework traps SIGTERM and drains, the container stops cleanly when the orchestrator sends a termination signal — no `--init` shim or signal-forwarding wrapper is required.

Selecting production adapters

Swap the in-process defaults for real infrastructure at the composition root — nothing else changes:

Rust

```

use std::sync::Arc;
use firefly_starter_core::{Core, CoreConfig};

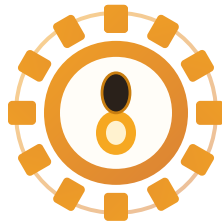
let core = Core::new(CoreConfig {
    app_name: "orders".into(),
    cache: Some(Arc::new(redis_adapter)), // firefly-cache-redis
    broker: Some(Arc::new(kafka_broker)), // firefly-eda-kafka
    ..CoreConfig::default()
});

```

A deployment checklist

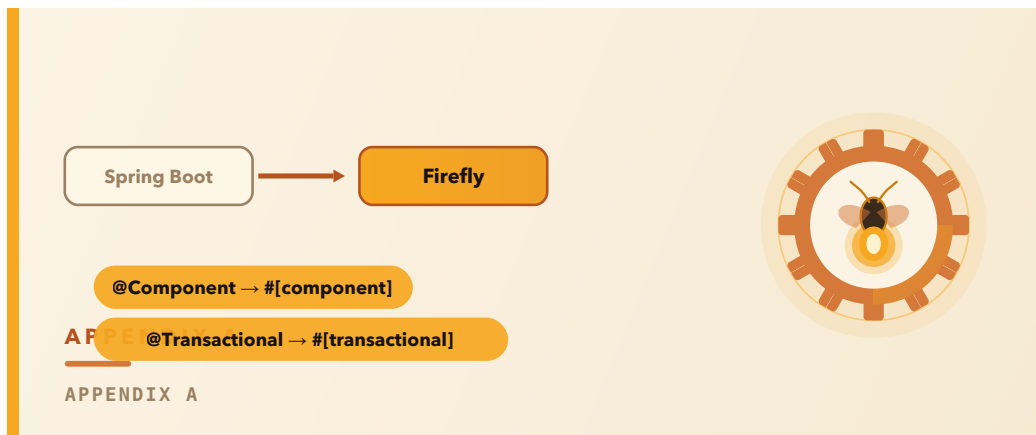
- [] Actuator on a **separate, firewalled** port from the public API.
- [] Liveness/readiness probes pointed at `/actuator/health/{liveness,readiness}`.
- [] `security_headers`, `cors`, and (for browser flows) `csrf` enabled.
- [] `request_log` + `request_metrics` on, logs shipped as JSON, metrics scraped.
- [] Correlation propagation verified end-to-end across services.
- [] Idempotency store moved off in-memory to Redis/Postgres for multi-replica.
- [] TLS terminated (at ingress or in-process), secrets injected via environment.
- [] Graceful shutdown drain budget tuned for your slowest in-flight request.
- [] Real cache/broker/IDP adapters wired; the `make ci` gate green.

That completes the guided tour. The appendices follow: a [Spring Boot migration map](#), the full [Module Index](#), and the [Glossary](#).



Appendices





APPENDIX A

Spring Boot → Firefly Cheat-Sheet

If you are porting a Firefly Java/Spring Boot service to Rust — or simply come from a Spring background — this appendix is your translation table. Each Firefly Rust crate is wire-shape compatible with its Java counterpart but expressed in idiomatic Rust: explicit construction, `tower` middleware composition, and `async` / `await`.

Module mapping

Java / Spring	Rust crate
<code>firefly-common</code>	<code>firefly-kernel</code>
<code>firefly-web</code> + <code>firefly-spring-utils</code>	<code>firefly-web</code>
<code>firefly-common-cache</code>	<code>firefly-cache</code>
<code>firefly-otel-spring-boot-starter</code>	<code>firefly-observability</code>
<code>firefly-common-data</code>	<code>firefly-data</code>
<code>firefly-common-cqrs</code>	<code>firefly-cqrs</code>
<code>firefly-common-eda</code>	<code>firefly-eda</code>
<code>firefly-event-sourcing-...</code>	<code>firefly-eventsourcing</code>
<code>firefly-common-domain</code> (orchestration)	<code>firefly-orchestration</code>
<code>firefly-service-client</code>	<code>firefly-client</code>
Spring WebFlux (Netty)	<code>axum</code> on <code>tokio</code> , <code>tower::Layer s</code>
Reactor <code>Mono</code> / <code>Flux</code>	<code>firefly_reactive::</code> { <code>Mono</code> , <code>Flux</code> }
<code>@ConfigurationProperties</code>	<code>firefly-config</code>
<code>SpringApplication</code>	<code>firefly-lifecycle</code>
<code>spring-boot-starter-actuator</code>	<code>firefly-actuator</code>
<code>@Scheduled</code>	<code>firefly-scheduling</code>
<code>resilience4j-*</code>	<code>firefly-resilience</code>
<code>spring-security</code>	<code>firefly-security</code>
Flyway	<code>firefly-migrations</code>
<code>springdoc-openapi</code>	<code>firefly-openapi</code>

Java / Spring	Rust crate
<code>MessageSource</code>	<code>firefly-i18n</code>
<code>ServerSentEvent</code>	<code>firefly-sse</code>
<code>@Transactional</code>	<code>firefly-transactional</code>
<code>spring-boot-starter-test</code>	<code>firefly-testkit</code>
<code>@Component</code> / <code>@Autowired</code> DI	<code>firefly-container</code> (opt-in)
<code>@Aspect</code> / Spring AOP	<code>firefly-aop</code>
<code>HttpSession</code> / Spring Session	<code>firefly-session</code>
Spring Shell <code>@ShellMethod</code>	<code>firefly-shell</code>
Spring WebSocket / <code>@ServerEndpoint</code>	<code>firefly-websocket</code>
Spring Boot Admin	<code>firefly-admin</code>
<code>@KafkaListener</code>	<code>firefly-eda-kafka</code>
Spring AMQP (RabbitMQ)	<code>firefly-eda-rabbitmq</code>
<code>spring-data-redis</code> cache	<code>firefly-cache-redis</code>
<code>JavaMailSender</code> (SMTP)	<code>firefly-notifications-smtp</code>

Reactive types

The Java framework uses Project Reactor — and so does the Rust port. `firefly-reactive` is a faithful `Mono` / `Flux` reimplementation, so a Reactor operator chain maps almost one-to-one (see [The Reactive Model](#)):

Project Reactor	firefly-reactive
<code>Mono<T></code>	<code>Mono<T></code>
<code>Flux<T></code>	<code>Flux<T></code>
<code>Throwable</code> (error signal)	<code>firefly_kernel::FireflyError</code> (fixed)
<code>Mono.empty()</code>	<code>Ok(None)</code> from a <code>Mono</code>
<code>Mono.error(new NotFound())</code>	<code>Mono::error(FireflyError::not_found("..."))</code>
<code>Schedulers.parallel()</code>	<code>Scheduler::Parallel</code>
<code>subscribeOn</code> / <code>publishOn</code>	<code>subscribe_on</code> / <code>publish_on</code>
<code>Mono.timeout(d)</code>	<code>Mono::timeout(d)</code>
<code>Flux.onBackpressureBuffer()</code>	<code>Flux::on_backpressure_buffer(n)</code>
<code>Retry.backoff(..)</code>	<code>Backoff</code> + <code>Mono::retry_backoff</code>
<code>Mono.toFuture()</code> / <code>await</code>	<code>Mono::into_future</code> / <code>.await</code>
<code>Flux.toStream()</code>	<code>Flux::to_stream</code>

NOTE

Ambient request metadata that Reactor carries in the subscriber context (`deferContextual`) travels in Rust task-local scopes instead — `firefly_kernel::correlation_id()`, `request_id()`, `tenant_id()` — set by the `CorrelationLayer` HTTP middleware. Cancellation, which Reactor models as subscription disposal, is future-drop in Rust (and a `CancellationToken` for the cooperative orchestration engines).

Programming-model mapping

Spring concept	Rust entry point
<code>@SpringBootApplication</code> auto-config	<code>Core::new(CoreConfig { .. })</code>
<code>SpringApplication.run()</code>	<code>core.new_application().on_server(..).run().await</code>
<code>@Component</code> / <code>@Autowired</code>	explicit <code>Arc<dyn Trait></code> injection; opt-in <code>Container</code> for a locator
<code>@RestController</code>	an axum handler mounted on <code>core.apply_middleware(router)</code>
<code>@RestController</code> returning <code>Mono<T></code>	a handler returning <code>MonoJson(Mono<T>)</code>
<code>@RestController</code> returning <code>Flux<T></code> (NDJSON)	a handler returning <code>NdJson(Flux<T>)</code> / <code>Sse(Flux<T>)</code>
<code>@ControllerAdvice</code> <code>ProblemDetail</code> handler	<code>ProblemLayer</code> + <code>WebResult<T></code> (? renders RFC 7807)
<code>@CommandHandler</code> / <code>@QueryHandler</code>	<code>bus.register(\ c: Cmd\ async move { .. })</code>
<code>@Cacheable</code>	<code>Typed::get_or_set(key, ttl, loader)</code>
<code>@Scheduled(cron = "0 9 * * MON-FRI")</code>	<code>scheduler.cron("name", "0 9 * * 1-5", run)</code>
<code>@CircuitBreaker</code> <code>@RateLimiter</code> <code>@Bulkhead</code>	<code>Chain::new().with(Timeout::...).with(CircuitBreaker::...)</code>
<code>@PreAuthorize("hasRole('ADMIN')")</code>	<code>FilterChain::new().require("/admin/", &["ADMIN"]) + BearerLayer</code>
R2DBC <code>ReactiveCrudRepository<T, ID></code>	<code>firefly_data::ReactiveCrudRepository<T, ID></code>
<code>Pageable</code> / <code>Sort</code> / <code>PageRequest</code>	<code>firefly_data::{Pageable, Sort, Order} + Page<T></code>
Axon / event-sourced aggregates	<code>firefly_eventsourcing</code> (<code>AggregateRoot</code> , <code>EventStore</code>)
Temporal / Camunda workflows	<code>firefly_orchestration::Workflow</code> (DAG + compensation)

Spring concept	Rust entry point
<code>@Transactional</code>	<code>with_tx(ctx, &db, \ ctx\ { .. })</code>
Flyway <code>V001__init.sql</code>	<code>firefly_migrations::run</code> over a <code>Database</code> port
<code>WebClient</code>	<code>firefly_client::WebClientBuilder</code> → <code>body_to_mono</code> / <code>body_to_flux</code>
<code>RestTemplate</code> / <code>RestClient</code>	<code>firefly_client::RestBuilder</code> → <code>client.request(..)</code>
<code>MessageSource.getMessage(...)</code>	<code>bundle.t(locale, key, args)</code>
<code>@SpringBootTest</code>	<code>tower::ServiceExt::oneshot</code> + <code>firefly_testkit</code>

What changes, and what stays the same

Stays the same — the wire contracts. The `application/problem+json` shape, the `Idempotency-Key` semantics, the event envelope JSON, the saga step definitions, the HMAC webhook signatures, the `Page<T>` JSON, the `Authentication` claims mapping, and the configuration keys are byte-identical to the Java release line. A Java service and a Rust service interoperate on the wire with no adapter.

Changes — the wiring style. Spring's reflective DI and annotation scanning become explicit construction (`Core::new`, constructor injection, optional `Container`). Annotation-driven aspects become explicit weaving at the call site. There is no leading `ctx` parameter and no XML — ambient state lives in task-locals, and dependencies are `Arc<dyn Trait>` fields the compiler checks.

A worked port

A Spring WebFlux controller:

Java

```

@RestController
class OrderController {
    @GetMapping("/orders/{id}")
    Mono<Order> get(@PathVariable String id) {
        return service.find(id)
            .switchIfEmpty(Mono.error(new ResourceNotFound("order " + id)));
    }
}

```

becomes a Firefly Rust handler:

Rust

```

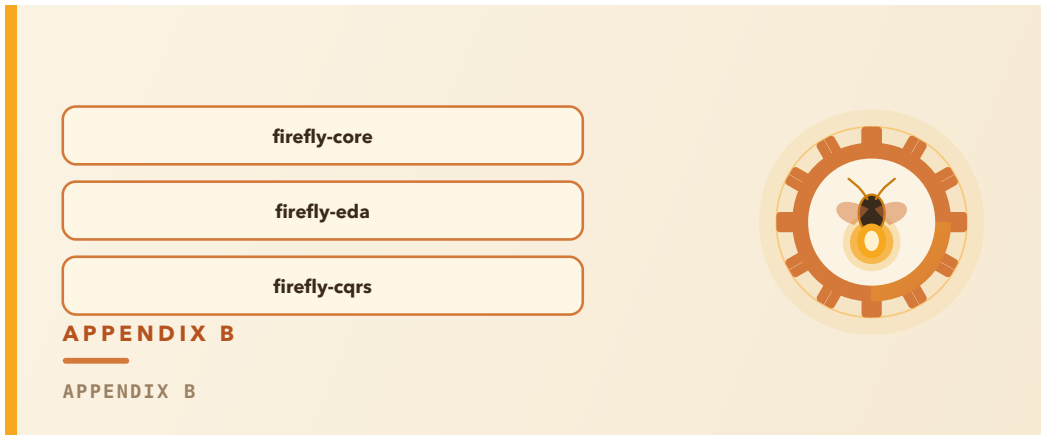
use axum::extract::Path;
use firefly_reactive::Mono;
use firefly_web::MonoJson;

async fn get_order(Path(id): Path<String>) -> MonoJson<Order> {
    // Mono::empty() (Ok(None)) renders a 404 problem automatically – no
    // switchIfEmpty needed; an Err(FireflyError) renders that error's problem.
    MonoJson(service.find(&id))
}

```

The `Mono<T>` model carries over directly; the empty-Mono → 404 behaviour is built into the `MonoJson` responder, so the `switchIfEmpty` boilerplate disappears.

For the full crate catalogue, see the [Module Index](#).



APPENDIX B

Crate & Module Index

The Firefly Rust workspace ships **76 members** — 72 framework crates under `crates/`, plus the cross-crate integration suite and three samples (`orders`, `reactive-banking`, `macro-quickstart`). Each crate carries its own `README.md` with its full public surface, design rationale, and a runnable quick-start.

The canonical, always-current catalogue is `MODULES.md` in the repository root. This appendix reproduces it tier-by-tier as a navigable index.

Front door

Crate	What it provides
<code>firefly</code>	The one-dependency facade: <code>use firefly::prelude::*;</code> , per-crate aliases, the <code>__rt</code> macro contract, feature-gated adapters
<code>firefly-macros</code>	Declarative macros: <code>#[derive(Command/Query/Component/DomainEvent/AggregateRoot)]</code> , <code>#[command_handler]</code> / <code>#[query_handler]</code> , <code>#[scheduled]</code> , <code>#[rest_controller]</code> + verbs, <code>#[event_listener]</code>

Foundational

Crate	What it provides
<code>firefly-kernel</code>	RFC 7807 <code>ProblemDetail</code> , <code>FireflyResult<T></code> , <code>Clock</code> , the <code>FireflyError</code> hierarchy, task-local correlation/request/tenant scopes, the <code>ddd</code> kit (<code>Entity</code> / <code>Specification</code> / domain events)
<code>firefly-reactive</code>	The <code>Mono<T></code> / <code>Flux<T></code> reactive core — the Project Reactor analog and keystone of every reactive surface
<code>firefly-utils</code>	Try helpers, retry with exponential backoff, slug, AES-256-GCM, templates
<code>firefly-validators</code>	IBAN, BIC, Luhn, currency, phone, password, VAT, national/tax IDs
<code>firefly-web</code>	Problem renderer, correlation, idempotency, PII masking, CORS, CSRF, security headers, server metrics, the reactive <code>MonoJson</code> / <code>NdJson</code> / <code>Sse</code> responders, TLS bootstrap
<code>firefly-config</code>	Typed YAML+env+flag binding, profiles, <code>\${...}</code> placeholders, runtime reload, masked property sources, <code>ApplicationEventBus</code> , config-server client
<code>firefly-i18n</code>	Locale-aware message <code>Bundle</code> + Accept-Language picker
<code>firefly-session</code>	Server-side HTTP sessions + <code>SessionStore</code> + <code>SessionLayer</code>

Platform

Crate	What it provides
<code>firefly-cache</code>	<code>Adapter</code> port + Memory/NoOp/Fallback + typed <code>Typed<T></code> memoisation
<code>firefly-observability</code>	<code>tracing</code> + correlation enrichment, health composite, startup banner, W3C trace-context, metrics
<code>firefly-data</code>	Storage-agnostic ports: Filter DSL + <code>Specification</code> , <code>SqlDialect</code> (pg/mysql/sqlite) + <code>Specification::to_mongo()</code> , <code>Page<T></code> , <code>Repository<T, K></code> , <code>ReactiveCrudRepository</code> , <code>PostgresReactiveRepository</code> , auditing + soft-delete, derived queries, paging
<code>firefly-cqrs</code>	Command/query <code>Bus</code> with validation/cache/authorization middleware + reactive <code>send_mono</code> / <code>query_mono</code>
<code>firefly-eda</code>	<code>Event</code> envelope, <code>Publisher</code> / <code>Subscriber</code> / <code>Broker</code> ports, <code>InMemoryBroker</code> , glob topics, consumer groups, retry/DLQ, reactive <code>Flux</code> subscriptions
<code>firefly-eventsourcing</code>	Aggregate roots, event store, snapshots, projections, global stream, transactional outbox, multi-tenancy
<code>firefly-orchestration</code>	<code>Saga</code> , <code>Workflow</code> (DAG), <code>Tcc</code> — compensation, per-step retry, <code>StepContext</code>
<code>firefly-rule-engine</code>	YAML DSL → AST → evaluator with <code>between</code> / null / <code>regex</code> , <code>EvaluationMode</code> , validator + <code>ActionHandler</code>
<code>firefly-plugins</code>	Lifecycle SPI + composite registry
<code>firefly-lifecycle</code>	<code>Application::run()</code> orchestrator with signal trap + drain
<code>firefly-actuator</code>	<code>/actuator/{health,info,metrics,env,tasks,version}</code> + probes, loggers, httpexchanges, threaddump, refresh
<code>firefly-scheduling</code>	Cron + FixedRate + FixedDelay <code>Scheduler</code> with zones

Crate	What it provides
<code>firefly-resilience</code>	<code>CircuitBreaker</code> , <code>RateLimiter</code> , <code>Bulkhead</code> , <code>Timeout</code> , composable <code>Chain</code>
<code>firefly-security</code>	<code>BearerLayer</code> , <code>RBAC</code> , <code>FilterChain</code> , <code>JwksVerifier</code> , <code>oauth2</code> , <code>RoleHierarchy</code> , <code>CsrfLayer</code> , <code>BcryptPasswordEncoder</code>
<code>firefly-migrations</code>	Versioned forward-only SQL migrations over a <code>Database</code> port
<code>firefly-openapi</code>	OpenAPI 3.1 generator + Swagger-UI shim
<code>firefly-sse</code>	Server-Sent Events writer with heartbeat + Last-Event-Id
<code>firefly-transactional</code>	<code>with_tx(ctx, db, f)</code> over pluggable <code>Database</code> / <code>Transaction</code> ports
<code>firefly-testkit</code>	HMAC signers, <code>SpyBroker</code> , JSON test helpers
<code>firefly-aop</code>	Spring-style aspect advice — <code>Pointcut</code> , <code>JoinPoint</code> , <code>Aspect</code> , <code>intercept</code>
<code>firefly-shell</code>	Spring-Shell-style CLI framework + <code>CommandLineRunner</code> / <code>ApplicationRunner</code>
<code>firefly-websocket</code>	WebSocket server over axum + topic <code>BroadcastHub</code>

Adapters

Crate	Port → backend
<code>firefly-client</code>	REST <code>RestClient</code> + reactive <code>WebClient</code> + SOAP/gRPC/GraphQL/WS
<code>firefly-config-server</code>	Spring-Cloud-Config-compatible REST endpoint
<code>firefly-idp</code> + <code>idp-internal-db</code> / <code>idp-keycloak</code> / <code>idp-azure-ad</code> / <code>idp-aws-cognito</code>	Identity providers
<code>firefly-ecm</code> + <code>ecm-storage-aws</code> / <code>ecm-storage-azure</code> / <code>ecm-esignature-*</code>	Content management + e-signature
<code>firefly-notifications</code> + <code>notifications-smtp</code> / <code>-twilio</code> / <code>-firebase</code> / <code>-sendgrid</code> / <code>-resend</code>	Notification channels
<code>firefly-callbacks</code>	Outbound webhook subsystem (HMAC dispatcher + audit + admin)
<code>firefly-webhooks</code>	Inbound ingestion (Stripe / GitHub / Twilio / generic HMAC + DLQ)
<code>firefly-data-sqlx</code>	<code>firefly_data</code> ports → relational (Postgres / MySQL / SQLite over <code>sqlx</code>)
<code>firefly-data-mongodb</code>	<code>firefly_data</code> ports → document store (MongoDB)
<code>firefly-cache-redis</code> / <code>firefly-cache-postgres</code>	<code>cache::Adapter</code> → Redis (RESP) / Postgres key-value table
<code>firefly-eda-kafka</code> / <code>-rabbitmq</code> / <code>-postgres</code> / <code>-redis</code>	<code>eda::Broker</code> → Kafka / RabbitMQ / Postgres outbox / Redis Streams
<code>firefly-session-redis</code> / <code>firefly-session-postgres</code>	distributed <code>SessionRegistry</code> → Redis sorted set / Postgres table

Starters

Crate	What it bundles
<code>firefly-starter-core</code>	web + cache + observability + eda + cqrs + actuator + lifecycle + scheduling
<code>firefly-starter-application</code>	starter-core + plugins registry
<code>firefly-starter-domain</code>	starter-core + in-memory event-sourcing stores
<code>firefly-starter-data</code>	starter-core (you supply the DB)
<code>firefly-starter-web</code>	<code>WebStack</code> — <code>Core</code> + CORS + security headers + request metrics + access log
<code>firefly-backoffice</code>	starter-application + back-office context middleware

DI / Operations / Tooling

Crate	What it provides
<code>firefly-container</code>	Opt-in <code>TypeId</code> -keyed DI container (service locator)
<code>firefly-admin</code>	Spring-Boot-Admin-style embedded dashboard + JSON API + SSE
<code>firefly-cli</code>	The <code>firefly</code> developer binary (<code>new</code> / <code>generate</code> / <code>db</code> / <code>openapi</code> / <code>actuator</code> / <code>doctor</code> / <code>completion</code> / <code>sbom</code> / <code>license</code>)

For per-crate detail, open the crate's `README.md` in the [repository](#), or read the `MODULES.md` catalogue.

Glossary

Definitions of the terms and types that recur throughout this book, in the precise sense Firefly uses them.

Actuator

The management surface (`firefly-actuator`) exposing the endpoints `/actuator/health` , `/actuator/info` , `/actuator/metrics` , `/actuator/env` , `/actuator/tasks` , `/actuator/version` , `/actuator/loggers` , `/actuator/httpexchanges` , and `/actuator/refresh` . Mounted by `core.actuator_router(..)` , typically on a separate, firewalled port.

Adapter

A concrete implementation of a **port**. Selected at wiring time as an `Arc<dyn Port>` so heavy SDK dependencies stay out of services that do not use them. Examples:

`RedisAdapter` (a `cache::Adapter`), `KafkaBroker` (an `eda::Broker`).

Aggregate root

The consistency boundary in DDD. The non-event-sourced variant holds a `PendingEvents<E>` buffer (`firefly_kernel::ddd`); the event-sourced variant is an `AggregateRoot` whose state is rebuilt by replaying its `DomainEvent` s (`firefly-eventsourcing`).

Backpressure

A slow consumer throttling a fast producer. Firefly's reactive `Flux` streams (NDJSON/SSE responders, `WebClient::body_to_flux` , Postgres row streams) honour backpressure end-to-end so a large stream never lands fully in memory.

Bus

The CQRS command/query dispatcher (`firefly_cqrs::Bus`). Handlers register by input type; dispatch is keyed on `std::any::TypeId` . `send` / `query` are async; `send_mono` / `query_mono` are the reactive twins.

CompensationPolicy

How a saga/workflow rolls back on failure: `BestEffort` (continue compensating even if one compensation fails) or `StopOnError` (abort at the first compensation failure).

Core

The wired infrastructure bundle returned by `Core::new(CoreConfig)` (`firefly-starter-core`): cache, CQRS bus, event broker, health composite, metrics, scheduler, logging, and the middleware chain.

Correlation id

A per-request identifier carried in the `X-Correlation-Id` header and a kernel task-local scope. It auto-enriches every log line, every published event, and every outbound client call so a request stitches together across services.

DomainEvent

The event-sourced, versioned, wire-formatted event in `firefly-eventsourcing` (distinct from the transient `TransientDomainEvent` in `firefly_kernel::ddd`). Its JSON is byte-compatible across the ports.

Event (EDA)

The envelope every `firefly-eda` event flows through — `id` , `type` , `source` , `topic` , `correlationId` , `time` , `headers` , `payload` , `key` . Constructed with `Event::new` , wire-compatible across the ports.

FireflyError

The framework's error type (`firefly_kernel::FireflyError`). It renders as an RFC 7807 `application/problem+json` response, and it is the fixed error channel of the reactive `Mono` / `Flux` (their terminal `Err` signal).

FilterChain

The path-based authorization matcher in `firefly-security` (`permit` / `require` / glob `permit_pattern` / `require_pattern`). Fail-closed once any rule is declared (Spring Security 6 deny-by-default).

Flux

A reactive publisher of $0..N$ values plus a terminal completion-or-error (`firefly_reactive::Flux`). The Rust analog of Reactor's `Flux<T>` .

Idempotency

The replay behaviour applied to `POST` / `PUT` / `PATCH` requests carrying an `Idempotency-Key` header. A repeat replays the stored response (`Idempotent-Replay: true`); reuse with a different body is a 409.

Mono

A reactive publisher of *at most one* value plus a terminal error (`firefly_reactive::Mono`). The Rust analog of Reactor's `Mono<T>` . An empty `Mono` (`Ok(None)`) is the equivalent of `Mono.empty()` .

NDJSON

Newline-delimited JSON (`application/x-ndjson`) — one compact JSON document per line. The `NdJson(Flux<T>)` responder streams it with backpressure.

Outbox (transactional)

A pattern (`TransactionalOutbox` , `firefly-eda-postgres`) that writes events in the same transaction as the state change and delivers them to consumers afterward, giving at-least-once delivery without a separate broker.

Port

An object-safe `async_trait` trait defining an integration point — `cache::Adapter` , `eda::Broker` , `notifications::Channel` , `idp::Adapter` . Code depends on the port; an `adapter` implements it.

Problem (RFC 7807)

The `application/problem+json` error envelope (`type` , `title` , `status` , `detail`) that every Firefly service renders for errors and panics, identical across the ports.

Projection

A read-side handler that builds a read model from events (`firefly_event sourcing::Projection`), driven per-aggregate (`replay`) or over the global stream (`drive_once` / `replay_all`).

Reactive

The `Mono` / `Flux` programming model (`firefly-reactive`) and everything built on it — reactive endpoints, repositories, the `WebClient` , reactive EDA/CQRS. The Rust analog of Project Reactor / Spring WebFlux.

Saga

A sequential distributed-transaction engine (`firefly_orchestration::Saga`) with reverse-order compensation on failure. See also `Workflow` (DAG) and `Tcc` .

Scheduler

The task runner (`firefly_scheduling::Scheduler`) owning Cron, FixedRate, and FixedDelay triggers, each on its own tokio task with panic recovery.

SSE (Server-Sent Events)

A one-way streaming protocol (`text/event-stream`). The `Sse(Flux<T>)` responder and `firefly-sse`'s `SseWriter` emit it; `WebClient::body_to_flux` decodes it.

Specification

A composable business-rule predicate (`firefly_kernel::ddd::Specification<T>`) combined with `.and()`, `.or()`, `.not()`. Any `Fn(&T) -> bool` is one.

Starter

A crate that bundles a sensible default stack so a service depends on one crate. `firefly-starter-core` is the common starting point.

TCC (Try-Confirm-Cancel)

A two-phase distributed-transaction engine (`firefly_orchestration::Tcc`): Try all participants, Confirm all on success, Cancel the tried participants on any Try failure.

Verifier

The async port (`firefly_security::Verifier`) that validates a bearer token and returns an `Authentication`. `JwksVerifier`, IDP adapters, and `VerifierFn` closures all satisfy it.

WebClient

The reactive HTTP client (`firefly_client::WebClient`) whose terminal operators return `Mono` / `Flux` (`body_to_mono`, `body_to_flux`, `exchange`). The Rust analog of WebFlux's `WebClient`.

Workflow

A DAG distributed-transaction engine (`firefly_orchestration::Workflow`): independent nodes run concurrently within a wave, with reverse-order compensation under a configurable `CompensationPolicy`.